

Agent Harness：实践者教材

这是一份基于事实的 agent harness 工程综述，主要整理自 [Awesome Harness Engineering](#) 阅读列表和 [OpenReview](#) 综述论文 [Agent Harness Engineering: A Survey](#)。正文中的每个关键论断都在相应位置给出行内引用。

22 sections · generated from Markdown sources

前言

本书讨论的是：当语言模型真正开始执行任务时，围绕它运行的整套系统。这套系统如今有了一个名字——*harness*，如何构建它也已经形成一批规模不大、却在迅速成熟的研究与实践。接下来的章节会把这些资料串联成一条完整的脉络，并在每项论断旁标明原始出处，方便读者继续追溯。

这个领域建立在一个简单的前提之上。LangChain 的 Vivek Trivedy 将其概括为：“Agent = Model + Harness。如果你不是模型，那你就是 *harness*。”(LangChain - The Anatomy of an Agent Harness)。按照这一框架，模型周围的一切——系统提示、工具、沙箱、记忆、子代理、控制流和评估基础设施——都属于 *harness*。本书研究的正是如何设计好这套周边系统。

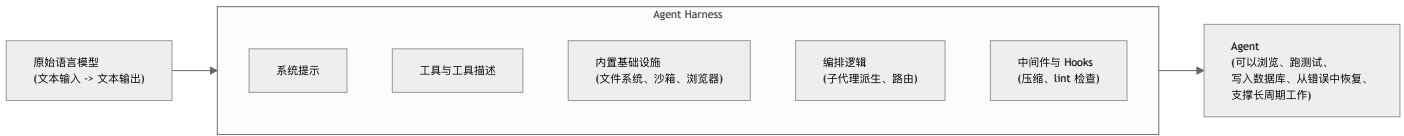
阅读本书时，可以把模型理解为一个接收 token、输出 token 的组件。模型既可能生成供用户阅读的文本，也可能生成请求执行某项操作的结构化文本，但真正执行操作的始终是周边软件。正因为模型输出不等于现实世界中的实际效果，后续章节才会重点讨论上下文、工具、状态、测试、沙箱和评估。

配套的第一卷已经介绍了本书所依赖的模型内部机制，包括 token、attention 与 KV-cache、context window、采样、检索和 tool-call 协议。本书只会简要回顾这些概念，并在此基础上讲解 *harness engineering*。新增内容大多属于软件工程范畴：上下文、工具、状态、沙箱和评估。

开始之前

本书是两卷系列中的第二卷。我们假定读者已经读过 *LLM Foundations for Harness Engineering*，并熟悉 token、attention 与 KV-cache、context window、采样、post-training、检索、agent loop、tool-call 协议、prompt injection，以及 pass@k/pass^k。遇到第一卷已经讲过的概念时，本书会标出对应章节（例如“Foundations 第 9 章”），而不再从头推导。没有读过第一卷的读者仍可理解本书主线，但应把这些简短回顾视为进一步阅读的入口，而不是完整讲解。

核心公式



要点

- 语言模型本身不能维护状态、执行代码或访问实时知识；这些都是 *harness* 层面的能力。
- Harness engineering* 不等同于 *prompt engineering*：它改进的是整个系统，而不只是单个提示词。
- 这个领域仍然年轻，许多关键文章发表于 2025 和 2026 年，但实践正在快速成熟。
- 本书为每个论断都附上引用，方便读者回到原文。

延伸阅读

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>

第 1 章：什么是 Agent Harness?

1.1 Model + Harness 公式

原始语言模型只能接收文本、输出文本；配套的前置卷介绍了它的原生能力与局限（见《LLM Foundations》第 1 章）。要把模型变成 agent，让它能够浏览代码库、运行测试、写入数据库、与用户对话、从错误中恢复，乃至在长达数小时的任务中持续推进，就必须围绕模型构建所有这些额外能力。LangChain 将由此形成的 harness 分为几类：系统提示；工具及其描述；文件系统、沙箱和浏览器等内置基础设施；子代理派生、模型路由等编排逻辑；以及负责压缩、续跑、lint 检查等确定性操作的 hooks 或中间件 ([LangChain - The Anatomy of an Agent Harness](#))。

这一框架把系统设计问题清楚地摆在了台面上。模型本身无法跨多次交互保存持久状态、执行代码、获取实时知识，也不能自行配置环境和安装软件包。这些都属于 harness 层的能力 ([LangChain - The Anatomy of an Agent Harness](#))。即便是最基本的聊天也依赖一种 harness 模式：while-loop 保存历史消息，再把新消息加入上下文，于是模型看起来像是“记得”刚才的对话。

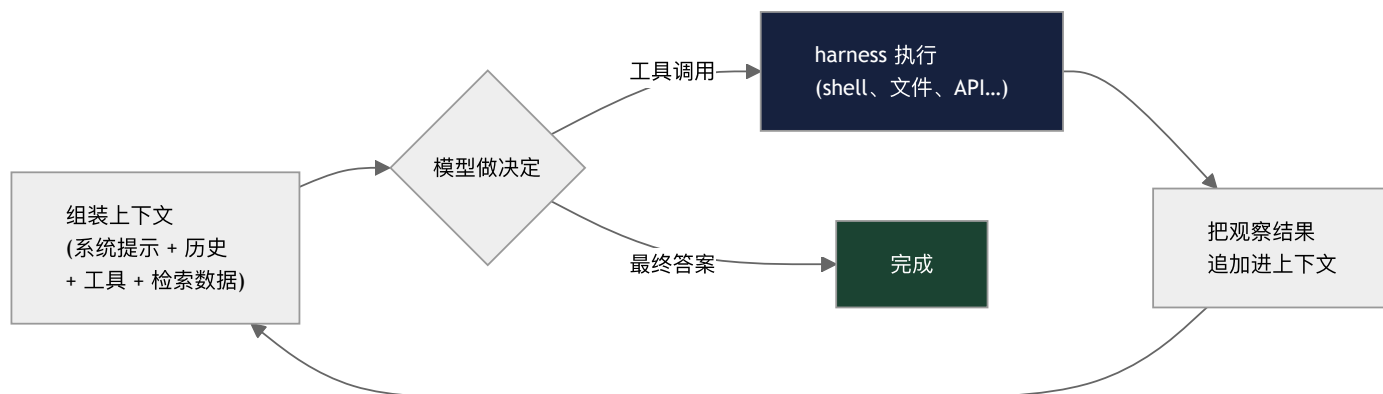
HumanLayer 从配置 coding agent 的角度给出了几乎相同的公式：“coding agent = AI model(s) + harness”。它把 harness 称为 agent 的运行时或“外围设备”，也就是模型与环境交互所依赖的各个组件 ([HumanLayer - Skill Issue: Harness Engineering for Coding Agents](#))。

1.2 Agent 循环

配套的前置卷已经介绍了 agent 循环，以及它背后的一项关键原则：工具调用只是交给 harness 执行的结构化输出，并不是模型亲自采取的行动（见《LLM Foundations》第 1 章与第 12 章）。简要回顾一下：原始模型调用是一次性的，也就是文本输入、文本输出；agent 则把这次调用放进一个循环。Harness 先组装上下文，模型再输出最终答案或一个工具调用。工具调用通常是 JSON 格式的结构化输出，其中包含工具名称和参数。随后，确定性的 harness 代码执行该调用，把观察结果加入上下文，并以扩展后的上下文开始下一轮。

HumanLayer 用一句话概括了同一个意思：“tools are just structured outputs” ([HumanLayer - 12-Factor Agents](#))。无论是编辑文件、执行 shell 命令、点击浏览器，还是写入数据库，都只有在 harness 接受并执行模型请求之后，才会在现实中产生效果。位于这个循环中心的模型，就是 Anthropic 所说的 *augmented LLM*，也是研究文献中的 *ReAct* (reason + act)（具体机制见《LLM Foundations》第 12 章）。

这个循环会带来两个贯穿全书的后果。第一，**上下文单调增长**：每一轮都会追加一次工具调用及其观察结果，因此一个包含 N 个步骤的任务会积累 N 轮历史。正因如此，第 2 章把上下文视为有限资源，而压缩、子代理和记忆都是管理这种资源的手段。第二，**模型本身从不执行任何操作**。模型只会发出请求，确定性的 harness 代码则负责决定是否执行、如何执行。请求与执行之间的这段距离，为后续章节中的护栏、沙箱、hook 和审批关卡提供了介入点。Harness 就位于循环之中，连接模型提出的请求与最终发生的现实效果。



1.3 Harness 的边界：内层与外层

“Harness”这个词并没有完全统一的边界。Thoughtworks 的作者指出，不同人所说的 harness 可能覆盖不同层次。Birgitta Böckeler 建议将其理解为三层同心圆：核心是模型；中间层是 coding agent 的 *builder harness*，包括 Anthropic、OpenAI 等厂商提供的系统提示和工具；外层是 *user harness*，包括团队为适配自己的代码库而添加的 AGENTS.md、hooks、skills 和 review agents ([Thoughtworks / Martin Fowler - Harness Engineering](#))。多数工程师的日常工作主要集中在最外层。

OpenReview 论文 *Agent Harness Engineering: A Survey* 对这条边界给出了更正式的定义：harness 是一套软件和接口基础层，用来管理 foundation model 如何获取上下文、调用工具、持续执行任务，并在部署环境中保持可审计性 ([OpenReview - Agent Harness Engineering: A Survey](#))。论文进一步将 harness 视为一种潜在的 *binding constraint*：对于长周期 agent，可靠性往往既取决于模型本身的质量，也同样取决于模型周围的运行基础。

1.4 ETCLOVG：七层系统地图

这篇综述用 **ETCLOVG** 组织 harness 的设计空间：Execution environment、Tool interface、Context、Lifecycle、Observability、Verification 和 Governance ([OpenReview - Agent Harness Engineering: A Survey](#))。这张地图提醒我们，harness 不能被简化为只有 prompt 或 tool。

- **Execution environment**：让操作产生实际效果的沙箱、浏览器、操作系统、代码执行器或托管云环境。
- **Tool interface**：协议、schema、registry、function calling、MCP/A2A 式边界和工具选择策略。
- **Context**：prompt 组装、检索、记忆、压缩、状态压缩，以及允许模型看到哪些信息。
- **Lifecycle**：任务启动、规划、checkpoint/resume、失败恢复、handoff、session 终止和长时间运行所需的状态。
- **Observability**：trace、telemetry、成本归因、延迟、token 用量核算和失败取证。
- **Verification**：eval harness、grader、任务集、outcome check 和 readiness gate。
- **Governance**：权限、策略语言、审计记录、人类审批、constitutional/rule-based 控制和跨层安全。

接下来的大部分章节，都可以看作对这七个层次的逐一展开。第 2—3 章主要讨论 context 和 memory；第 4—5 章讨论 tools 和 execution；第 7—9 章转向 lifecycle；第 10—12 章则展开 verification、observability 和 governance，并在展望部分再次回到这些主题。

1.5 Harness 为什么存在：从模型缺陷倒推

LangChain 提供了一种倒推 harness 组件的方法：先列出你希望 agent 表现出的行为，再找出模型原生无法做到的部分。所需的 harness 组件便可以从这些能力缺口中推导出来 ([LangChain - The Anatomy of an Agent Harness](#))。

例如，模型只能处理上下文窗口中的信息，因此文件系统需要承担持久存储、信息卸载，以及 agent 与人类共享工作空间的作用。Bash 和代码执行解决的是另一项局限：我们不可能预先定义 agent 可能用到的每一种工具，因此需要提供通用执行通道，让模型按需创建工具。执行过程又必须置于安全边界之内，所以需要沙箱。模型无法凭空获知权重和当前上下文之外的信息，所以需要借助记忆与搜索把新信息注入上下文。上下文窗口本身容量有限，而且内容越多，性能越可能下降，因此还需要压缩、工具结果卸载和 skills。

每个组件都在弥补一项具体局限，而 harness 就是这些组件共同构成的系统。

1.6 历史脉络：从 Prompt Engineering 到 Harness Engineering

Anthropic 将这几次转变描述为一种自然演进。早期 LLM 应用主要关注 *prompt engineering*，也就是为一次性任务编写和组织指令。随着应用发展成能够多轮交互、长时间运行的 agent，关注范围扩展到了 *context engineering*：在 LLM 推理期间整理并持续维护最有用的一组 token，其中也包括提示词之外进入上下文的所有信息 ([Anthropic - Effective Context Engineering for AI Agents](#))。

Harness engineering 又比 context engineering 高一个层次。按照 Mitchell Hashimoto 的说法，每当 agent 出错，都应投入时间设计系统性解决方案，避免它以后重犯同样的错误 ([HumanLayer - Skill Issue: Harness Engineering for Coding Agents](#) quoting Hashimoto)。Prompt engineering 调整的是单个提示；harness engineering 改进的则是提示运行于其中的整套系统。

2026 年，这一演进又出现了名为 *loop engineering*（循环工程）的新视角。随着 agent 开始在更长周期内无人值守地运行，工作的关注点再次转移：先是 prompt，再到 context，如今则是决定“给模型什么提示、何时调用模型、结果是否合格”的 *loop* ([Addy Osmani - Loop Engineering](#))。Loop engineering 与其说是 harness engineering 的替代方案，不如说是从运行管理角度观察其外层控制循环，重点关注围绕 agent 的 trigger、verifier 和 stop rule。第 8 章将完整展开这一概念。

1.7 Framework、Runtime 与 Harness

这三个词有时会被混用。LangChain 的 Harrison Chase 对它们作了如下区分 ([LangChain - Agent Frameworks, Runtimes, and Harnesses, Oh My!](#))：

Framework（如 LangChain、Vercel AI SDK、CrewAI、OpenAI Agents SDK 或 Google ADK）提供开发抽象，帮助开发者快速入门，并统一应用的构建方式。*Runtime*（如 LangGraph、Temporal 或 Inngest）处理基础设施问题，包括持久执行、流式输出、human-in-the-loop，以及线程内和跨线程的状态持久化。*Harness*（如 LangChain 的 DeepAgents 或 Anthropic 的 Claude Agent SDK）则又高一层，通常内置默认提示、对工具处理方式的明确取舍、规划工具、文件系统访问，以及其他“开箱即用”的能力。三者的边界并非绝对，例如 LangGraph 既可以被视为 runtime，也可以被视为 framework；即便如此，这一区分仍有助于团队判断应该采用什么。

1.8 更好的模型会让 Harness 过时吗？

一个始终伴随 harness engineering 的问题是：模型变得更好之后，周边系统会不会就不再重要？HumanLayer 在“Skill Issue”中指出，团队常常把问题归咎于模型——“GPT-6 会解决”“我们只需要模型更听指令”——但真正的症结往往在于

harness 配置 (HumanLayer - Skill Issue: Harness Engineering for Coding Agents)。更好的模型会消除一些现有的失败模式，但也会被交付更困难的任务，并继续以意想不到的方式失败。这样的意外失败，是非确定性系统的基本属性。因此，harness engineering 是一项需要持续投入的工作，而不是等模型足够强大后就能丢弃的临时脚手架。

LangChain 也得出了类似结论。随着模型能力提升，今天位于 harness 中的一部分功能可能会被模型吸收。即便如此，harness engineering 仍有价值：既可以弥补模型的不足，也可以围绕模型智能构建系统，让这种智能得到更有效的发挥 (LangChain - The Anatomy of an Agent Harness)。

图：模型到 Harness 层



要点

- **Agent = Model + Harness**: 任何超出原始文本输入输出的能力，都必须通过周边系统构建出来。
- **Agent 循环是基础**: 组装上下文、由模型发出工具调用、由 harness 执行，再把结果追加到上下文——循环每运行一轮，上下文都会继续增长。
- **工具调用是结构化请求**: 模型以文本形式提出操作请求，harness 决定哪些请求会产生实际效果。
- **三层同心圆**: LLM 核心、AI 实验室提供的 builder harness、团队自行构建的 user harness。
- **ETCLOVG 提供系统地图**: execution、tools、context、lifecycle、observability、verification、governance 都是 harness 层。
- **Harness 组件来自模型局限**: 文件系统、沙箱、记忆和压缩分别弥补不同的能力缺口。
- **Harness engineering 需要持续投入**: 模型越强，承担的任务也越难，新的失败模式仍会出现。
- **Framework 不等于 Runtime，也不等于 Harness**: 理解这些区别有助于团队做出技术选型。

延伸阅读

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, Oh My!, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

第 2 章：上下文是一种有限资源

2.1 上下文退化与注意力预算

对 agent harness 来说，最重要的运行约束是：有用的上下文不等于模型所能容纳的最长上下文。其背后的失败模式称为 *context rot*（上下文退化），即输入越长，模型的输出质量反而可能越差。配套卷已将这一现象拆成两层：一层来自长度本身，另一层来自无关信息的不断累积（见《LLM Foundations》第 9 章）。本章把上下文退化作为贯穿全章的设计约束。Anthropic 还将它与 *needle-in-a-haystack benchmark* 联系起来：上下文中的 token 越多，模型从中准确找回信息的能力就越可能下降 ([Anthropic - Effective Context Engineering for AI Agents](#))。具体影响取决于模型、任务以及相关信息所在的位置，因此应把它视为一种概率性的工程规律，而不是对每个 prompt 都必然成立的定律。

Anthropic 对其机制的解释并不是 transformer 会在字面意义上“耗尽”注意力。真正的问题是，上下文越长，模型需要表示的 token 两两关系就越多，而训练数据和位置机制通常更擅长处理较短、较局部的依赖。位置编码插值等长上下文技术虽然能让模型处理超出原始训练长度的序列，却仍可能牺牲位置分辨率或检索可靠性 ([Anthropic - Effective Context Engineering for AI Agents](#))。

工程上的结论是：上下文是一种边际收益递减的有限资源。Anthropic 将它比作“注意力预算”，每加入一个 token 都会消耗一部分预算。HumanLayer 的说法更直接：即使模型支持更长的上下文窗口，生产系统通常也应优先采用短小、聚焦的 prompt 和上下文。根据他们的经验，开放式的“tool-calling loop”运行约 10-20 轮后，往往就很难恢复到良好状态 ([HumanLayer - 12-Factor Agents](#))。

如果模型确实需要访问更多信息，更大的上下文窗口当然有帮助；但窗口变大，并不会自动改善注意力分配或指令遵循能力。HumanLayer 指出，扩展上下文版本往往使用 YaRN 等技术来延长可处理的序列，而不是提高模型实际可用的“指令预算”。对于 *needle-in-a-haystack* 式任务，更大的窗口有时只是带来一个更大的草堆 ([HumanLayer - Skill Issue](#))。

2.2 有效上下文的结构

在这一预算约束下，目标就是 Anthropic 所说的：找到“尽可能少的一组高信号 token，使期望结果出现的概率最大化” ([Anthropic - Effective Context Engineering for AI Agents](#))。落实到 harness 中，主要涉及以下几个部分：

系统提示应处在合适的“抽象高度”：既不要为每个边界情况硬编码一套 if-else，也不要只给出模糊的高层原则。它需要具体到足以引导行为，同时给模型留出运用启发式判断的空间。Anthropic 建议用 XML 或 Markdown 分隔符来组织 prompt，例如 `<background_information>`、`<instructions>`、`## Tool guidance` 等；不过随着模型能力提高，具体格式的重要性正在下降。

工具定义了 agent 与环境之间的契约。每个工具都应当自包含、说明清楚，并尽量避免与其他工具功能重叠。Anthropic 观察到的常见问题是工具集过于臃肿：功能覆盖太广，工具之间的选择边界又不明确。如果人类工程师都无法断定某个场景应该使用哪个工具，就更不能指望 AI agent 做出更好的判断。

Few-shot 示例应当既多样又典型，而不是罗列所有边界情况。Anthropic 的类比是：对 LLM 而言，示例就像“胜过千言万语的图片”。

2.3 KV-Cache：为什么稳定前缀有价值

配套卷已经介绍过 KV-cache 和前缀稳定规则：只要改动前部的一个 token，例如系统提示中的时间戳，后面的所有 token 都必须重新计算（见《LLM Foundations》第 9 章）。本节关注的是 KV-cache 作为生产成本杠杆的价值。虽然上下文工程的学术文献很少讨论它，但它是生产级 agent 设计的核心。Manus 甚至认为，KV-cache 命中率是“生产阶段 AI agent 最重要的指标” ([Manus - Context Engineering for AI Agents](#))。

KV-cache 之所以主导成本模型，与典型的 agent 循环有关。Agent 接收输入，从工具空间中选择并执行一个动作，再把动作和观察结果追加到上下文中，供下一轮使用。上下文每轮都在增长，模型输出却通常很短，因此预填充（prefill）与解码（decoding）的比例会严重失衡。Manus 报告的平均输入输出 token 比约为 100:1。如果前缀保持不变，KV-cache 就可以直接复用它，从而缩短首 token 延迟并降低推理成本。Manus 引用的 Claude Sonnet 价格显示：每百万缓存输入 token 的价格是 \$0.30，未缓存输入则为 \$3，相差 10 倍。

Manus 给出了三条保持 cache 命中的规则。第一，保持 prompt 前缀稳定：哪怕只有一个 token 不同，也会使 cache 从该处开始失效，因此在系统提示中加入精确到秒的时间戳会付出很高代价。第二，让上下文只追加、不改写，并使用确定性的 JSON 序列化；有些库不保证 key 的顺序，会在不易察觉的情况下破坏 cache。第三，如果推理框架要求，应显式标记 cache 断点。

2.4 屏蔽（Mask），而不是删除

Manus 的第二条原则涉及动作空间。随着工具数量增长，MCP（Model Context Protocol，模型上下文协议；见第 4 章）让用户可以轻松接入数百个工具，人们很容易想到在 agent 循环中动态加载和卸载工具。Manus 的实验给出了一条明确规则：不要这样做。工具定义位于上下文前部，一旦发生变化，后续内容的 cache 就会全部失效。此外，早先的对话轮

次可能还引用了已经移除的工具，进而导致 schema violation 或产生并不存在的工具调用 ([Manus - Context Engineering for AI Agents](#))。

替代方案是 action masking：让工具界面在上下文中保持稳定，同时根据当前状态限制 agent 可以选择的动作。具体实现取决于 provider 和 harness，可以使用 logit constraint、tool-choice 控制、response prefill，也可以由运行时 validator 拒绝不允许的动作 ([Manus - Context Engineering for AI Agents](#))。Manus 会为同类动作采用一致的名称前缀，例如浏览器工具使用 `browser_*`，shell 工具使用 `shell_*`。这样，一条简单的约束就能允许或排除整个工具组。

2.5 文件系统作为工作记忆

即使上下文窗口达到 128K token（这是 Manus 当时引用的数字；如今的窗口更大，但基本规律没有改变），真实的 agentic 任务仍会经常超出容量。网页和 PDF 可能产生体量巨大的观察结果；模型性能也可能在触及技术上限之前就开始下降；即便能够使用 cache，长输入的成本仍然很高 ([Manus - Context Engineering for AI Agents](#))。

Manus 的解决方案，也与 Anthropic 和 LangChain 的思路一致：把文件系统当作 agent 的工作记忆。文件系统的容量远大于上下文窗口，可以跨轮次持久保存，agent 也能直接操作其中的内容，并通过路径引用保存的产物。但它并不是人类意义上的“记忆”。如果文件名、摘要、索引或检索习惯设计得不好，agent 仍然可能找不到自己写下的内容。因此，Manus 刻意采用可恢复的压缩策略：只要留下 URL，就可以把网页正文移出上下文；只要保留文件路径，就可以暂时省略文档内容。

LangChain 称文件系统是“可能最基础的 harness primitive”。它既为数据、代码和文档提供统一的工作区，也让 agent 能够逐步把中间成果转存到外部，而不必把所有内容都留在上下文中；同时，它还是多 agent 协作和人机协作的天然界面 ([LangChain - The Anatomy of an Agent Harness](#))。再叠加 git，就能获得版本管理、回滚和分支能力。

对于 coding agent，这个原则还应再进一步推进：代码仓库应当成为项目事实的权威来源。OpenAI 的 Codex harness 指南强调，面向 agent 的仓库会把所需上下文放在代码附近：由 `AGENTS.md` 指明查找方向，用短小的设计文档解释局部架构，再通过结构检查把关键规则固化下来 ([OpenAI - Harness Engineering](#))。如果一项决策只存在于 Slack、ticket 或资深工程师的记忆中，那么新的 agent session 就无法把它当作可靠上下文。

这并不意味着要把更多文字塞进 prompt，而是要让仓库中的事实容易被发现。一个实用的冷启动测试是：启动一个没有任何口头背景的新 agent session，只让它查看仓库，然后提出五个问题：这是什么系统？它如何组织？如何运行？如何验证？当前处于什么状态？如果 repo 文件和标准命令无法给出答案，说明 harness 存在知识可见性缺口。要补上这个缺口，通常需要在代码附近放置短小文档，提供稳定的入口、进度文件和验证命令，而不是堆出一个庞大的根级指令文件。

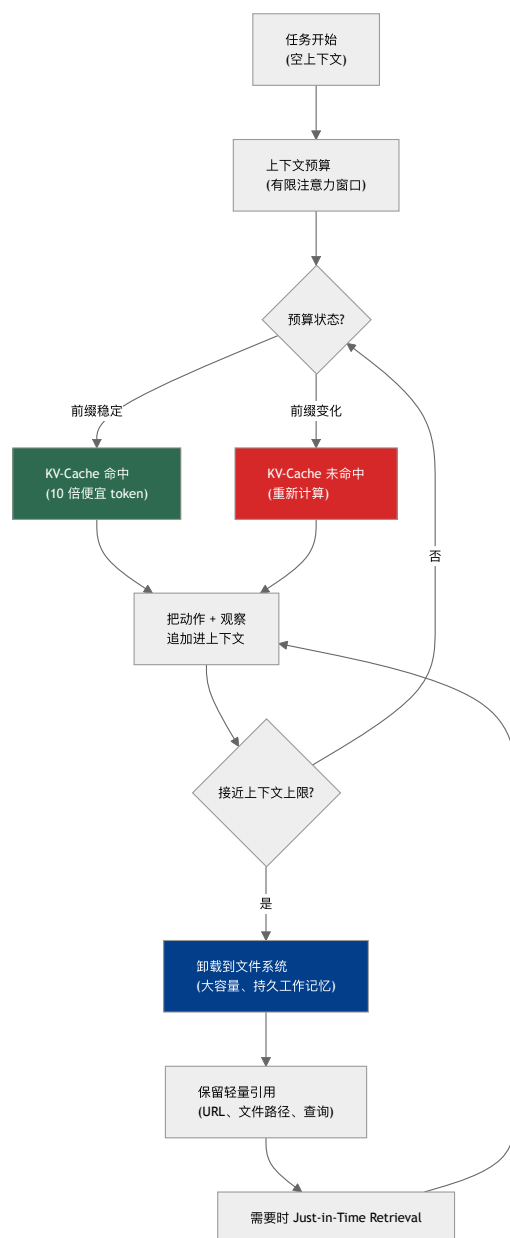
2.6 即时检索 (Just-in-Time Retrieval)

传统的 RAG 流水线会预先对所有内容进行 embedding，检索最相关的 top-k chunk，再把它们放到上下文前部（见《LLM Foundations》第 11 章）。如今，*just-in-time* 方法正成为一种重要补充：agent 不必预处理并预加载全部内容，而是先保留文件路径、查询和链接等轻量标识符，只在真正需要时才把原始数据载入上下文 ([Anthropic - Effective Context Engineering for AI Agents](#))。

Anthropic 的 Claude Code 在处理大型代码库时就采用了这种模式。模型会编写有针对性的查询、保存结果，并借助 `head`、`tail` 等工具查看大型数据集，而不是一次性加载全部内容。文件路径本身也包含有用信息：`tests/` 目录中的 `test_utils.py`，与 `src/core_logic/` 目录下的同名文件，通常承担完全不同的职责。目录层级、名称和时间戳都可以成为 agent 的导航线索。

这种做法也有代价：运行时探索比检索预计算数据更慢；如果缺少明确的工具使用指导，agent 还可能在无效路径上浪费上下文。因此，实践中常采用混合模式：预先提供少量高价值上下文，例如 `CLAUDE.md` 或 `AGENTS.md` 中的项目说明，再让 agent 按需探索其他内容。

图：上下文预算 -> KV-Cache -> 文件系统记忆



要点

- 上下文退化必须纳入设计：更大的上下文窗口确实有帮助，但不能取代对上下文的筛选与组织。
- **KV-cache** 是重要生产指标：稳定前缀对延迟和成本的影响可能与模型质量同样关键。
- 能屏蔽就不要频繁更换工具界面：运行中动态增删工具会破坏 cache locality，还可能导致 schema violation。
- 文件系统是工作记忆，不是万能记忆：它容量更大、保存更持久，但仍然需要清晰的路径、摘要和检索规则。
- 仓库是 **agent** 的权威事实来源：关键项目信息应当能在冷启动时通过 repo 文件和命令找到。
- 即时检索往往优于预加载：先保留轻量标识符，只在需要时载入原始数据。

延伸阅读

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>

第 3 章：压缩、记忆与子代理模式

即使上下文工程做得足够严谨，长周期任务仍可能超出任何一个上下文窗口的容量。相关文献大体归纳出三类延续任务的方法。

3.1 压缩

压缩 (compaction) 是指：当对话接近上下文窗口上限时，先总结已有内容，再以这份摘要为起点开启新的上下文窗口 ([Anthropic - Effective Context Engineering for AI Agents](#))。在 Claude Code 中，harness 会把消息历史交给模型，要求它保留架构决策、尚未解决的 bug 和实现细节，同时舍弃冗余的工具输出。随后，agent 根据压缩后的上下文和最近访问过的文件继续工作。何时触发压缩——通常以 token 数或上下文使用比例为阈值——本身也需要调优：太晚触发，可能在一轮尚未结束时就溢出；太早触发，又可能丢掉仍有价值的细节。

Anthropic 建议在真实、复杂的 trace 上调优压缩 prompt。第一步先提高 recall，确保所有相关信息都进入摘要；然后再改善 precision，删除无助于继续任务的内容。最轻量的压缩形式是清理工具结果：当 agent 已经根据某次工具调用的结果采取行动后，原始输出通常就可以丢弃。

必须注意，压缩是有损的。它适合保留决策、目标、约束以及产物的位置，却无法保留漫长调试 trace 中的每一个细节。因此，设计良好的 harness 会把摘要与可恢复的引用结合起来，例如 commit hash、文件路径、issue ID、URL 和短笔记。这样，当摘要信息不足时，后续 agent 仍能重新加载一手证据。

3.2 结构化笔记

与压缩互补的模式是 *agentic memory*：agent 定期把结构化笔记写入磁盘，并在之后按需重新加载。Anthropic 用 Claude 玩宝可梦来说明这种做法。在数千个游戏步骤中，agent 会持续记录进度（例如“过去 1,234 步一直在 1 号道路训练宝可梦，皮卡丘已提升 8 级，目标是 10 级”）、绘制地区地图并整理战斗策略。这些笔记让它即使经历上下文重置，也能继续完成持续数小时的训练序列 ([Anthropic - Effective Context Engineering for AI Agents](#))。

Manus 的 `todo.md` 技巧是结构化笔记的一种特殊形式，同时还有下一节将介绍的另一层作用。

3.3 反复复述：把注意力拉回上下文尾部

Manus 的 agent 在处理复杂任务时会创建 `todo.md`，随着工作推进反复重写，并逐项勾掉已经完成任务。这样做不只是为了整理工作。典型的 Manus 任务平均包含约 50 次工具调用；随着上下文变长，模型很容易偏离任务或忘记早期目标。反复重写 todo list，相当于不断把目标“复述”到上下文末尾，让全局计划重新进入模型最近的注意范围，从而缓解“lost-in-the-middle”问题（底层机制见配套卷《LLM Foundations》第 9 章）([Manus - Context Engineering for AI Agents](#))。

3.4 子代理与上下文防火墙

第三种模式是子代理分解，它对系统架构的影响也最大。一个专门的 sub-agent 会接收范围明确的任务，并在独立的上下文窗口中工作。它内部可能消耗数万 token，最终却只向父 agent 返回一份 1,000-2,000 token 的压缩摘要 ([Anthropic - Effective Context Engineering for AI Agents](#))。HumanLayer 将这道边界称为 *context firewall*（上下文防火墙）：父线程只负责编排，看不到子代理工作过程中的噪声，只接收浓缩后的结果，因此能够更长时间地保持上下文有效，避免进入“dumb zone” ([HumanLayer - Skill Issue](#))。

HumanLayer 明确区分了角色扮演与上下文控制。仅仅把子代理设定为“前端工程师”或“后端工程师”之类的 persona，通常没有多少帮助；真正有价值的是把边界清楚的工作委派出去，保护父 agent 的上下文。最适合交给子代理的任务，往往最终答案很简洁，却需要大量中间工具调用，例如在代码库中定位定义、追踪跨服务的信息流，或开展大范围研究。

子代理还可以用来控制成本。HumanLayer 使用昂贵的模型 (Opus) 负责编排，再让更便宜的模型 (Sonnet 或 Haiku) 执行委派任务。像 `grep` 这样的简单操作，没有必要消耗 orchestrator 所用的最强模型。

Anthropic 的多代理研究系统是这一模式规模化应用的典型案例。Lead agent 先分析查询，再并行派出多个专门的 sub-agent，分别调查不同方面。每个 sub-agent 都在自己的上下文窗口中工作，随后把压缩后的结果返回给 lead，由 lead 汇总成最终报告。在 Anthropic 的内部研究评估中，由 Opus 担任 lead、Sonnet 担任 sub-agent 的配置，比单 agent Opus 相对高出 90.2% ([Anthropic - How We Built Our Multi-Agent Research System](#))。这一提升很大程度上来自 token economics：在他们的分析中，三个因素解释了 BrowseComp benchmark 上 95% 的性能方差，而 token 使用量单独就解释了 80%。

代价则是更高的成本。根据 Anthropic 的数据，single-agent run 使用的 token 约为普通 chat 的 4 倍，多 agent 系统则约为 15 倍。由此可推得，多 agent 系统的成本约为单 agent 的 4 倍；这个比值是根据来源数据推算得出的，并非来源直接给出的数字。因此，只有当任务价值较高，而且并行确实能带来收益时，多 agent 系统才划算。对于共享可变状态、彼此紧密耦合的子任务，它通常并不合适，许多实现工作量较大的 coding task 就属于这一类。不过，如果 coding

workflow 中委派的是只读调查，或任务可以沿清晰的 ownership 边界拆分，子代理仍然很有价值。当前模型也不擅长在 agent 之间实时协调，因此 coordinator 必须明确划定任务边界。

3.5 不要把 Few-Shot 做成惯性

Manus 提出了一个反直觉的原则：上下文过于一致也可能有害。模型善于模仿它看到的模式。如果 trace 中充满相似的 action-observation 对，模型可能在这种模式已经不再适用时仍继续照做，进而出现偏离、过度泛化或幻觉。Manus 举的例子是批量审阅 20 份简历：agent 很容易形成固定节奏，最终只是为了重复而重复 (Manus - Context Engineering for AI Agents)。

相应的修复方法，是引入少量而有结构的变化，例如使用不同的序列化模板、替代表述、轻微调整顺序，或加入受控噪声。让 trace 保持一定多样性，可以避免某一种模式垄断模型的注意力。

3.6 保留有用的错误

另一个与之互补的原则是保留有用的错误。面对失败动作，人们自然会选择重试并隐藏失败 trace；但 Manus 认为，这样做会删掉模型用来避免重蹈覆辙的证据 (Manus - Context Engineering for AI Agents)。把最近一次失败动作及相关 stack trace 留在上下文中，模型才能据此调整下一次尝试。当然，这不意味着要无限期保存所有日志。反复出现的同一种失败，应压缩成简短诊断和 retry counter。Manus 将错误恢复称为“真正 agentic 行为最清晰的指标之一”，并指出学术 benchmark 往往只衡量理想条件下能否成功，因此没有充分覆盖这种能力。

HumanLayer 将这一思路总结为 Factor 9：把错误压缩进上下文。Agent 读取错误并调整下一次调用的能力，也就是 self-healing，是 LLM agent 真正有价值的特性之一；而它只有在错误仍然可见时才能发挥作用 (HumanLayer - 12-Factor Agents)。再配合限制连续相同错误次数的计数器，这一模式会更加稳健。

3.7 成本、延迟与模型路由

前面的子代理模式已经把成本作为设计变量：昂贵模型负责编排，便宜模型执行辅助工作。这里需要把原则说得更明确：成本和延迟是 harness 的一等设计目标，而不是系统完成后才考虑的问题。

文献中反复出现三个杠杆：

- **模型路由**。并非每一步都需要最强的模型。harness 可以把便宜且高频的任务——例如一次 grep、一次分类或一段简短摘要——交给更小、更快的模型，把 frontier 模型留给推理密集的步骤。HumanLayer 使用 Opus 作为 orchestrator，让 Sonnet 或 Haiku 担任子代理 (HumanLayer - Skill Issue)；Anthropic 的研究系统也采用相同的 lead agent / sub-agent 拆分（见第 7 章）。
- **KV-cache**。如果上下文前缀保持稳定，就可以由 cache 直接提供，价格大约是未缓存 token 的十分之一，延迟也只有其一小部分（机制见配套卷《LLM Foundations》第 9 章；亦见第 2 章）。对于生产级 agent，严格遵守缓存规则往往是最大的单项成本杠杆。
- **Token 核算**。Agentic 工作负载的大部分开销来自 prefill。Manus 报告的输入输出比约为 100:1，而成本会随上下文长度增长。多 agent 系统可能消耗普通 chat 约 15 倍的 token，因此只有在高价值任务上才值得使用。

延迟的构成有所不同。首 token 延迟主要取决于 prefill，因此与 cache 命中密切相关；端到端延迟则主要取决于需要依次完成多少次模型往返。并行调用工具或并行运行子代理，可以显著缩短实际等待时间——在 Anthropic 的研究工作负载中最多可缩短 90%（见第 7 章）——但不会减少 token 总成本。一般原则是：把 token、金钱和秒数分别列为明确预算，并弄清楚每个设计杠杆影响的是哪一项。

3.8 具名记忆架构

第 3.1-3.3 节把压缩、笔记和复述作为 harness 技术来讨论。研究文献还将这些思路组合成若干具名的记忆系统，了解它们有助于识别不同的架构选择。

- **MemGPT** 直接借用了操作系统的类比。它把上下文窗口视为高速“主存”，把外部存储视为“磁盘”，再让模型通过函数调用，在固定容量的上下文中换入、换出信息。这种虚拟上下文管理 (virtual context management)，让一个容量有限的窗口呈现出远大于自身的可用空间 (Packer et al. - MemGPT: Towards LLMs as Operating Systems)。该系统现已产品化为 Letta。
- **Mem0** 是一个记忆层。它会从对话中动态抽取重要事实，与已有存储进行整合，并在后续轮次中检索；其图结构变体还能够表示实体之间的关系。它所报告的优势主要体现在运行效率上：对于跨多个会话的长对话，token 成本和延迟都远低于回放完整历史 (Chhikara et al. - Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory)。
- **Sleep-time compute (睡眠期计算)** 利用请求之间的空闲时间。Agent 不再只是等待，而是离线处理已有上下文，预判可能出现的后续问题并提前计算相关推断，从而减少后续查询所需的 test-time compute。在部分 benchmark 上，达到相同准确率所需的推理预算因此降低了约 5 倍 (Lin et al. - Sleep-time Compute: Beyond Inference Scaling at Test-time)。

从 harness 的角度看，警告仍与第 3.1 节相同：受管记忆虽然强大，却也是有损的，而且会引入新的失败模式。系统可能取回错误的记忆、错误地整合信息，或信心十足地陈述已经过时的事实。只有在会话很长、确实需要跨会话回忆时，这些系统的复杂性才有价值；对于短任务，它们往往只是在第 3.2 节所述结构化笔记之上增加额外开销。

3.9 多代理拓扑及其失败原因

第 3.4 节把子代理作为上下文防火墙引入，并介绍了 orchestrator-worker 配置；第 6、7 章将进一步讨论编排。除此之外，实践中还常见以下几种多代理拓扑 (topology)：

- **Orchestrator-worker (supervisor, 主管)**：由一个 lead agent 分解任务、把各部分委派给 worker，再综合它们的结果（第 6、7 章）。
- **Hierarchical (分层)**：由 supervisor 管理其他 supervisor，适用于任务分解层次太深、单个 lead 无法在上下文中容纳完整结构的情况。
- **Blackboard / shared memory (黑板 / 共享记忆)**：agent 通过读写公共工作区进行协调，而不是彼此直接发送消息。当多个 agent 共同构建同一份不断演进的产物时，这种拓扑尤其有用。
- **Debate / voting (辩论 / 投票)**：多个 agent 通过辩论或投票来提高可靠性，是第 6.3 节 parallelization-voting 模式的多代理版本。

人们很容易把更多 agent 等同于更强能力，但实证结果要冷静得多。MAST 研究人工标注了七个流行多代理框架中的 200 多个任务，并总结出 14 种失败模式，分为三大类：**规格问题 (specification issues)**，即角色和 prompt 定义不足；**代理间错位 (inter-agent misalignment)**，即 agent 之间各说各话、丢失信息或偏离共同目标；以及**任务验证 (task verification)**，即对最终结果的检查薄弱或完全缺失 (Cemri et al. - [Why Do Multi-Agent LLM Systems Fail?](#))。这项研究的关键结论是：很多失败并不是模型能力不足，而是**协调与验证**出了问题，而这些恰好都属于 harness 的职责范围。

由此可以直接得出几条实践原则：优先采用能够满足需求的最简单拓扑（第 6.1 节）；明确划分任务边界，避免 worker 重复工作或遗漏任务（第 3.4 节、第 7 章）；把验证作为一等职责，而不是事后补丁，例如采用第 7 章的 generator-evaluator 拆分。MAST 已将验证薄弱列为三大失败类型之一。

3.10 记忆投毒与信任升级

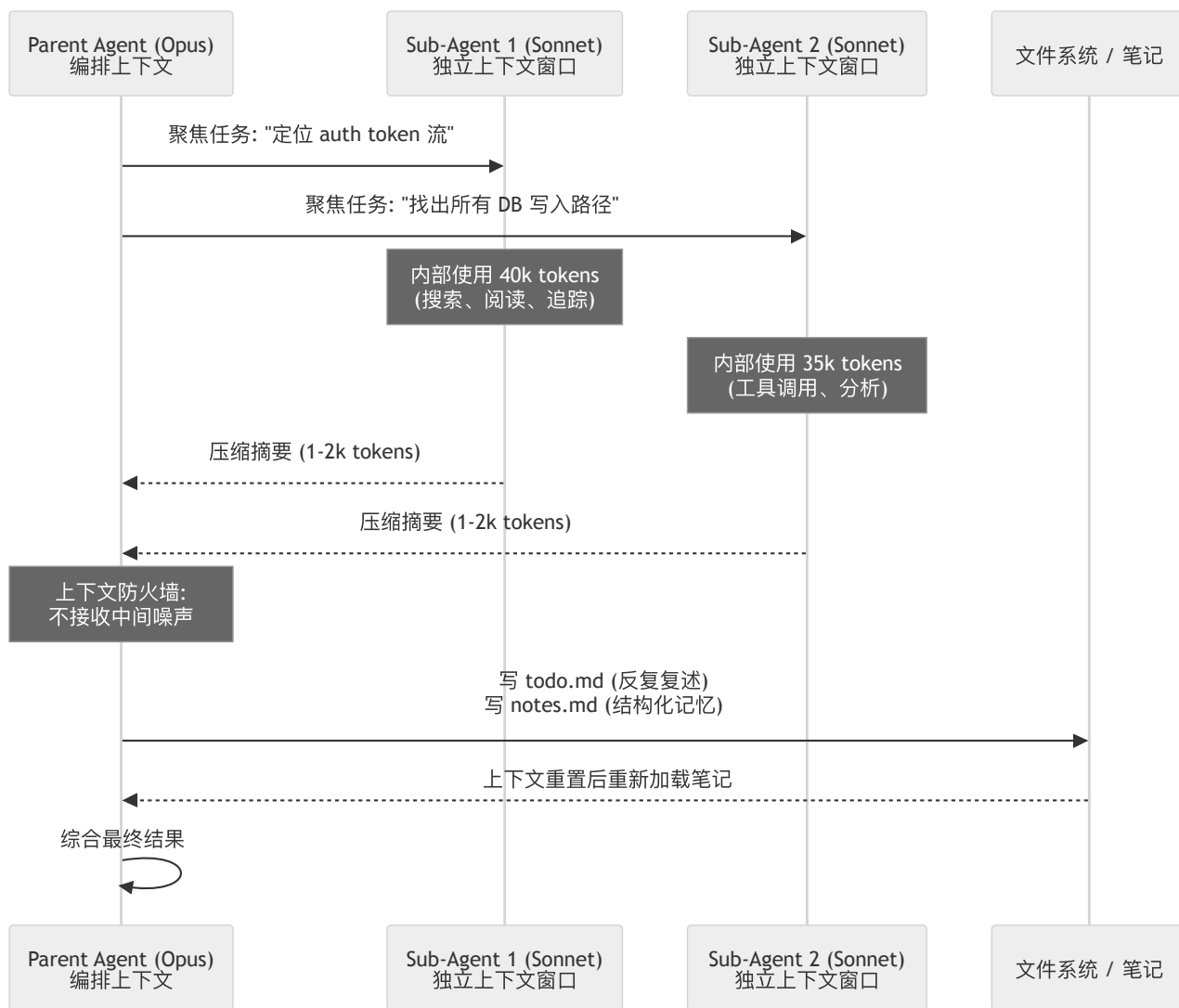
持久记忆会改变系统的安全模型。网页中的注入指令通常只会威胁当前一次运行；但如果 agent 把这条指令写入持久笔记、偏好存储或共享黑板，攻击就可能跨过上下文重置，继续影响后续运行。Anthropic 将这种现象称为**持久记忆投毒 (persistent memory poisoning)**：不可信内容跨越写入边界，进入了未来 agent 会当作可信状态的存储 (Anthropic - [How We Contain Claude](#))。

防御的关键，是把记忆视为带有溯源信息的存储，而不是上下文的中性延伸：

- 为每条记忆标注来源、授权写入的身份、创建时间和信任等级；
- 将观察与指令分开，绝不自动把检索到的内容提升为策略；
- 不可信信息在进入持久的共享记忆前，必须经过审查或确定性验证；
- 支持过期、替代和回滚，以便从所有后续上下文中移除被投毒的事实；
- 检索时重新检查授权，因为写入者可以访问的事实，读取者未必有权访问。

多 agent 系统还会引入**信任升级 (trust escalation)**。低权限 worker 可能向高权限 coordinator 返回一份看似可信的摘要，随后 coordinator 使用 worker 从未拥有的权限采取行动。压缩会进一步放大这种风险，因为溯源信息和不确定性往往最先在摘要中消失。因此，子 agent 的结果应附带引用或产物位置、置信程度和未决问题，以及生成结果时所使用的身份与权限。父 agent 必须先核验证据，才能把这些结果转化为高权限动作。第 5 章将展开 containment 控制，第 18 章则介绍如何通过身份与控制平面模型，在整个 fleet 中执行这些控制。

图：父 Agent -> 子 Agent -> 压缩结果（上下文防火墙）



要点

- 压缩能延长任务跨度，但一定会丢失细节：应保留关键决策和可恢复引用，而不是每一条原始观察。
- 结构化笔记支持跨会话连续工作：把进度写入磁盘后，agent 可以在上下文重置后恢复任务。
- 反复复述可以缓解 **lost-in-the-middle**：持续重写 todo list，能把目标重新带回模型最近的注意范围。
- 上下文防火墙是子代理模式的核心价值：父 agent 不接收中间噪声，只获取浓缩后的结果。
- 应保留有用的错误：self-healing 依赖可见的错误 trace，但重复失败应当压缩。
- 成本与延迟都是设计变量：把低成本工作路由给小模型，保持 KV-cache 命中，并通过并行缩短实际等待时间。
- 具名记忆系统封装了常见记忆模式：MemGPT（OS 式虚拟上下文）、Mem0（抽取-整合-检索）和 sleep-time compute（离线预处理）都是有用的参照，但也各自引入了检索错误和信息过时等失败模式。
- 更多 agent 不仅放大成本，也会放大协调失败：MAST 分类法表明，规格缺口、代理间错位和薄弱验证主导了多代理失败，而这些都属于 harness 的职责范围。应优先选择最简单的拓扑，并把验证作为一等职责。
- 持久记忆是一道信任边界：保留溯源、区分数据与指令，并在把不可信信息提升为共享状态前完成验证；否则，一张被注入的网页就可能影响之后的多次运行。
- 委派不能悄然提升权限：父 agent 在使用 worker 从未拥有的权限采取行动前，必须先验证 worker 提供的证据。

延伸阅读

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025.
<https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>

- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
- Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
- Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>

第 4 章：工具与 Agent-Computer Interface

4.1 为什么工具设计不同

Anthropic 借鉴 HCI（人机界面）的思路，提出了 *agent-computer interface* (ACI)：设计 agent 使用工具的方式，应该像设计人类使用界面的方式一样受到重视 ([Anthropic - Building Effective Agents](#))。具体到工具格式，有三条建议：

- 给模型留出足够的 token 来“思考”，再让它提交难以修改的语法。
- 尽量采用模型在训练数据中熟悉的格式。
- 避免不必要的格式处理，例如精确计算 diff header 的行数，或对嵌入 JSON 的代码做过度字符串转义。

Anthropic 在构建 SWE-bench agent 时，优化工具 schema 所花的时间甚至超过了优化 prompt。一个改动就能说明工具设计的影响：把相对路径改为绝对路径后，agent 离开仓库根目录时出现的路径错误几乎全部消失 ([Anthropic - Building Effective Agents](#))。

4.2 选择正确工具与正确数量

Anthropic 后续在 “Writing Effective Tools for Agents” 中强调了一个常见误区：给 agent 更多工具，不一定会带来更好的结果 ([Anthropic - Writing Effective Tools for Agents](#))。一种典型错误，是不考虑 agent 的使用方式，直接把每个 API endpoint 都包装成工具。Agent 所需要的 *affordance*（可供性）与传统软件不同。例如，用 `list_contacts` 搜索通讯录时，agent 只能逐个读取联系人，既浪费 token，也挤占有限的上下文；`search_contacts` 或 `message_contact` 才是更直接的工具。

设计工具时，还应合并 agent 经常连续执行的操作。与其分别提供 `list_users`、`list_events` 和 `create_event`，不如直接提供 `schedule_event`；与其提供 `read_logs`，不如提供 `search_logs`；与其让 agent 依次调用 `get_customer_by_id`、`list_transactions` 和 `list_notes`，不如把它们合并为 `get_customer_context`。

4.3 工具从哪里来：Model Context Protocol

本章接下来的内容假设 agent 已经拥有一组设计良好的工具。实践中，许多工具会通过一个标准接口提供：*Model Context Protocol* (MCP，模型上下文协议)。简要说，MCP 是一项开放的客户端—服务器标准。*MCP server* 通过统一协议暴露工具，也可以同时提供资源和可复用的 prompt；任何兼容的 *client*，例如 Claude Code、IDE 或自定义 agent，都能发现并调用这些能力，不必为每项能力单独开发集成。传输机制和“不应信任工具返回结果”的风险已在《LLM Foundations》第 12 章介绍；本章关注的是如何把 MCP 作为 harness 的一个设计面 ([Anthropic - Code Execution with MCP](#))。

MCP 的主要价值在于可组合性。团队可以把独立构建和维护的 Google Drive server、Salesforce server 与内部数据库 server 接入同一个 agent；对 agent 来说，它们共同组成了一个统一的工具界面。这也是 MCP 在本书中反复出现的原因：它为下文的命名空间、masking 和代码执行模式提供了基础。

MCP 的代价是，它也让工具过度供给变得非常容易。正如第 2 章所说，用户可以轻易接入数百个工具，使上下文被大量工具定义占满；第 2 章“mask，而不是删除”的原则，以及上文合并操作的建议，正是为了控制这种膨胀。MCP 只是连接工具的管道，不能替代工具设计：设计糟糕的 MCP server，只会批量提供设计糟糕的工具。无论工具是手写的，还是通过 MCP 接入，都应该遵循本章的原则：合并常见的连续操作、设置命名空间、限制响应大小，并像编写新人入职文档一样认真撰写工具说明。

MCP 同时形成了一条部署边界。不能因为托管 agent 需要访问内部 server，就把该 server 直接暴露到公网。OpenAI 的安全 MCP tunnel 会在私有网络内运行一个仅出站（**outbound-only**）的 client：该 client 只连接预先明确配置的目的地址，保留 streaming 和 authentication，同时仍由客户掌控并可供检查，因此无需开放入站端口 ([OpenAI - Connect Private MCP Servers to OpenAI Products](#))。这不只是网络实现技巧，也体现了一种可复用的安全模式：跨边界访问工具时，应由拥有私有能力的一侧发起连接，把访问限制在指定目的地，并按照 *delegated authority* 的标准进行审计。

4.4 四类集成边界

OpenReview 综述指出，比较工具和协议标准时，按它们所跨越的边界分类，比按厂商或发布时间分类更有意义 ([OpenReview - Agent Harness Engineering: A Survey](#))：

- **Model -> Function**：以 function calling 为代表的结构化调用。模型发出机器可读的请求，再由确定性代码执行。
- **Agent -> External capability**：以 MCP 为代表的解耦方式。Agent runtime 发现外部 server 暴露的 tools、resources 和 prompts。
- **Agent -> Agent**：以 A2A 为代表的委托方式。一个 agentic application 把工作交给另一个拥有自身状态和工具、但内部不透明的 agent。
- **Agent -> Repo/environment**：由版本控制管理的 policy 和 affordance，例如 AGENTS.md、本地 skills、仓库命令以及针对具体环境的工具规则。

从边界出发，就能理解为什么 MCP、A2A、OpenAPI、function calling 和 AGENTS.md 不能互相替代：它们解决的是不同的集成问题。Harness 设计者应先确定实际跨越的是哪条边界，再选择相应的协议和治理模型，确保 provenance、permission、cost 和 failure evidence 都能跨越该边界得到保留。

4.5 命名空间

当 agent 能访问几十个 MCP server 和数百个工具时，名称冲突和用途不清会成为严重的失败来源。Anthropic 建议使用统一前缀组织相关工具：先按服务设置 asana_*、jira_* 等前缀，再按服务内的资源细分为 asana_projects_*、asana_users_* 等前缀。评估结果表明，使用前缀还是后缀并非无关紧要，它会影响工具使用表现，而且最佳方案取决于具体 workload ([Anthropic - Writing Effective Tools for Agents](#))。

Manus 也用这一模式控制 agent 的动作空间：为所有浏览器工具加上 browser_ 前缀，为所有 shell 工具加上 shell_ 前缀，harness 就能通过简单的 logit 约束一次 mask 整组工具 ([Manus - Context Engineering for AI Agents](#))。

4.6 返回有意义的上下文

工具响应应该优先提供相关信息，而不是追求最大灵活性；标识符也应尽量采用有含义的名称，而不是不透明的技术 ID。Anthropic 发现，把字母数字 UUID 替换为有语义的标签，甚至改用从 0 开始的简单 ID，都能显著提高 Claude 的准确率并减少幻觉 ([Anthropic - Writing Effective Tools for Agents](#))。如果 agent 需要可读名称，而下游调用又必须使用技术 ID，可以通过 response_format enum 提供 concise 和 detailed 两种模式。在 Anthropic 的 Slack 示例中，concise 响应的体积只有 detailed 响应的三分之一。

4.7 Token 高效响应

工具响应是上下文膨胀的主要来源之一。Anthropic 默认将 Claude Code 的工具响应限制在 25,000 token，并建议为分页、范围选择、过滤和截断设置合理的默认值，再组合使用这些机制 ([Anthropic - Writing Effective Tools for Agents](#))。响应被截断时，还应告诉 agent 如何换用更高效的策略，例如执行多次小范围的精准搜索，而不是一次搜索过宽的范围。错误响应也应给出有用的下一步，而不只是返回难以理解的 traceback。

HumanLayer 在自己的代码库中用 “back-pressure” 落实了这一思路：build 和 test hook 成功时保持静默，只在失败时返回错误。早期，他们曾让 agent 每次修改后都运行完整测试套件；4,000 行通过测试的输出会迅速占满上下文，导致 agent 忘记真正的任务，甚至开始臆测测试文件的内容 ([HumanLayer - Skill Issue](#))。

4.8 对工具描述做 Prompt Engineering

Anthropic 把工具描述视为最有效的设计杠杆之一，并指出，改进工具描述是 Claude Sonnet 3.5 在 SWE-bench Verified 上达到 SOTA 的关键因素 ([Anthropic - Writing Effective Tools for Agents](#))。撰写描述时，可以把读者想象成一位刚入职的初级工程师：明确写出原本隐含的上下文，包括特殊查询格式、领域术语和资源之间的关系；参数名也要避免歧义，例如使用 user_id，而不是含义宽泛的 user。随后在 workbench 中运行大量示例，分析错误，再持续迭代。

一个具体案例说明了这种做法的价值。Claude 的 web search 工具刚推出时，trace 显示 Claude 会无故在 query 参数中附加 2025，从而使搜索结果产生偏差。这个问题无需重新训练模型，只要把工具描述写得更清楚即可解决。

4.9 代码执行作为元工具

一种较新的做法，是不再把 MCP 工具直接暴露为调用，而是把它们组织成代码 API，让 agent 通过编写代码来调用。Anthropic 在 “Code Execution with MCP” 中指出，当 agent 面对几十个 MCP server 和数百个工具时，传统方式会产生大量浪费：所有工具定义都要预先装入上下文，每个中间结果也都要经过模型 ([Anthropic - Code Execution with MCP](#))。

替代方案是把 MCP server 表示成一组 TypeScript 文件，每个工具对应一个文件，其中包含 callMCPTool 的类型化 wrapper。Agent 可以先查看目录，再按需读取具体工具文件，从而逐步发现工具。在 Anthropic 的 Google Drive -> Salesforce 示例中，这种方式把 token 使用量从 150,000 降到 2,000，节省了 98.7%。

这种设计带来的多项收益会相互增强：

- **渐进披露**：只在需要时加载工具，减少前置上下文成本。
- **提高结果的上下文效率**：agent 可以先在执行环境中把 10,000 行 spreadsheet 过滤到 5 行，再把筛选结果送入模型上下文。
- **改善控制流**：循环、条件分支和错误处理都可以采用熟悉的代码模式。条件由 runtime 求值，无需模型消耗 token 来逐步推演。
- **保护操作中的隐私**：中间结果默认留在执行环境中，只有 agent 显式 log 的内容才会进入模型。设计得当的 proxy 还可以在 MCP client 边界对 PII 做 token 化，使原始值不必到达模型。
- **持久化状态并积累 skills**：agent 可以把有效代码保存为由 SKILL.md 支持的可复用函数，逐步形成自己的工具箱。

Cloudflare 以 “Code Mode” 为名报告了相似结果。这些实践共同指向一个直接的结论：LLM 擅长编写代码，工具接口应当利用这种能力 ([Anthropic - Code Execution with MCP](#))。

OpenAI 的 **programmatic tool calling**（程序化工具调用）**把同一思路应用在一次 Responses API 调用内部：模型编写一段短程序，调用获准使用的工具，对结果进行过滤或连接，最后只把精简后的产物返回模型。这种方式很适合处理有界数据流，例如过滤、连接、排序、去重、聚合和验证，因为确定性的 runtime 控制可以替代多次模型往返。但如果每次观察都会显著影响下一步判断、每项动作都要单独审批，或者所有来源结果都必须保留给引用和审查，它就不适用 ([OpenAI - Programmatic Tool Calling](#))。简而言之，可以用代码压缩机械性的编排，但不能用它遮蔽后果重大的决策。

代码执行也有代价：它需要沙箱基础设施，因此会增加运营和安全成本。同时，审计对象也会发生变化。Harness 不能只记录最后的精简结果，还必须记录执行的程序、程序获准调用的工具，以及由此产生的副作用。

4.10 用 Eval 迭代工具

Anthropic 推荐的工具开发流程有四阶段 ([Anthropic - Writing Effective Tools for Agents](#)):

1. **Prototype**: 在本地 MCP server 中制作工具原型，手动测试并建立直觉。
2. **Build an evaluation**: 用真实任务、真实数据和可验证的成功标准构建评估，而不是只使用玩具式 sandbox。
3. **Run the evaluation**: 以程序化方式运行评估，并捕获 trace，其中包括 planning summary、工具调用、工具结果、runtime、token count 和工具错误。如果模型提供 visible thinking mode，可以借此调试行为，但 eval 不应依赖不可见的 chain-of-thought。
4. **Analyze results**: 阅读 transcript，既关注 agent 说了什么，也关注它没有说什么——LLM 不一定会准确表达其实际判断——然后据此重构工具。

Anthropic 把这个循环用于内部 Slack 和 Asana 工具。在 held-out 测试集上，经 Claude 优化的版本超过了专家手写实现。这个结果既支持了上述 workflow，也提供了 agent 改进自身工具的早期案例。

4.11 Agent 到 Agent 的边界：A2A

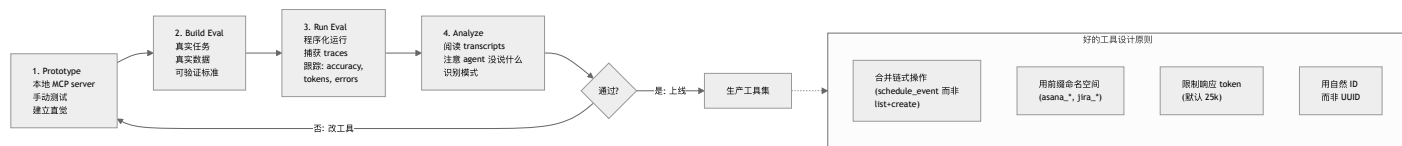
第 4.4 节列出的四类集成边界中，上文已经介绍了 model-to-function、agent-to-external-capability (MCP) 和 agent-to-repo/environment。第四类 *agent 到 agent* 边界也出现了自己的标准。**Agent2Agent (A2A)** 协议由 Google 于 2025 年 4 月推出，并在 2025 年 6 月捐赠给 Linux 基金会。这项开放标准允许一个 agentic 应用把工作委托给另一个*不透明 (opaque)* 的 agent。所谓不透明，是指这个对等 agent 拥有自己的模型、工具、记忆和内部状态，但不会把这些内部细节暴露给调用方 ([Agent2Agent \(A2A\) Protocol](#); [Linux Foundation - Agent2Agent Protocol Project](#))。

为了复用现有 Web 基础设施，A2A 刻意采用了常规的技术机制：通过 HTTP(S) 和 JSON-RPC 2.0 通信；每个 agent 发布一张描述自身能力的 *Agent Card*，供其他 agent 发现；任务生命周期则涵盖任务提交、交互形式协商（文本、文件或结构化数据），以及把结果流式返回给调用方。

理解 A2A 的关键，在于它与 MCP 的差异。MCP 把 agent 向下连接到由它控制、也可以检查的工具和资源；A2A 则把 agent 横向连接到一个既无法控制、也无法检查其内部状态的对等体。因此，首要工程问题不再是工具 schema 设计，而是**如何在组织边界之间建立信任并保留溯源信息**。调用方无法查看对等 agent 的上下文或 sandbox，所以 A2A 返回的内容必须按照第 5 章所说的不可信内容处理。对网页适用的 lethal trifecta 防护原则，同样适用于对等 agent；治理措施，包括受限身份、委托授权和审计，也必须覆盖整次 A2A 调用，而不能止于本地进程边界（第 5 章、第 17 章）。A2A 没有消除信任问题，只是把需要解决这一问题的边界标准化了。

这条边界也带来一条实用的拓扑规则：原生多 agent 执行最适合**相互独立、范围有界且结果可合并**的任务，例如并行研究、隔离审查或分别制作不同产物。面对存在先后依赖的任务链，或多个 writer 需要共同修改同一份可变状态的情况，多 agent 通常不是合适的抽象，因为协调成本和 race condition 会成为主要问题。此时应保留一个明确的 owner，再配合普通工具或确定性 workflow。即使任务确实适合交给 peer agent，第 18 章介绍的控制平面仍须为每次调用确认身份、委派权限、版本、lineage 和撤销状态。

图：工具设计流水线



要点

- 工具设计与提示设计同样重要：正如 HCI 面向人类设计界面，ACI 面向 agent 设计工具界面。

- **工具并非越多越好**：应把经常连续执行的操作合并为用途明确的工具。
- **MCP 标准化了工具的来源**：它让不同 server 提供的工具可以组合，也让工具过度供给变得容易，因此工具设计原则仍然不可缺少。
- **工具协议跨越不同边界**：function calling、MCP、A2A 和 repo-local policy 互补，不是互相替代。
- **命名空间并非表面修饰**：它既支持对整组工具进行 masking，也能减少大型 MCP 环境中的命名冲突。
- **工具响应是上下文膨胀的主要来源**：默认就应设置大小上限，并支持分页、过滤和截断。
- **把代码执行作为元工具是一项重要转变**：在 Anthropic 的示例中，把 MCP 工具暴露为类型化代码 API，节省了 98.7% 的 token。
- **程序化工具调用适合有界数据流**：过滤、连接、排序、去重、聚合和验证可以交给代码；自适应判断、审批和用于引用的证据仍应对 agent 保持可见。
- **私有 MCP 连接应仅允许出站**：连接应由拥有私有能力的网络发起，目的地必须受限，并按 delegated authority 进行审计。
- **四阶段 eval loop 是推荐 workflow**：prototype -> build eval -> run eval -> analyze transcripts -> iterate。
- **A2A 标准化了 agent 到 agent 的边界**：它让一个 agent 通过 JSON-RPC 和 Agent Card 把工作委托给不透明的对等体，也把工程重点从工具 schema 设计转向跨组织边界的信任与溯源；第 5 章的 lethal trifecta 防护和治理规则同样适用于这里。
- **并行 agent 需要彼此独立的任务归属**：如果存在共享可变状态或顺序依赖，通常更适合由一个 owner 负责，再配合确定性编排。

延伸阅读

- Ken Aizawa, *Writing Effective Tools for Agents - with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
- Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
- OpenAI, *Connect Private MCP Servers to OpenAI Products*, Jun 2026. <https://developers.openai.com/blog/connect-private-mcp-servers-to-openai-products>
- OpenAI, *Programmatic Tool Calling*, 2026. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>

第 5 章：沙箱、护栏与安全自治

5.1 Agent 安全威胁模型

本章主要讨论各种缓解措施，包括沙箱、hooks 和审批关卡。在介绍这些措施之前，必须先明确它们要防范什么。一个既能读取不可信内容、又能操作外部系统的 agent，面临以下几类特有风险 ([Anthropic - Beyond Permission Prompts](#)):

- **Prompt injection (提示注入)** (见《LLM Foundations》第 8 章 “Prompt Injection as Context Confusion” 与第 12 章) —— 模型无法可靠地区分数据与指令。因此，agent 读取的网页、issue 评论、源文件或工具结果等不可信内容，都可能像命令一样影响它的行为。
- **数据外泄** —— 一旦 agent 被操纵，只要它拥有网络访问权，就可能把 SSH key、API token 或专有源码等机密发送到攻击者控制的目的地。
- **破坏性操作** —— 被操纵的 agent 如果拥有文件系统或 shell 访问权，就可能删除或损坏文件，也可能提交并推送有害代码。
- **工具与供应链风险** —— 恶意或遭到入侵的 MCP server、软件包或依赖，可能引入敌对工具或指令，并诱使 agent 信任它们。

实践中最令人担忧的组合，有时被称为 *lethal trifecta* (致命三要素)：能够访问私有数据、会接触不可信内容，并且可以向外通信 ([Simon Willison - The lethal trifecta for AI agents](#))。每项能力单独存在时尚可管理；如果同一个 agent 同时具备三项能力，一条注入指令就可能读取机密并将其发给攻击者。本章的大多数控制措施，都是通过切断其中一环来降低风险：网络隔离切断外部通信，文件系统隔离限制私有数据访问，审批关卡则让人类介入后果重大的操作。

贯穿本章的前提是：模型不是可信组件。它能力很强，却可能受到操纵；harness 的作用，就是在敌对指令和现实后果之间建立一道可执行的边界。

Anthropic 后续的 containment 工作从两个维度进一步细化了这一威胁模型 ([Anthropic - How We Contain Claude](#))。从风险来源看，问题可能来自滥用系统的用户、模型自身的异常行为，也可能来自通过内容操纵模型的外部攻击者；从防御位置看，控制可以部署在模型、执行环境或外部内容边界上。这个矩阵之所以重要，是因为没有任何单层防御能覆盖所有风险来源：alignment 无法保证模型一定忽略提示注入，sandbox 也无法判断一封获准发送的邮件在语义上是否有害。生产环境中的 containment 必须在三个位置同时实施 defense in depth。

5.2 权限疲劳问题

完全无人监督的 coding agent 很危险，但每执行一步都要申请批准的 agent 同样无法实用。Anthropic 把后一种问题称为 approval fatigue：用户频繁点击 approve，不仅会拖慢开发循环，还会逐渐不再认真核对批准的内容，反而降低安全性 ([Anthropic - Beyond Permission Prompts](#))。解决办法不是增加更多弹窗，而是建立清晰的结构边界：agent 可以在边界内自由行动，只有越界时才请求权限。

Anthropic 的内部使用数据显示，在不降低安全性的前提下，沙箱把权限提示减少了 84% ([Anthropic - Beyond Permission Prompts](#))。

5.3 沙箱同时是笼子、重置按钮和许可证

OpenReview 综述指出，sandbox 的作用不止是保障安全。在 agent 系统中，沙箱同时承担三项任务 ([OpenReview - Agent Harness Engineering: A Survey](#)):

- **Security**：限制不可预测的模型动作和 prompt injection 所造成的 blast radius。
- **Reproducibility**：为 eval、训练轨迹和长时间运行的 session 提供可重置的 baseline。容器或 microVM 可以销毁后重建，开发者工作站则不能。
- **Liveness**：划定 agent 可以自主行动的区域，使它不必在每次写文件、安装软件包或发起网络调用时都询问人类。

第三项作用尤其体现了 agent 系统的特点。沙箱不只是限制行为的笼子，也是允许行动的许可证。它把逐项询问权限改为按 session 配置权限，使长周期自治真正可用，同时避免陷入权限疲劳。

Containment 的强度应随任务风险提高。以下三种常见模式构成了一条实用的防护阶梯 ([Anthropic - How We Contain Claude](#)):

- ****临时容器 (ephemeral container) ****可随时丢弃，成本也较低，适合范围明确、且 agent 需要在较小 blast radius 内充分行动的任务。
- **Human-in-the-loop sandbox**适合交互式工作：安全操作可以自动执行，一旦跨越边界，任务就会暂停并请求审批。
- ****密封虚拟机 (sealed VM) ****通过更强的 kernel 和网络边界隔离高风险 workload，但启动和运营成本也更高。

真正应该问的，不只是“是否使用了 sandbox”，而是“哪些资源仍然可以访问、哪些状态会在 reset 后保留、哪些权限能够跨越边界”。即使计算环境可以重置，也无法消除挂载在其中的 credential、写入环境外部的被投毒记忆，或能够传输私有数据的 egress 路径所带来的风险。

5.4 文件系统隔离必须与网络隔离配对

Claude Code 的沙箱同时建立文件系统和网络两类边界，Anthropic 认为缺一不可。文件系统隔离可以防止受到 prompt injection 的 agent 修改敏感文件，网络隔离则可以阻止它泄露数据或下载恶意软件。如果没有网络隔离，遭到入侵的 agent 可能外传 SSH key；如果没有文件系统隔离，它又可能逃出沙箱并获得网络访问能力 ([Anthropic - Beyond Permission Prompts](#))。

这一实现基于 OS 级 primitive，包括 Linux bubblewrap 和 macOS seatbelt；它不仅约束 Claude Code 的直接操作，也覆盖其启动的所有 subprocess。网络流量先通过 Unix domain socket 进入 proxy，再由 proxy 执行域名限制；agent 请求访问新域名时，也由 proxy 负责向用户确认。该 runtime 已经开源。

Claude Code on the web 把这一设计扩展到云端沙箱。在那里，git credentials、signing keys 等敏感凭据从不与 agent 一同放入沙箱。Git 交互由自定义 proxy 处理，只有在确认操作符合权限要求后，proxy 才会附加 scoped credentials，例如确保 push 的目标是预先配置的分支。

Egress allowlist 本质上是一种能力授予，而不是一份无害的目的地清单。允许访问 package registry，就意味着允许下载可执行代码；允许访问源码托管站点，也可能意味着允许发布内容；允许访问通用 Web endpoint，则可能补齐 lethal trifecta 中的数据外泄一环。因此，网络策略不能只判断域名，还应同时绑定目的地、协议、操作、身份和任务，并记录每条连接是由哪项规则授权的。

Containment 还必须在建立信任之前就生效。用户打开仓库时，系统可能在显示 trust dialog 之前，已经开始加载配置、发现依赖、启动 language server、运行 hooks 或创建本地 listener。因此，project-open 和 config-load 路径都应按敌对输入处理：可以只解析时就不要执行；关闭自动 hooks 和 listeners；在 workspace 被明确设为可信之前，不提供 credentials 和 egress ([Anthropic - How We Contain Claude](#))。

5.5 Governance：身份、策略与审计

沙箱边界必不可少，但仅有沙箱还不够。Governance 层还要回答几个问题：agent 代表谁行动、拥有多大权限、权限如何随任务上下文变化，以及行动结束后会留下哪些证据 ([OpenReview - Agent Harness Engineering: A Survey](#))。

生产系统中有三项关键设计：

- **身份与 delegated auth**：agent 应通过 scoped credentials 或 delegated identity 行动，而不是继承用户完整的 ambient authority（默认隐式权限）。Credential vault 和 proxy 只应在操作已经获得授权的边界上附加 secret。
- **上下文相关的权限策略**：静态 allow/deny list 容易检查，但粒度较粗。任务感知策略可以在每次调用前综合评估工具名、参数、session state、目标 repo、网络域名和用户角色，再由确定性 checker 执行决策。
- **可审计的 trace**：日志不仅要记录 tool call，还要记录调用身份、权限决策、policy 版本、参数、输出摘要，以及权限升级是否经过人类批准。

供应链攻击也在这里进入 harness 的治理范围。MCP tool poisoning、tool squatting、rug-pull update、幻觉包名和 retrieval-source poisoning，都跨越了 tool interface 与 governance 的边界。安全的 harness 必须检查工具、软件包、数据集和检索来源的 provenance 与 integrity；只在 prompt 中提醒一句“请小心”远远不够。

5.6 Hooks 与 Middleware 作为程序化执行

沙箱是一类程序化护栏，hooks 和 middleware 则提供了更细粒度的控制。Claude Code 允许用户定义命令或脚本，并在 agent 启动、工具调用后、停止等生命周期事件上自动运行它们 ([HumanLayer - Skill Issue](#))。LangChain 的 middleware 在结构上与此类似。有些 hook 是完全确定性的脚本，另一些则是过程性检查点，会把新的上下文注入模型。它们之所以可靠，是因为 harness 会自动执行这些规则，而不是寄希望于模型记住规则。

常见用途包括通知、自动批准或拒绝、系统集成和结果验证。例如，hook 可以在 agent 完成时播放声音，拒绝 migration 命令并要求用户手动执行，发送 Slack 消息或创建 PR，也可以在 agent 停止前运行 typecheck 和 build，把错误返回给 agent，要求它修复后才能结束。HumanLayer 的示例 hook 会在 Claude 每次尝试停止时并行运行 Biome 和 TypeScript；检查通过时静默退出，失败时只返回错误，并以 exit code 2 通知 harness 再次启动 agent。

LangChain 报告称，这类 middleware 是 deepagents-cli 在 Terminal-Bench 2.0 上从 Top 30 提升到 Top 5 的关键因素。PreCompletionChecklistMiddleware 会在 agent 退出前拦截它，提醒它按照任务 spec 验证结果；

LocalContextMiddleware 会在启动时梳理工作目录和可用工具；LoopDetectionMiddleware 则记录每个文件的编辑次数，如果同一文件被编辑了 N 次，就提示 agent 重新审视当前方案。这样可以打断“doom loop”，避免 agent 围绕一个已经失败的方法反复尝试细微变体 ([LangChain - Improving Deep Agents](#))。

综述中的 governance 分类把这些 hook 纳入一条更完整的执行管线：pre-invocation check 可以拒绝危险的工具调用；post-invocation hook 可以在不可信输出进入上下文之前添加 taint 标记或执行 redact；stop hook 可以要求完成验证或更新审计记录；escalation hook 则可以把难以判断的情况交给人类。操作的后果越严重，安全性就越不应依赖模型是否记得某条指令。

5.7 前馈与反馈：控制论视角

Thoughtworks 的 Birgitta Böckeler 从更高层次对这些控制进行了分类 ([Thoughtworks - Harness Engineering](#))。外层 harness 的控制可以分为两个方向：

- **Guides（前馈）** 在 agent 行动之前预判并引导其行为，目标是提高首次产出正确结果的概率。AGENTS.md、skills、参考文档和语言服务器提示都属于这一类。
- **Sensors（反馈）** 在 agent 行动之后观察结果，帮助它自行纠正错误。测试、linter、type checker 和 AI code review 都属于这一类。

如果 harness 只有前馈机制，它会不断发布规则，却无法判断规则是否奏效；如果只有反馈机制，它会反复发现同类错误，却无法提前避免这些错误。两者缺一不可。

这两个方向还可以沿另一条轴继续划分：

- **Computational** 控制，例如 linter、type checker 和结构测试，具有较强的确定性，通常可在毫秒到数秒内完成，结果也可靠。
- **Inferential** 控制，例如语义分析、AI code review 和 LLM-as-judge，能够处理细微判断，但速度更慢、成本更高，而且结果具有非确定性。

两条轴相互独立。AGENTS.md 中的编码约定属于 inferential feedforward；提交时检查模块边界的 ArchUnit 测试属于 computational feedback；`/code-review` skill 属于 inferential feedback；在启动前创建项目结构的脚本则属于 computational feedforward。设计良好的 harness 会组合使用这四类控制。

5.8 三类调节对象

Böckeler 还按照 harness 所调节的对象，把它们分成三类 ([Thoughtworks - Harness Engineering](#))：

- **Maintainability harness**：调节内部代码质量、重复、复杂度、覆盖率和风格。由于已有数十年的工具积累，这是最容易建设的一类。
- **Architecture fitness harness**：调节性能、可观测性和可调试性，覆盖应用中横跨多个模块的“fitness functions”。
- **Behavior harness**：判断应用的功能行为是否符合预期。这一类问题尚未解决。如今，多数团队把功能 spec 用作前馈，把 AI 生成的测试用作反馈，有时再加入 mutation testing；Böckeler 坦率地指出，目前还不能充分信任 AI 生成的测试。

这套分类有助于评估 harness 的覆盖范围。一个在 maintainability 上很强、在 behavior 上却很弱的 harness，可能会给团队带来虚假的安全感。

5.9 时机：把质量左移

CI 的经验表明，问题发现得越早，修复成本就越低；同样的原则也适用于 harness 设计。速度快的 computational sensors，例如 linter 和快速测试，应在 commit 前运行；成本较高的 computational 与 inferential sensors，例如 mutation testing 和更全面的 code review，可以在 pipeline 中完成 integration 后运行；用于发现持续漂移的 sensors，例如死代码检测、依赖扫描和日志异常 judge，则应脱离单次变更的生命周期持续运行 ([Thoughtworks - Harness Engineering](#))。

Böckeler 指出，OpenAI Codex 团队的 harness 也采用了类似结构：用自定义 linter 和结构测试落实分层架构，再通过周期性的“garbage collection”扫描漂移，并让 agent 提出修复建议。

5.10 Harnessability、Agentic Readiness 与环境可供性

不同代码库搭建 harness 的难度并不相同。强类型语言天然提供 type-checking sensor；清晰的模块边界使架构约束可以转化为可执行规则；Spring 等 opinionated framework 则封装了许多细节，使 agent 不必自行处理 ([Thoughtworks - Harness Engineering](#))。

Böckeler 借用 Ned Letcher 提出的 *ambient affordances*（环境可供性）来概括这一点：环境本身具有某些属性，可以让 agent 更容易理解、导航和处理其中的系统 ([Thoughtworks - Harness Engineering](#))。Greenfield 团队可以从第一天起就有意识地设计这些 affordance；legacy 团队则面临相反的处境——最需要 harness 的地方，往往也最难建立 harness。

展望未来，Böckeler 提出了 *harness templates*：针对不同服务拓扑，把所需的 guides 和 sensors 打包在一起，例如 JVM CRUD service、Go event processor 或 Node dashboard，再随现有 service template 一同分发。她借助 Ashby 的

必要变异度定律解释这一设想：调节器至少要拥有与被调节系统同等的变异度。由此可见，主动限制服务拓扑本身就在减少变异度，也让构建完整 harness 更容易实现 ([Thoughtworks - Harness Engineering](#))。

在生产环境中，一个更宽泛的概念是 **agentic readiness**：自治调用方能否安全地理解和调用系统，观察执行状态，在失败后重试，并在必要时撤销操作？某项服务对人类开发者可能很友好，但如果 API 隐藏副作用、错误信息含糊，或没有稳定的 operation ID，它对 agent 来说仍然很难使用。以下设计可以提高 readiness：

- mutation 接受 idempotency key，并提供可查询的 operation status；
- API 明确区分 read、propose、commit 和 compensate，而不是把这些阶段隐藏在一次不透明调用中；
- 把 machine identity 和 delegated authorization 作为一等概念；
- 错误信息说明失败位置、重试是否安全，以及哪些证据可以证明系统已经恢复；
- 状态变化应当可观察、可归因；如果无法真正 undo，则提供补偿操作。

这些设计就是自治软件所需的 ambient affordances。它们减少了必须由模型承担的概率性推理，也为第 18 章控制平面的 policy、lifecycle 和 audit 提供了稳定的执行接口。

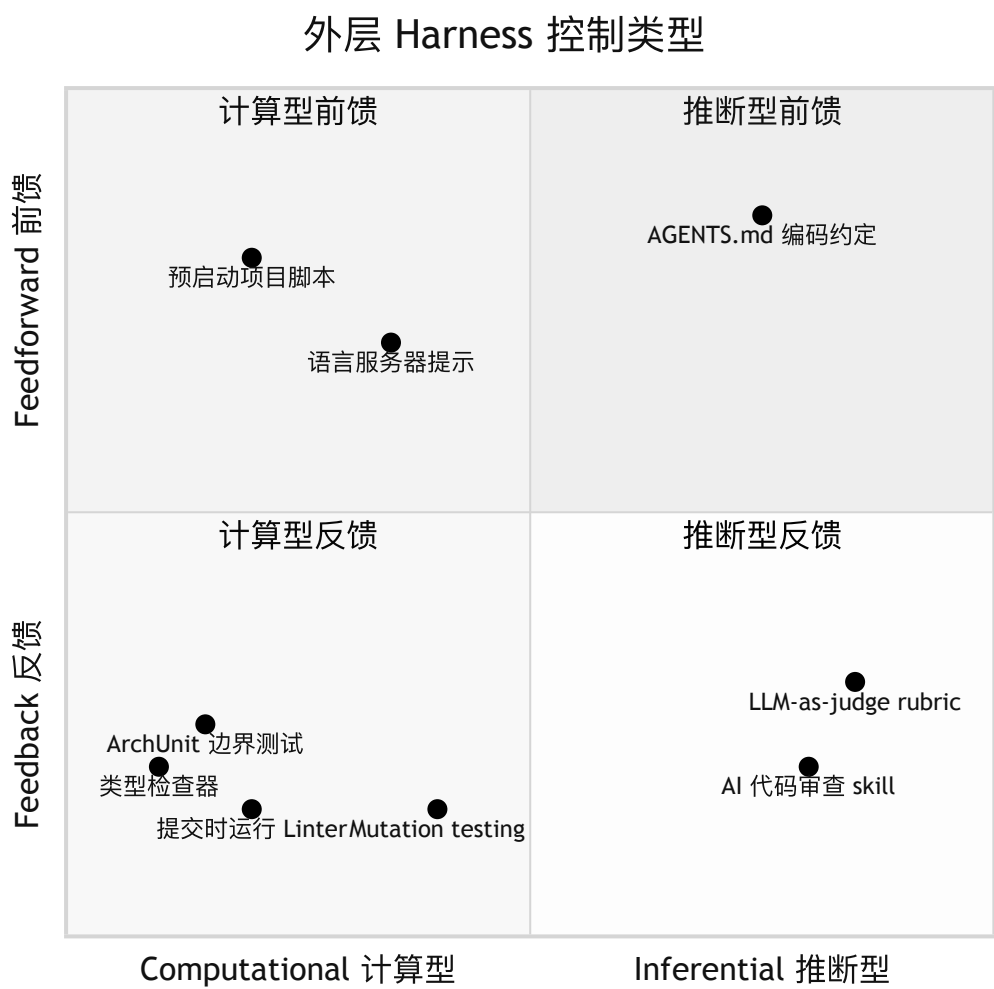
5.11 运营安全：熔断器、终止开关、预算与金丝雀

Sandbox、治理和 hooks（第 5.3-5.6 节）约束 agent 可以做什么。另一类控制则限制 agent 或工具在运行时行为异常所造成的后果。这些措施大多直接借鉴了分布式系统的可靠性工程和安全运营实践，也必须由 harness 执行——因为已经受到操纵或陷入循环的 agent，不能被指望自行约束行为。

- **熔断器 (circuit breaker)**。在不稳定或成本较高的依赖外加一层保护，例如工具、下游服务或子代理；失败次数达到阈值后，熔断器会跳闸，使后续调用立即失败，避免长时间挂起或形成重试风暴 ([Fowler - CircuitBreaker](#))。对 agent 来说，这能限制持续报错的工具，或反复重试同一失败操作的 agent 所造成的爆炸半径。它与第 3 章“保留有用错误”的建议相互补充：后者让一次失败保持可见，熔断器则防止同一失败无限重复。
- **终止开关 (kill switch)**。由人类或策略触发，能够立即停止一个 agent 或整个 fleet，并且不依赖 agent 自身的控制流。受到 prompt injection 的 agent 可能正在主动违背原有指令，因此终止开关必须存在于 harness 中，例如 supervisor 进程、可撤销凭证或 sandbox 拆除机制；不能只在 prompt 中写一句“收到要求时停止”。
- **动作预算、迭代上限与成本调节器**。为工具调用次数、token、墙钟时间或花费设置硬性上限。达到上限后，循环必须停止并上报，而不能继续失控运行。这是第 8 章循环停止规则和第 17 章单任务预算在运营层面的实现：无界循环既会造成账单失控，也会让爆炸半径失控。
- **金丝雀令牌 (canary token)**。在受到 prompt injection 的 agent 可能读取或外泄的位置放置假机密，例如未使用的 API key、诱饵文件或陷阱 URL。金丝雀触发回调时，会发出一个高可信度警报，表明 agent 已被操纵去接触本不该访问的数据 ([Thinkst - Canarytokens](#))。金丝雀与 sandbox 不同：它不会阻止 lethal trifecta 中的数据外泄环节（第 5.1 节），而是负责发现这类行为。因此，当预防措施并不完美时，它可以成为最后一道检测防线。

本节遵循的原则与全章一致：失败后果越严重，就越不能依赖模型主动选择避开风险。预防措施，如 sandbox 和策略，应与检测措施，如金丝雀和第 17 章的漂移告警，配合使用；任何一类措施单独使用都不充分。

图：前馈/反馈 x 计算/推断



要点

- **先明确威胁模型**：当私有数据、不可信内容和外部通信组成“致命三要素”时，prompt injection、数据外泄、破坏性操作和供应链风险会尤其危险。
- **沙箱可以减少 84% 的权限提示**：清晰的结构边界比反复弹出审批窗口更有效。
- **沙箱承担三项任务**：security、reproducibility 和 liveness。
- **文件系统与网络隔离必须配对**：二者对应不同攻击向量，单独使用都不足。
- **Containment 是一个矩阵，而不是单个 sandbox**：应同时在模型、环境和内容边界上防范用户滥用、模型异常与外部攻击，并根据风险选择临时容器、交互式 sandbox 或 sealed VM。
- **Egress 代表真实权限**：允许访问一个目的地，就赋予了 agent 一项实际能力，因此网络访问必须绑定操作、身份和任务；仅仅打开项目，不应自动获得 ambient trust。
- **Governance 不只是审批**：身份、scoped credentials、policy checks、provenance 和 audit trail 必须跨工具与 session 协同工作。
- **Hooks 和 middleware 提供程序化执行**：它们不依赖模型记住规则，因此比只写在 prompt 中的约束更可靠。
- **前馈与反馈都需要**：guide 没有 sensor 就没有学习回路；sensor 没有 guide 只能事后反应。
- **Harness 覆盖三类对象**：maintainability、architecture fitness 和 behavior，其中 behavior 仍然最难解决。
- **环境可供性很重要**：强类型语言和 opinionated framework 能降低搭建 harness 的难度。
- **Agentic readiness 是 API 的属性**：幂等性、明确的 operation status、machine identity、重试语义、可观察状态和补偿操作，都能让系统更适合自治调用方。
- **运行时安全还需要运营控制**：熔断器、终止开关、动作与成本预算、金丝雀令牌可以在异常发生时限制后果。它们必须存在于 harness 中，因为不能指望受到操纵的 agent 主动停止自己。

延伸阅读

- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
- Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>

第 6 章：Agentic 工作流程模式

6.1 工作流与代理

Anthropic 将广义的“代理式系统 (agentic systems)”分为两类架构 (Anthropic - Building Effective Agents)：

- **工作流 (workflows)** 通过预先定义的代码路径编排 LLM 与工具。
- **代理 (agents)** 自行决定执行过程以及如何使用工具。

Anthropic 的首要原则是：采用能够解决问题的最简单方案，只有在任务确实需要时才增加复杂度。许多场景根本不需要代理，单次 LLM 调用配合检索和上下文示例通常就已足够。对于定义清晰的任务，工作流能提供可预测性和一致性；如果执行路径无法预先确定，系统需要由模型在运行时做出决策，代理才更合适。

6.2 增强型 LLM

最基本的构件是**增强型 LLM (augmented LLM)**，也就是接入了检索、工具和记忆的模型。现代模型能够主动运用这些能力，例如自行生成查询、选择工具，以及决定保留哪些信息 (Anthropic - Building Effective Agents)。MCP (见第 4 章) 正逐渐成为向模型提供这些能力的常见方式。

6.3 组合式工作流程模式

下面这些模式从简单、受约束的形式逐步过渡到更灵活的形式：

Prompt chaining (提示链) 把任务拆成一系列顺序执行的步骤。每次 LLM 调用都处理上一步的输出，步骤之间还可以加入程序化检查。它适合能够清晰拆解的任务：缩小每次调用负责的任务范围，往往可以提高准确率，但代价是延迟增加。例如，第一次调用起草营销文案，第二次再完成翻译。它的主要风险是错误传播：链条越长，延迟累积越多，早期错误也可能污染所有后续结果。因此，应在关键边界设置检查关卡。

Routing (路由) 先对输入分类，再将其分派到专门的处理路径。它适合输入可以可靠地划分为不同类别，而且各类别需要不同处理方式的场景。例如，客服系统可以把请求分别送往退款、技术支持或一般咨询流程。分类错误和含糊输入往往不会显式报错，却会让后续流程走错方向，因此必须为分类器建立监测，并持续关注错误率。

Parallelization (并行化) 同时运行多个 LLM 调用，然后汇总输出。常见形式有两种：**sectioning** 将工作拆成相互独立的子任务，**voting** 则让同一任务重复运行多次。需要降低延迟，或希望通过多个视角提高可信度时，可以采用并行化，例如使用多组提示词审查漏洞，或让多个结果共同投票进行内容审核。不过，voting 会按运行次数成倍增加 token 成本；如果各次运行犯的是相关性很强的同类错误，即使结论全都一致，也可能只是制造虚假的确定感。

Orchestrator-workers (编排者—工作者) 由一个中心 LLM 动态拆解任务、把子任务委派给 worker LLM，再汇总各方结果。它与普通并行化的区别在于，子任务并非事先定义，而是在运行时根据输入决定。这种模式适合拆解方式取决于具体输入的复杂工作，例如需要修改多个文件的编程任务，或需要查阅大量来源的研究任务。它的主要风险是编排者无节制地派生 worker，因此必须同时限制 worker 数量和 token 总预算。

Evaluator-optimizer (评估者—优化者) 让生成与评估构成循环：通常由一个 LLM 产出答案，另一个 LLM 给出评价，再据此继续改进。它适合评价标准明确、迭代优化能够带来可测收益的任务。判断是否适用可以看两个信号：人类反馈是否能稳定改善结果，以及 LLM 是否有能力给出类似反馈。典型例子包括由批评者辅助的文学翻译，以及由相关性评估器指导的多轮研究。由于循环未必会自行收敛，必须设置最大迭代次数；每多一轮，都会增加延迟和成本。

6.4 Agent 实现的三条原则

Anthropic 最后总结了三条实现原则 (Anthropic - Building Effective Agents)：

1. 保持代理设计简单。
2. 优先保证透明度，明确展示代理的规划步骤。
3. 认真设计代理与计算机之间的接口，包括工具文档和测试。

框架可以加快早期开发，但它引入的抽象层也可能遮蔽真正决定系统行为的提示词和工具调用。Anthropic 建议，在尚未摸清问题结构时先直接调用 API；等到重复模式和运维需求逐渐稳定，再引入框架来承接这些共性。

6.5 小代理模式

HumanLayer 从工程实践中得出了相似结论 (HumanLayer - 12-Factor Agents)：“循环直到完成 (loop until done)”的模式通常在大约 10-20 轮后遇到瓶颈。超过这个范围，上下文和每一轮的误差不断累积，代理会逐渐失去连贯性（见第 2 章《上下文是一种有限资源》）。

更可靠的做法，是把小而专注的代理嵌入确定性的 DAG 中；这里的 DAG 指由代码定义各步骤的有向无环图。

LangChain 也描述了这种 harness 形态 (LangChain - The Anatomy of an Agent Harness)。在 deploybot 示例中，确定

性代码负责预发布环境部署、端到端测试和正式生产部署命令；LLM 只在需要理解自然语言反馈时介入，例如解读“能先部署后端吗？”，然后提出调整后的步骤。将代理的职责限制在 5-10 个步骤内，可以显著减少错误失控扩散。

这条原则并不会因为模型能力提升而失效。更强的代理或许能够处理更长的任务序列，但小而专注的代理能让团队现在就交付可靠系统，并在模型能力增长后逐步扩大其职责范围。

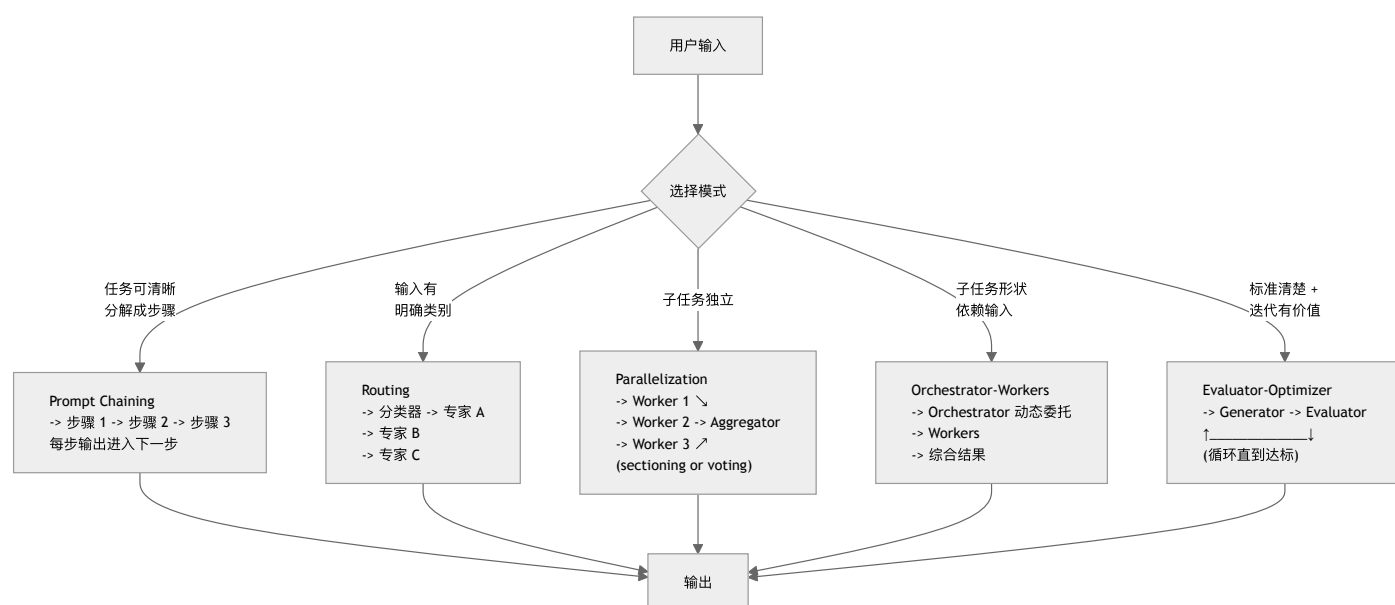
6.6 推理与自我纠错模式

第 6.3 节的五种模式属于组合式控制/流模式，解决的是如何编排调用。另一条研究路线关注推理模式，也就是单个代理或紧密循环如何组织推理与自我纠错。这两类模式可以彼此组合，而不是相互取代。下面这些名称经常出现在相关文献中：

- **ReAct** 将推理与行动交替进行：模型先思考，再调用工具、观察结果，然后重复这一过程。它是多数代理循环的基础模式（第 1 章；《LLM Foundations》第 12 章）。
- **Reflexion** 为代理增加自我反思记忆。一次尝试失败后，代理会用自然语言记录自己为何失败，再把这段反思放入上下文中重试。作者将其称为无需更新权重的“言语强化学习（verbal reinforcement learning）”（Shinn et al. - Reflexion）。
- **Self-Refine** 将 evaluator-optimizer 模式（第 6.3 节）收进同一个模型：模型先生成结果，再批评并修改自己的输出，如此迭代，直到结果令人满意（Madaan et al. - Self-Refine）。
- **CRITIC** 不让批评只依赖内省，而是借助搜索、代码执行、计算器等外部工具来验证。这样一来，纠错依据的是外部证据，而不是模型自身的信心（Gou et al. - CRITIC）。
- **Tree of Thoughts (ToT)** 不再沿单一推理链前进，而是搜索多个分支，通过前瞻和回溯探索若干分解，再决定采用哪一条路径（Yao et al. - Tree of Thoughts）。
- **LATS** 在代理轨迹上运行蒙特卡洛树搜索（MCTS），并由 LM 价值函数和反思引导搜索，从而把推理、行动与规划统一起来（Zhou et al. - Language Agent Tree Search）。
- **ReWOO** 将规划与执行分开：Planner 预先写出完整计划，Workers 执行工具调用，Solver 汇总出最终答案。由于无需在每次得到观察结果后重新推理，这种结构可以减少 token 消耗（Xu et al. - ReWOO）。

从 harness 的角度看，这些模式做的是相似的权衡：用更多 token 和延迟换取可靠性。正如第 6.1 节和第 14 章所提醒的，这种交换只在困难且可验证的任务上值得；对于简单任务，它只是额外开销。与单纯依靠模型自我批评相比，把自我纠错建立在工具或测试之上的模式更可信，例如本节的 CRITIC、第 14 章由验证器把关的级联，以及第 7 章的 generator-evaluator 分离。这也印证了本书反复强调的原则：验证比内省更可靠（第 7 章、第 10 章）。

图：五种工作流程模式



要点

- 从最简单的模式开始：许多任务只需要一次 LLM 调用，过早加入代理循环往往没有必要。
- workflow 提供可预测性，代理提供灵活性：应根据子任务结构能否预先确定来选择。
- 五种模式覆盖多数场景：chaining、routing、parallelization、orchestrator-workers、evaluator-optimizer。

- 小代理模式现阶段更容易可靠落地：在确定性 DAG 中嵌入只负责 5-10 个步骤的专注型代理，通常比“循环直到完成”更稳健。
- 引入抽象之前，先保留系统的可观察性：直接调用 API 更便于检查早期行为；等模式稳定后，框架的价值才会显现。
- 推理模式可以与工作流模式组合：Reflexion、Self-Refine、CRITIC、Tree of Thoughts、LATS 和 ReWOO 用来组织模型的推理过程。它们最适合困难且可验证的任务；如果自我纠错以工具或测试为依据，而不是只靠内省，结果会更可信。

延伸阅读

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
- Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
- Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
- Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
- Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
- Bin Feng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>

第 7 章：长时运行代理与跨上下文窗口任务

7.1 交接班问题

Anthropic 在 “Effective Harnesses for Long-Running Agents” 中用轮班来比喻这个核心难题：设想一个软件项目由多名工程师轮流接手，每位新到岗的工程师都完全不知道上一班发生了什么 ([Anthropic - Effective Harnesses for Long-Running Agents](#))。上下文窗口也形成了类似的边界。多数有一定规模的项目都无法在单个窗口内完成，因此代理需要一种可靠机制，将状态从一个会话传递到下一个会话。

单靠上下文压缩并不总能解决问题。在 Anthropic 的长时运行应用实验中，即使 Claude Agent SDK 会自动压缩上下文，一个简单的 Opus 4.5 循环仍无法根据 “build a clone of claude.ai” 这类概括性提示词，稳定构建出生产级应用。失败反复呈现为两种模式：一种是代理试图一次完成整个应用，却在实现中途耗尽上下文，只能把残局留给下一会话；另一种是后续代理看到已有部分功能完成，便误判整个项目已经结束。

7.2 Initializer + Coding Agent 模式

Anthropic 将工作分给两个不同角色，以解决这一问题 ([Anthropic - Effective Harnesses for Long-Running Agents](#))。

****初始化代理 (initializer agent) ****使用专门的提示词，只运行一次，负责生成：

- 用于启动开发服务器的 `init.sh` 脚本。
- 每个会话都要更新的 `claude-progress.txt` 日志。
- 初始 git commit。
- 一份完整的功能清单。Anthropic 采用 JSON，是因为实验中模型更容易不当地修改 Markdown 清单。在 `claude.ai` 克隆项目中，这份文件包含 200 多项功能，初始状态全部标记为 `passes: false`。

清单中的每项功能都是一个 JSON 对象，包含类别、说明、验证步骤和 `passes` 布尔值。编程代理可以修改 `passes` 的值，但指令明确禁止它删除或改写功能条本身。

****编程代理 (coding agent) ****负责之后的每个会话。它的提示词强调增量推进，每个会话都从一套结构化的准备流程开始：

1. 运行 `pwd` 确认目录。
2. 读取 git 日志和进度文件，了解上一会话处理了什么。
3. 读取功能清单，选择优先级最高的未完功能。
4. 运行 `init.sh` 启动开发服务器，并在实现新功能前完成一次基本的端到端测试。
5. 实现一项功能。
6. 对功能进行端到端验证。Anthropic 使用 Puppeteer MCP 驱动浏览器验证，因为仅靠单元测试时，代理可能在测试通过、实际功能却不可用的情况下误报完成。
7. 使用清晰的提交信息提交修改，并更新进度文件。

这套模式要求每个会话结束时，仓库都达到代码合并到主分支前所需的干净状态。这样，下一位代理可以直接继续工作，不必先花时间收拾上一位留下的问题。

7.3 会话生命周期与干净退出

长时运行的 harness 需要明确规定会话生命周期：启动、准备、选择范围有限的任务、验证结果、记录证据，然后干净退出。最后一步尤其重要。如果会话结束时仍有失败的测试、遗留的临时文件、未更新的功能清单或含糊的进度说明，下一位代理就必须先重建上一轮的状态，之后才能继续推进。

因此，“干净退出”应成为完成标准的一部分：

- 标准启动路径仍然可用。
- 已运行相关的 `build`、`lint`、`test` 或端到端检查；任何失败要么已经修复，要么已记录为阻塞项。
- 进度工件清楚说明修改了什么、验证了什么、还有哪些不确定之处，以及下一步最适合做什么。
- 功能清单或任务清单与实际状态一致：没有证据的条目不得标记为通过。
- 临时调试文件、被注释掉的实验代码和过时笔记已经删除，或被明确隔离。

OpenAI 在 Codex harness 的实践中描述了类似的维护循环：代理生成的系统往往会复刻仓库中已有的模式，因此架构规则和“黄金原则”应写入文档、linter 和周期性清理流程，而不能只依赖人类偶尔进行主观把关 ([OpenAI - Harness Engineering](#))。从会话边界来看，Anthropic 的 initializer/coding-agent 模式得出了同样的工程结论：新会话应当能依靠仓库工件接续工作，而不是依赖上一位代理头脑中的私有记忆。

对于更大的项目，可以在进度日志之外再维护一份轻量的质量文档。进度日志回答“上一轮发生了什么”，质量文档则回答“哪些模块状态健康、哪些存在风险、哪些让代理难以理解，以及哪些缺少验证”。两者的区别很重要：下一位代理不仅需要知道接下来实现哪项功能，也需要知道代码库的哪些部分正在退化。

7.4 Generator-Evaluator (GAN 启发)

Prithvi Rajasekaran 的后续文章将这一模式扩展到更困难的问题：如何根据很短的提示词构建生产级应用 ([Anthropic - Harness Design for Long-Running Application Development](#))。整个设计基于一个重要观察：让代理评价自己的工作时，即使结果平庸，它的判断也会持续偏向正面。因此，将负责产出的代理与负责评判的代理分开，是提升质量的有力手段。相比要求 generator 始终严厉审视自己的输出，调校一个独立且持怀疑态度的 evaluator 更容易。

受生成对抗网络 (GAN) 启发，这套架构为三个代理分配了不同职责：

- **Planner** 将 1-4 句话的提示词扩展成完整的产品规格。它有意停留在产品与架构层面，不预先规定详细技术设计，以免早期错误向后级联；同时，它也会被鼓励把 AI 功能纳入规格。
- **Generator** 使用 React + Vite + FastAPI + SQLite 技术栈，按照规格逐项实现功能，并用 git 管理版本。
- **Evaluator** 使用 Playwright MCP，以普通用户的方式操作运行中的应用，测试 UI、API 端点和数据库状态，再按照产品深度、功能、视觉设计和代码质量等标准评分。每项标准都有硬性门槛；任何一项不达标，整个 sprint 就判定失败，并生成详细反馈。

Generator 和 evaluator 通过 *sprint contracts* 协调。在每个 sprint 开始前，generator 提出要构建的内容以及如何验证成功；evaluator 审查方案，直到双方达成一致；随后 generator 按照已接受的合同实施。双方通过文件通信：一个代理写入文件，另一个读取并回应。

代价也相当显著。面对“create a 2D retro game maker”这条提示词，单代理运行耗时 20 分钟、花费 \$9，得到的应用看似合理，但游戏本身无法运行：实体显示在屏幕上，却完全不响应输入。完整 harness 耗时 6 小时、花费 \$200，但产出了一个真正可用的游戏制作工具，包含 sprite editor、level editor、AI-assisted level generation 和 playable mode。超过 20 倍的成本差距，换来的是可工作的应用，而不是一组表面完整、实际失效的 stub。

7.5 自验证是头号杠杆

LangChain 的 Top-30-to-Top-5 案例研究从另一个方向得出了相同结论 ([LangChain - Improving Deep Agents with Harness Engineering](#))。通过分析运行轨迹，他们发现一种常见失败模式：代理写出解决方案，重新读一遍自己的代码，觉得“看起来没问题”，随即停止。LangChain 因此在系统提示词中加入了结构化指引：Plan、Build with verification in mind、Verify by running tests and comparing output to spec、Fix；同时引入 `PreCompletionChecklistMiddleware`，在代理退出前拦截它，并要求完成一次验证。

这一做法与开发者社区流传的“Ralph Wiggum loop”类似：由一个 hook 拦截代理的退出尝试，再把原始提示词注入干净的上下文窗口，要求代理继续围绕最初目标工作 ([LangChain - The Anatomy of an Agent Harness](#))。

LangChain 将多项改动组合起来，包括用上下文 middleware 映射当前工作目录和工具、加入 build-verify 指导和 loop detection，以及采用 high-low-high 推理算力分配的“reasoning sandwich”。在模型不变的情况下，这些改动将分数从 52.8% 提高到 66.5%，增幅为 13.7 分。

7.6 Context Reset 与 Compaction

Anthropic 在后续的 harness 设计文章中明确区分了 compaction 与 context reset ([Anthropic - Harness Design for Long-Running Application Development](#))。Compaction 是在原对话中总结较早的内容，让同一个代理带着缩短后的历史继续工作。Context reset 则彻底清空上下文，启动一个全新的代理，再通过结构化交接传递上一位代理的状态和下一步计划。

两种方法解决的问题不同。Compaction 用来维持连续性；context reset 则用来缓解“context anxiety（上下文焦虑）”。Anthropic 在 Sonnet 4.5 中观察到，代理一旦接近自己认为的上下文上限，就会过早收尾。Reset 能让新代理从干净状态开始，但代价是交接工件必须保存足够多的状态，确保下一位代理可靠地恢复工作。

当 Opus 4.5 本身基本消除了 context anxiety 行为后，Anthropic 便可以从 harness 中完全移除 context reset。这正是第 12 章所讨论的模型与 harness 耦合关系的一个清晰例子。

7.7 Managed Agents：解耦 Brain、Hands 与 Session State

OpenReview 的综述描述了一种平台架构，Anthropic 后来将其称为 managed agents：把模型侧负责决策的 **brain**、执行侧负责操作的 **hands**，以及持久化的 **session/event log** 分离开来 ([OpenReview - Agent Harness Engineering: A Survey](#))。Brain 决定应该做什么；hands 在可替换的环境中运行 shell 命令、编辑文件、浏览网页并调用外部服务；session log 则记录足以重建任一侧的状态。

这个拆分对长时运行任务很重要：

- 如果模型上下文耗尽，可以根据 event log 和仓库工件启动新的 brain，继续原有任务。
- 如果 sandbox 损坏、超时或遭到入侵，可以用干净镜像重新构建 hands。
- 如果操作需要凭据，可以由 proxy 和 vault 在系统边界附加凭据，而不必把 secret 放入 sandbox。
- 如果代理运行期间发生部署变更，平台可以在逐步交接过程中同时保留新旧 worker 版本。

由此可见，context reset 只是恢复手段之一。生产级的长时运行 harness 还需要环境重置、凭据隔离、支持恢复的 event log，以及对运行中会话的迁移规则。

即使产品界面把它们统称为一个“agent”，也应保持三套生命周期彼此独立：

- **session** 是持久保存的对话与事件历史；
- **harness run** 是某套模型、策略和编排配置的一次具体执行；
- **sandbox** 是可替换的计算环境，拥有自己的镜像、文件系统和网络租约。

混淆这三套生命周期会使恢复过程变得不安全。替换崩溃的 sandbox 不应抹掉 session；重置模型上下文不应悄悄保留已经受损的进程状态；升级 harness 也不应改写早期事件的来源记录。三者都应具有明确的 ID 和版本，使控制平面能够分别对它们执行恢复、迁移或撤销。

7.8 多代理研究系统

对于具有并行结构的任务，例如需要探索许多独立线索的研究工作，第 6 章的 orchestrator-worker 模式非常适合。Anthropic 的研究功能使用 Claude Opus 4 担任 lead agent，Claude Sonnet 4 担任 sub-agents ([Anthropic - How We Built Our Multi-Agent Research System](#))。Lead agent 分析查询、制定策略并行启动 sub-agents；每个 sub-agent 负责搜索并返回提炼后的发现；lead agent 汇总这些结果，最后由 citation agent 为相关论断标注来源。

Anthropic 从实践中总结出八条提示词工程原则：

1. **像代理一样思考**：在 Console 中使用代理将会使用的同一组工具来模拟提示词，再逐步观察其行为。
2. **教会 orchestrator 如何委派**：为每个 sub-agent 明确目标、输出格式、工具使用方式和任务边界。模糊的委派容易造成重复劳动或错误理解。
3. **根据查询复杂度分配投入**：在提示词中写明投入级别，例如事实查找使用 1 个代理、调用 3-10 次；比较任务使用 2-4 个 sub-agent、每个调用 10-15 次；复杂研究使用 10 个以上 sub-agent，从而避免投入过度。
4. **重视工具设计与选择**：明确写出启发式规则，例如先检查所有可用工具、根据用户意图选择工具，以及优先使用专用工具而非通用工具，避免代理沿错误方向行动。
5. **让代理改进自身工具**：一个工具测试代理使用存在缺陷的 MCP 工具、观察失败，再重写工具说明，使后续使用中的任务完成时间降低了 40%。
6. **先宽后窄**：要求代理先使用简短、宽泛的查询，再逐步缩小范围。代理的自然倾向往往恰好相反。
7. **引导思考过程**：extended thinking 可以作为可控的规划草稿区；interleaved thinking 则帮助 sub-agent 在多次工具调用之间评估质量、细化查询。
8. **并行调用工具能显著提速**：并行启动 sub-agents，同时允许每个 sub-agent 并行调用多个工具，在复杂查询中最多可将研究时间缩短 90%。

7.9 有状态 Agent 的生产可靠性

Anthropic 的研究系统文章还记录了代理长时间运行后出现的工程挑战 ([Anthropic - How We Built Our Multi-Agent Research System](#))：

- **错误会累积放大**：如果没有 checkpoint-and-resume 基础设施，小型系统故障也可能演变成灾难性后果。Anthropic 将 AI 的适应能力——告知代理工具正在失败，并允许它自行调整——与 retry logic、定期 checkpoint 等确定性保护措施结合起来。
- **调试需要新的工具**：代理在不同运行之间具有非确定性，因此完整的生产 tracing 成为主要诊断界面。这类追踪可以揭示决策模式和交互结构，而无需暴露对话内容。
- **部署需要协调**：当大量代理仍在运行时发布代码变更，需要采用 *rainbow deployments*，在新旧版本同时可用的情况下，逐步把流量从旧版本迁移到新版本。
- **同步执行会形成瓶颈**：在 Anthropic 当前的架构中，lead agent 必须等待所有 sub-agents 完成才能继续。这样虽然简化了协调，却会让整个系统被最慢的 sub-agent 阻塞。异步执行可以释放更多并行能力，但也会带来结果协调、状态一致性和错误传播方面的新挑战。

7.10 持久化执行：Checkpoint、Replay 与恢复

第 7.9 节说明了错误如何累积放大，也介绍了 Anthropic 如何将 AI 的适应能力与确定性的 checkpoint、retry 机制结合起来。持久化执行 (*durable execution*) 是支撑这种做法的通用系统工程方法。持久化执行引擎把工作流的每一步写入持

久日志。如果进程因为机器故障、超时、部署或上下文窗口耗尽而停止，它可以从最后记录的步骤恢复，而不必从头开始。

这一思想早于代理系统。Temporal、DBOS 等工作流引擎正是用它来提供容错能力 ([Temporal - Durable Execution Meets AI](#))。它也直接对应第 7.7 节的 managed-agent 架构：旧 brain 消失后，新 brain 能够继续工作，依靠的正是持久化事件日志。

对代理而言，需要持久化的基本单位是它的状态，包括上下文、工具调用结果，以及当前执行到计划的哪个位置。LangGraph 通过 *checkpointer* 提供这一能力，在每个 super-step 保存图状态。借助这些 checkpoint，系统可以在失败后恢复、暂停并等待人工介入，甚至回到先前状态。开发者还可以选择不同的持久化模式，在性能与崩溃时可能丢失的工作量之间做权衡 ([LangChain - Durable Execution](#))。

这里的核心设计矛盾是**确定性与模型的非确定性**。基于 replay 的持久化机制假设：重新执行某个步骤，就能复现它的效果。然而，模型调用和工具结果并不确定；再次运行可能偏离已经记录的历史。通常的解决办法，是把模型和工具调用视为会产生副作用的 *activity*：结果只生成并记录一次，之后直接从日志 replay，而不是重新计算。这与 ReWOO（第 6.6 节）和 context reset（第 7.6 节）背后的原则相同——“记录观察结果，不要重新计算”——只是如今被提升为基础设施层面的保证。

因此，持久化执行把第 7.3 节“干净退出、可从工件恢复”的纪律，从代理必须主动遵守的约定，变成了平台强制保证的属性。它还能减少浪费：任务中途崩溃时，不必重新支付此前已经消耗的全部 token。持久状态也是第 17 章所需回滚能力的基础；当某次 harness 变更在生产环境中表现异常时，系统正是依靠它来回滚。

Google 的 Agent Executor 将这些设计结果落实到了分布式规模 ([Google Cloud - Agent Executor](#))。系统从 event log 和 snapshot 中恢复状态，因此连接中断并不意味着任务丢失。**Single-writer rule** 可以避免多个执行者并发修改同一 session，同时平台仍能分布式运行大量 session。Runtime 还可以从较早的 checkpoint 分叉出一条新 trajectory，用于人工干预、反事实调试，或在不破坏原始 lineage 的前提下尝试另一种模型或策略。

这些能力揭示了一条通用原则：durability 不只是 retry。稳健的 runtime 还需要幂等 activity 或已记录的结果、ownership lease、optimistic 或 single-writer concurrency control、重连语义，以及每个分支的 lineage。缺少这些保护时，“resume”可能重复触发副作用，并行 worker 也可能把一条连贯历史拆成多个互不兼容的版本。

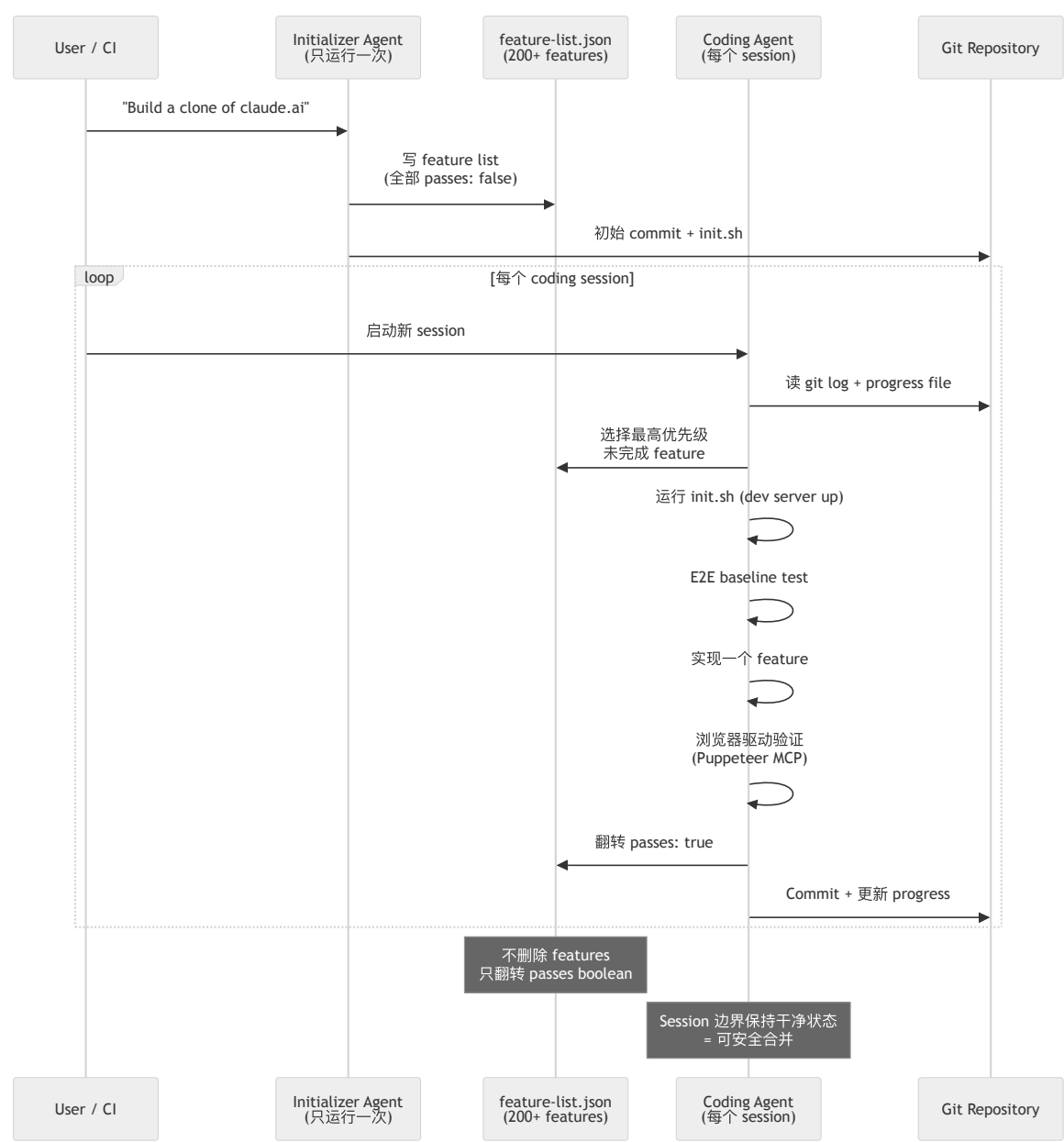
7.11 “长”到底有多长？时间视野指标

本章讨论的是超出单个上下文窗口的任务，但“长”需要一个更精确的度量。METR 提出了**时间视野 (time horizon) 指标**：以熟练人类完成任务所需的时间来衡量，模型能以 50% 可靠性完成的任务长度。比如，一个“50 分钟时间视野”的模型，在熟练人类大约需要五十分钟完成的任务上，有一半概率能够成功。对 2019 至 2025 年前沿模型的测量显示，这个时间视野大约每七个月翻一番 ([Kwa et al. - Measuring AI Ability to Complete Long Tasks; METR](#))。

这个指标对本章有两点意义。第一，时间视野是**模型与其 harness 共同具有的属性**，而非只取决于模型本身。交接、checkpoint 和自验证机制可以延长有效时间视野，使系统能够处理超出裸模型自身能力范围的任务。这就是从能力角度观察第 12 章所说的模型与 harness 耦合。

第二，这个指标有助于判断何时值得建设长时运行基础设施。随着模型自身的时间视野增长，一些脚手架会变得多余。Anthropic 就曾随着模型进步，先后移除 context reset 和 sprint 分解（第 7.6 节、第 12 章）。与此同时，值得处理的长时任务边界也会继续向外扩展。因此，harness 工程不会消失，而是转向更困难的问题（第 19 章）。

图：Initializer Agent -> Feature List -> Coding Agent Sessions



要点

- 交接班问题是根本性的：受上下文限制，代理需要结构化交接机制，不能只依赖更大的窗口。
- **Initializer + coding agent** 是实用的长时任务模式：由不同角色分别负责规划和增量执行。
- 干净退出是完成标准的一部分：每个会话都应留下可用的启动路径、更新后的状态工件、验证证据，以及不会拖累下一会话的工作区。
- 分离 **generator** 与 **evaluator** 是提升质量的有力手段：代理评价自己的输出时往往偏向正面，独立 **evaluator** 更可靠。
- **Sprint contracts** 用来协调多个代理：每个构建 sprint 开始前，通过文件沟通并约定成功标准。
- **Context reset** 可以缓解 **context anxiety**：有时，带有结构化交接的全新上下文比 **compaction** 更有效。
- **Managed agents** 将 **brain**、**hands** 和状态解耦：模型上下文、**sandbox** 执行、凭据与 **event log** 应能独立发生故障并恢复。
- **Session**、**harness run** 和 **sandbox** 拥有不同的生命周期：应分别标识并版本化，确保重置、迁移或撤销只影响目标层。
- 自验证是最关键的杠杆：退出前强制执行验证，在模型不变的情况下将分数提高了 13.7 分。
- 持久化执行把可恢复性变成基础设施保证：将每一步写入持久日志，使新代理能在崩溃、上下文耗尽或部署后继续工作；非确定性的模型和工具调用应被视为已记录的副作用，而不是重新计算的步骤。

- 分布式 **durability** 需要 **ownership** 与 **lineage**：single-writer session state、重连、snapshot、幂等 activity 和 trajectory branching 共同把简单 retry 转化为安全恢复。
- 时间视野用来衡量任务到底有多“长”：METR 的时间视野是模型能以 50% 可靠性完成的任务长度，以熟练人类所需时间计；这一指标大约每七个月翻一番。它由模型与 harness 共同决定，因此本章的机制可以延长它。

延伸阅读

- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025.
<https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026.
<https://openai.com/index/harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025.
<https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
- LangChain, *Durable Execution* (LangGraph documentation), 2025.
<https://docs.langchain.com/oss/python/langgraph/durable-execution>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025.
<https://arxiv.org/abs/2503.14499>
- Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026.
<https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>

第 8 章：Loop Engineering（循环工程）

Agent = Model + Harness。第 1 章介绍了每个 agent 的核心循环：组装上下文，让模型发出工具调用，由 harness 执行，将观察结果追加到上下文，然后重复。前面几章关注的是循环内部的各个组成部分：模型能看到什么、可以使用哪些工具、在哪里运行，以及如何跨 session 恢复。本章则把循环本身作为首要的工程单元，讨论什么会启动它、每轮执行什么、由谁检查结果，以及工作何时结束。2026 年，这种实践有了一个名字：*loop engineering*，以及一句口号：不要再逐次 prompt agent，而要构建那个负责 prompt agent 的系统。

8.1 从 Prompting 到 Looping

1.6 节梳理了从 prompt engineering 到 context engineering，再到更广义的 harness engineering 这一演进过程。Loop engineering 是这条路径上的下一步，也让这种转变在实践中变得更加具体。Addy Osmani 在 2026 年 6 月的文章《Loop Engineering》中为这一模式命名，描述了它的典型结构，并引入了许多如今广为使用的术语 ([Addy Osmani — Loop Engineering](#))。同一周，Peter Steinberger 将这个观点浓缩成一句一天内触达数百万人的话：你不应再逐次 prompt coding agent，而应设计那些负责 prompt agent 的 loop ([O'Reilly Radar — Loop Engineering](#))。Claude Code 的构建者 Boris Cherny 则给出了更直白的实践者版本：他不再逐轮 prompt 模型，他的工作是编写驱动模型的 loop ([The New Stack — Loop Engineering](#))。

这种视角转换听起来不大，影响却很深。在 prompt engineering 中，人始终位于循环内部：推进每一步，并判断每个结果。Loop engineering 将人移出这个位置，并提出一个更难的问题：如果现场没有人决定下一步做什么，也没有人判断工作是否足够好，那么应由什么机制来作出这些决定？本章其余内容都在回答这个问题。底层机制仍然是第 1 章介绍的 agent loop，变化的是关注重点：不再只看模型的单个回合，而是看周围那个在更长周期内调度、验证和约束 agent 的控制结构。这正是 harness 的职责所在。Loop engineering 与 harness engineering 关注的是紧密相关的工作，但前者采用的是运维者视角，重点在外层循环。

8.2 Loop 就是带检查的任务

那份实践指南给出了一个很实用的定义：loop 就是带检查的任务；没有检查的任务，只能寄希望于结果碰巧正确 ([The Agentic Loop — A Practical Field Guide](#))。展开来说，一个结构良好的 loop 每轮包含四步：观察当前状态，采取一个有边界的动作，依据固定标准检查结果，然后决定继续还是停止。这个结构揭示了四个设计杠杆。Loop engineering 的主要工作，就是清楚地定义它们：

- **Trigger（触发器）**——什么会启动一轮执行：人给出的目标、schedule、webhook，或另一个 agent。
- **Topology（拓扑）**——loop 如何嵌套和交接：是由单个 agent 执行，由 maker 与 checker 配合，还是由 orchestrator 管理多个 worker。
- **Verifier（验证器）**——以什么固定标准判断“足够好”，以及由谁执行检查。
- **Stop rules（停止规则）**——loop 在什么明确条件下成功、放弃或请求帮助。

其中任何一项没有明确规定，失败模式都可以预见。没有 trigger，loop 就只是一段对话；没有 verifier，它可能把错误结果判为成功；没有 stop rule，它可能一直运行下去，直到耗尽预算。本章余下部分将依次讨论这些杠杆。

8.3 Trigger 与嵌套 Loop

Trigger 让 agent 从一个需要按需调用的工具，变成能够自行运行的系统。Osmani 将它称为 *heartbeat（心跳）*：一种无需人类 prompt 就能唤醒 loop 的 schedule 或事件 ([Addy Osmani — Loop Engineering](#))。从这一步开始，coding agent 就不再只是编辑器，而成为需要纳入运维的系统：cron 可以每晚启动它，webhook 可以在新 issue 到来时启动它，监督 agent 也可以将它作为子任务发起。

Andrew Ng 指出，这些 loop 会彼此嵌套，每层都有不同的负责人和时间尺度。这为理解 loop 的拓扑提供了一张实用的地图 ([Andrew Ng — The Batch, 2026 年 6 月](#))：

- **Agentic coding loop** 以分钟计：给定 spec 和 evals 后，agent 编写代码、运行测试并持续迭代，直到满足 spec；各轮之间不需要人介入。
- **Developer feedback loop** 以小时计：开发者检查已经完成的工作，并指导 agent 接下来做什么。
- **External feedback loop** 以天计：alpha 测试、A/B 测试和生产环境信号揭示产品在实际使用中的表现。

Loop engineering 会最大限度地自动化内层 loop，而人类判断在外层 loop 中仍然不可替代。这是因为人具备 agent 所缺少的上下文优势：人知道工作的真实意图，也理解产品最终要解决什么问题。在 Ng 的例子中，一个 coding agent 无人值守地工作了约一小时，期间多次在浏览器中检查自己的成果，之后才返回请求进一步指导。设计目标是让每层 loop 尽可能长时间地有效运行；只有当它需要更广泛的上下文时，才将控制权交给上一层。

8.4 Verifier 是瓶颈

在四个杠杆中，verifier 最值得投入精力，因为它决定了无人值守运行能否做到安全。第 7 章从另一个角度讨论过同一问题：agent 往往会对自己的工作作出过于正面的判断，因此，将产出工作结果的 agent 与评估结果的 agent 分开，是一项重要的安全措施。Loop engineering 将这一观察提升为设计原则：maker 不能同时担任 checker ([Loop Engineering Crash Course](#))。独立的 reviewer agent 应拿到 spec 而不是 diff，并被要求以怀疑的态度审查结果。这样，它才有机会发现 generator 可能自行合理化并忽略的问题。因此，7.4 节的 generator-evaluator 拆分和第 6 章的 evaluator-optimizer 工作流不再只是实用技巧，而是决定 loop 能否安全独立运行的关键。

社区里的一句口号概括了这种重心转移：编写 verifier，就是新的 prompt engineering ([The Agentic Loop — A Practical Field Guide](#))。真正困难而有价值的工作，不再只是组织请求的措辞，而是把“完成”定义得足够精确，让机器能够识别。由此可以区分两类 loop：

- **Closed loop (闭环)** 预先将验收标准设为明确、可检查的通过条件，例如测试全部通过、schema 校验成功或截图匹配。它的预算可预测，适合无人值守运行。
- **Open loop (开环)** 面向不够精确的目标进行探索。它更需要强有力的 verifier，因为缺少 verifier 时，它不会以明显的方式失败，反而可能反复、自信地产出看似合理却并不正确的结果。

在任务允许的范围内，验证应尽量采用机械、确定性的方式。优先使用测试、类型检查、schema 校验和浏览器断言；只有无法确定性检查的属性，才交给 LLM-as-judge。这对应第 5 章“计算型先于推断型”的排序，也对应第 10 章的 grader 分类。更可靠的做法是使用一个全新的模型，让它不了解工作结果的生成过程，从而减少继承 maker 盲点的可能。

8.5 Stop Rule 与三项硬限制

能够自行启动的 loop，也必须能够自行停止，而且停止原因不能只有“成功”。每个结构良好的 loop 都需要明确处理四种结果：success、no-op（没有剩余工作）、ask-for-approval，以及 blocked-or-exhausted（受阻或资源耗尽）。此外，还需要三项不可缺少的硬限制，用来约束失控的执行过程 ([The Agentic Loop — A Practical Field Guide](#))：

1. **最大迭代次数**——设置硬上限，例如“测试全部通过，或最多执行六轮，以先达到的条件为准”。
2. **无进展检测**——如果连续 N 轮没有产生可测量的变化，就停止执行，而不是继续空转。
3. **预算上限**——设置 token 或金额上限；达到上限后，loop 停止并请求指示。

这些限制就是从 loop 内部观察第 17 章所说的 per-task budget。它们在这里尤其重要，因为现场没有人能够及时发现执行过程正在空转。失败并非理论风险：Uber 的一个团队曾因某个无人值守系统在四个月内耗尽全年 AI 预算，之后将 agent 的支出上限设为每月 \$1,500 ([AI Builder Club — Loop Engineering Guide](#))。Ask-for-approval 则构成了通往第 15 章的桥梁：将升级请求建模为工具调用后，loop 可以挂起，把决定交给人类，并在人类回应后从 event log 恢复。这仍是同一种持久化 approval 模式，只是现在成为 loop 遇到重大决策时的指定出口。

8.6 Ralph 谱系

Loop engineering 并非突然出现，而是这个领域自 2022 年以来逐步演进的最新阶段 ([The Agentic Loop — A Practical Field Guide](#))：

- **ReAct** (2022) 确立了 reason-act-observe 的基本循环，也就是第 1 章所说的 agent loop ([Yao et al. — ReAct](#))。
- **AutoGPT** (2023) 使 loop 能够围绕目标自主运行，同时也暴露了这门工程学科如今着力防范的问题：没有 verifier 和 stop rule 的 loop，可能无限运行或偏离目标。
- **“Ralph Wiggum” loop** (2025) 加入了 7.5 节介绍的关键修正：每次迭代都从干净的 context window 开始，将状态保存在磁盘文件而不是不断膨胀的历史中，并重新注入目标，让 agent 始终围绕目标工作。
- **可验证完成命令** (2026)，例如由独立 validator 模型决定能否退出的 `/goal`，将 stop rule 变成一项由机器检查的一等步骤，而不再依赖 agent 对自身工作的判断。
- **Orchestration** (当前) 让 loop 可以监督其他 loop：它们由系统调度，以 git 为持久化基础，并在 Ng 所描述的嵌套时间尺度之间上下交接工作。

这个演进过程通过两个观点与本书其余内容相连。第一，Ralph 的 reset 解释了为什么 *记忆应保存在磁盘，而不是上下文中* (第 2–3 章)：每轮都会 reset 的 loop 必须从文件重新加载状态，这正是那几章所介绍的结构化笔记和反复复述模式。第二，可验证完成解释了为什么 *event log 至关重要* (第 9 章)：当 loop 状态存储在追加式 log 中时，loop 才能停止、恢复和重放，这些能力使长周期自治变得可调试。

8.7 Loop 的组成

Osmani 总结了持久化 loop 的组成部分。每一部分都对应前文介绍过的一项能力 ([Addy Osmani — Loop Engineering](#))：

- **Heartbeat**——触发一轮执行的 schedule 或事件 (8.3 节)。
- **Worktrees**——彼此隔离的工作目录，避免并行执行相互冲突；其基础是第 5 章介绍的 sandbox 隔离。

- **Skills**——可复用、以文件为载体、编写一次并按需加载的项目知识，也就是第 4 章的 `SKILL.md` 模式。
- **Connectors**——连接工作所依赖的实际工具，包括 MCP server 和插件（第 4 章）。
- **Sub-agents**——分别担任 maker 和 checker 的 agent（8.4 节、第 3 章）。
- **Spine（主干状态）**——能够跨多次运行持续存在，并在 reset 之间保存 loop 记忆的状态文件（第 2–3 章，以及 7.6 节的结构化 handoff）。

Loop 的效果取决于它所操作的代码库。因此，实践者提出，仓库在适合 loop 工作之前需要具备三项属性 ([AI Builder Club — Loop Engineering Guide](#))。第一是**清晰易读 (legible)**：精简的 `AGENTS.md` 索引和定制 lint 应让 agent 了解代码库的组织方式，以及哪些内容不能修改。第二是**可执行 (executable)**：dev server 应能以接近零 token 的成本启动，并支持并行 worktree。第三是**可验证 (verifiable)**：end-to-end 测试和浏览器驱动检查应覆盖核心流程，为 8.4 节的 verifier 提供可机械判定的依据。这些都属于第 5 章所说的**环境赋能条件 (ambient affordances)**；对于自治工作，它们是前提，而非锦上添花。

8.8 成熟度阶梯与无人值守的风险

由于 loop 会通过重复执行不断放大自身行为，采用过程应当分阶段推进。社区提出的成熟度阶梯要求一次只提升一个级别，并且只有当前级别已经能够可靠完成原本需要人工处理的工作时，才进入下一级 ([The Agentic Loop — A Practical Field Guide](#))：

0. **Manual（手动）**——每一轮都由你发出 prompt。
1. **Triage（分诊）**——loop 将发现写入 markdown 文件，但不作任何修改。
2. **Draft（草稿）**——loop 在隔离分支中进行修复。
3. **Verified PR（已验证 PR）**——先由独立 verifier 检查变更，再交给人 review。
4. **Auto-merge（自动合并）**——只留给低风险类别。

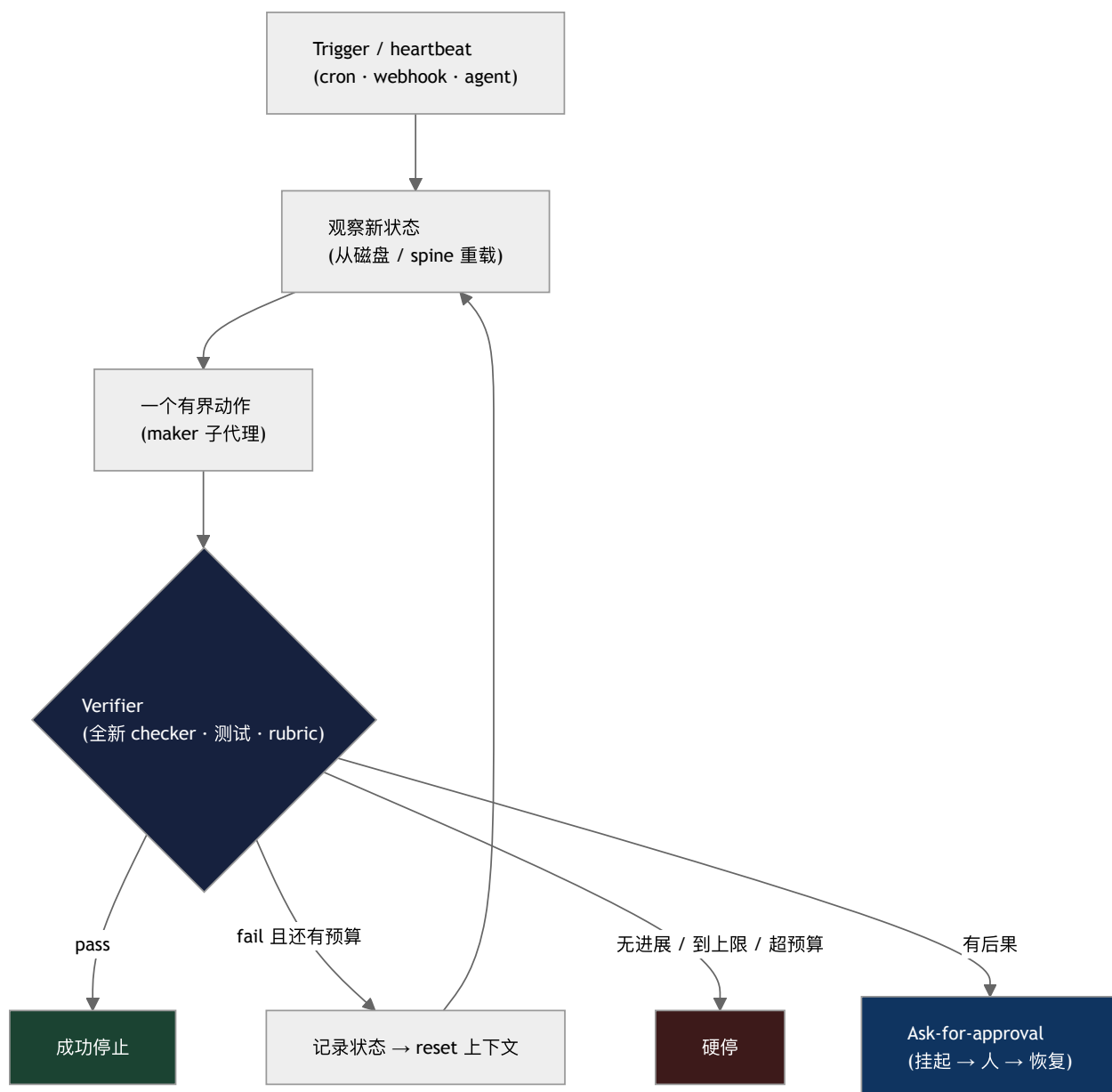
Loop engineering 并没有消除困难，只是改变了困难所在的位置。速度既会放大成果，也会放大错误：无人值守的 loop 同样会在无人值守时犯错，而且它提交代码的速度可能超过人类审查代码的速度。由此会积累 *comprehension debt*（理解债），使代码库的所有者逐渐无法完全理解其中的实现 ([The New Stack — Loop Engineering](#))。因此，人类仍需在两个不可替代的端点承担责任：定义什么结果算“好”的意图，以及对最终发布内容承担的**所有权** ([Loop Engineering Crash Course](#))。这给出了一条实用的范围规则：结构化 loop 适合**重复、无人值守、定时触发且后果重要的工作**。如果你本来就在交互式地观察 agent，那么人的判断已经构成检查机制，普通对话通常更合适。为一次性任务过度设计 loop，本身也是一种失败模式。

8.9 超越单个 Loop：控制与 Evaluator Integrity

对于单个自治任务，loop engineering 是恰当的设计单元。然而，生产系统最终会在不同身份、版本、预算和策略下运行许多 loop。达到这种规模后，下一个需要设计的对象是**控制平面 (control plane)**：负责注册 agent、授予权限、调度和撤销运行、记录 lineage 并治理整个 fleet 的系统。第 18 章将详细讨论这一层。

Verifier 本身同样需要治理。即使 checker 与 maker 分开，它仍可能因为知道判断会带来什么后果而产生偏差，也可能与 maker 具有相同的盲点，或被其他系统针对其判定规则进行优化。因此，第 10 章将 **evaluator integrity**——盲评、确定性证据、校准、弃权和审计——作为独立于“设置 verifier”的另一项问题。只有检查机制本身值得信任时，一个 engineered loop 才真正形成闭环。

图示：被工程化的 loop



本章要点

- **Loop engineering** 是从外层循环看 **harness engineering**：不要逐轮 prompt agent，而要设计负责 prompt agent 的系统，并规定由什么机制判断工作已经足够好。
- **Loop** 是带检查的任务：观察 → 执行一个有边界的动作 → 依据固定标准验证 → 继续或停止。没有检查的任务，只能寄希望于结果碰巧正确。
- 四个杠杆定义一个 **loop**：trigger、topology、verifier 和 stop rules。其中任何一项没有明确规定，都会产生可预见的失败模式。
- **Loop** 会彼此嵌套：agentic coding 以分钟计，developer feedback 以小时计，external feedback 以天计。自动化内层 loop，同时保留外层 loop 中的人类判断。
- **Verifier** 是瓶颈：编写 verifier 就是新的 prompt engineering。Maker 不能同时担任 checker；薄弱的检查机制可能在没有明显报错的情况下接受看似自信却不正确的工作。
- **Stop rule** 不可缺少：设置最大迭代次数、无进展检测和预算上限，因为现场没有人能够及时发现失控的执行过程。
- **Ralph** 谱系展示了模式的演进：ReAct → AutoGPT → 干净上下文 reset → 可验证完成 → orchestration。记忆保存在磁盘，状态记录在 event log 中。
- **成熟度应逐级提升**：从 triage 到 draft，再到 auto-merge，并且只将更高自治级别用于重复、无人值守且后果重要的工作。无人值守的速度会同时放大错误和理解债。

- 单个 **loop** 之后的下一个设计单元是 **fleet**：身份、生命周期、策略、lineage 和撤销属于控制平面；verifier 的可信度则属于评测系统。

延伸阅读

- Addy Osmani, *Loop Engineering*, addyosmani.com, 2026 年 6 月。 <https://addyosmani.com/blog/loop-engineering/>
- *Loop Engineering*, O'Reilly Radar, 2026。 <https://www.oreilly.com/radar/loop-engineering/>
- Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, 2026 年 6 月。
<https://www.deeplearning.ai/the-batch/>
- *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026。
<https://thenewstack.io/loop-engineering/>
- *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026。 <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
- *Loop Engineering Guide (2026)*, AI Builder Club。 <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
- Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, 2022 年 10 月。
<https://arxiv.org/abs/2210.03629>

第 9 章：生产级 Agent 的十二要素

前几章分别讨论了上下文管理、工具、沙箱、工作流，以及长周期任务的 handoff。本章将视角拉远，考察这些 harness 技术如何融入常规的软件架构。HumanLayer 的“12 Factor Agents”最适合被理解为一份生产实践清单。它借用了经典 Twelve-Factor App 的名称，但其中的要素专门面向 LLM agent。它是一份原则宣言，而不是完整的参考架构。

本章建立在两个关键的软件概念之上。状态是继续执行所需的全部信息，包括当前步骤、重试次数、审批、用户消息、工具结果，以及此前受到影响的业务对象。事件日志是一份只追加、不覆写的记录，可以据此重建状态。将 agent 建模为作用于这些事件的 reducer 后，暂停与恢复、重放、调试和测试就会成为普通的软件问题，而不再依赖隐藏在对话中的状态。

9.1 作为软件架构的十二要素

这十二条原则总结自许多生产部署 (HumanLayer - 12-Factor Agents)：

1. **Natural Language to Tool Calls**：基本模式是将用户请求转换成结构化 JSON call，再交给确定性代码执行。
2. **Own Your Prompts**：不要把 prompt engineering 交给 framework 黑箱。应将 prompt 视为一等代码，以便测试、评估和调优。
3. **Own Your Context Window**：标准 message format 是一种选择；另一种选择是使用带 XML 标记的 event log，将历史记录压缩到一条 user message 中。无论采用哪种方式，目标都是用尽可能少的 token 保留尽可能多的有效信息。
4. **Tools Are Just Structured Outputs**：工具调用本质上是模型生成的结构化 JSON，其中包含意图及其参数；随后由确定性代码决定如何执行。
5. **Unify Execution State and Business State**：不要将“当前步骤/下一步骤/retry count”和“对话中发生了什么”分别维护，而应从同一份 event log 推导执行状态。
6. **Launch / Pause / Resume with Simple APIs**：agent 是程序，因此应支持标准 lifecycle 操作，包括在工具选择之后、实际执行之前暂停。
7. **Contact Humans with Tool Calls**：不要依赖模型自行决定使用普通文本还是结构化输出。应提供明确的 `request_human_input` 工具，并设置 urgency、format、choices 等结构化字段。
8. **Own Your Control Flow**：主动控制 loop，以便暂停并等待审批、总结工具结果、用 LLM-as-judge 检查输出、管理记忆、记录日志和 trace、实施限流，或进行持久休眠 (durable sleep)。
9. **Compact Errors into Context Window**：保留可见的错误信息，让 agent 能够尝试自我恢复；同时使用连续错误计数器，在达到阈值后升级给人类处理。这里的关键词是 compact：原始 stack trace 会迅速消耗 token 预算，并加剧 context rot（见《LLM Foundations》第 9 章）。Context 中应只保留最新错误或有用的摘要，并折叠或删除更早的重复 trace，避免它们挤占继续工作所需的信息。
10. **Small, Focused Agents**：将每个 agent 的工作范围控制在约 3–10 步，最多可以放宽到 20 步。上下文越大，性能通常越差。
11. **Trigger from Anywhere**：允许通过 Slack、email、SMS、webhook 和 cron job 启动 agent。它与 factor 7 结合后会形成 outer loop：事件负责启动 agent，agent 则在遇到关键决策点时联系人类。
12. **Make Your Agent a Stateless Reducer**：将 agent 建模为对 events 执行 fold 的纯函数，使其可以序列化和重放。只有当每次 LLM 响应和工具结果都记录为 event 时，重放才是确定性的。恢复时，应对已记录的结果执行 fold，而不是再次调用模型或重新运行具有副作用的工具。

贯穿这些要素的核心主张是：“好的 agent 至少不是‘给你一个 prompt、一袋工具，然后循环到目标完成’的模式。它们大多只是软件” (HumanLayer - 12-Factor Agents)。换句话说，这些要素是在将熟悉的软件工程纪律应用于一个有状态、非确定性的组件。它们并不是放之四海而皆准的定律：研究原型、本地 coding assistant 和受监管的客服 agent 各有不同的权衡。真正重要的是实践方向：让状态显式可见，让控制流可以检查，并通过结构化接口处理人类交互。

9.2 从 Agent Program 到 Agent Platform

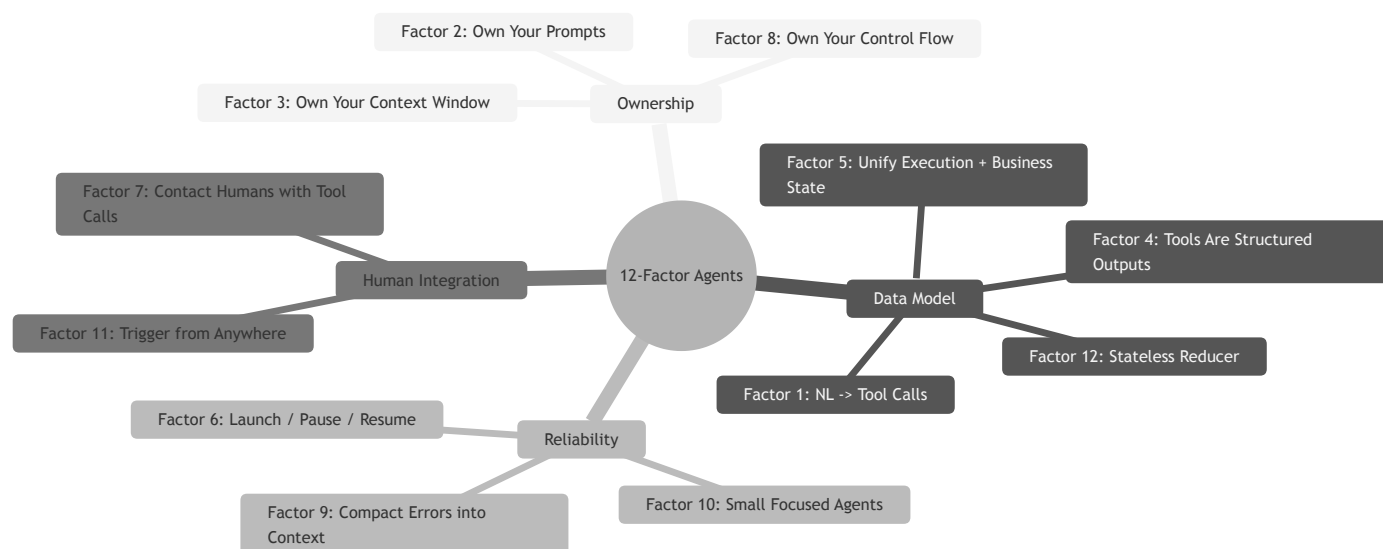
OpenReview 的综述指出，整个生态正在从 agent framework 走向 agent platform (OpenReview - Agent Harness Engineering: A Survey)。Framework 封装的是 agents、tools、memory stores 和 loops 等本地抽象。Platform 则增加了共享基础设施，为多次运行和多个用户提供 durable workspace、managed sandbox、identity、billing、observability、evaluation、governance 和 human handoff。

这种转变不会取代十二要素，而是扩大它们的适用范围。Launch / pause / resume 变成平台 API；Unify execution state and business state 变成带 tenancy 和 migration 语义的 event-log storage；Contact humans with tool calls 变成能够追踪 permission state 和 audit history 的 handoff interface；Own your control flow 则要求明确决定哪些检查同步运行、哪些离线运行，以及哪些失败值得启动成本较高的恢复流程。

平台边界也会改变责任划分。本地 agent 或许可以使用临时的 state files，共享平台却需要明确的状态所有权、保留策略、billing attribution、credential scoping 和可重放的 audit trail。在这一规模上，harness 不再只是包围单次模型调用的一层结构，而是管理多个 agents、多个 environments 和多方人类参与者的控制系统。

这个控制系统并不只是另一个 agent framework，而是第 18 章展开的 **agent control plane**：registry 记录可用的 agents 和 capabilities；identity layer 记录谁在代表谁行动；policy enforcement 决定每次运行可以做什么；lifecycle、lineage 和 audit service 则跨 session 运作。十二要素仍然是每个 agent program 内部的设计纪律，而控制平面使由这些 program 组成的 fleet 具备可治理性。

图：按主题分组的十二要素



本章要点

- “大多只是软件”：好的 agent 主要是由确定性软件包围一个非确定性的 LLM 组件，而不是简单地把一袋工具交给模型，让它循环到完成为止。
- **Own your prompts**：不要让 framework 隐藏 prompt；应将 prompt 作为一等代码纳入版本控制。
- **使用 stateless reducer 模式**：对 event log 执行 fold 以得到当前状态，使 agent 可以序列化、重放和测试。
- 平台范围会改变十二要素：lifecycle、state、identity、billing、observability 和 human handoff 会变成共享基础设施问题。
- **Fleet 需要控制平面**：十二要素塑造每个 agent program；registry、identity、policy、lifecycle 与 audit 治理整个集合。
- **保持 agent 小而聚焦**：将每个 agent 限制在 3-20 步，因为性能通常会随着上下文增长而下降。
- **通过工具联系人类**：结构化的 `request_human_input` 工具，比依赖模型选择自由文本更可靠。
- **压缩错误，而不是隐藏错误**：可见的错误信息支持自我恢复，连续错误计数器则提供安全的升级路径。

延伸阅读

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

第 10 章：评估

10.1 为什么需要 Evals

没有 eval，调试只能被动进行：等用户投诉，手动复现问题，完成修复，然后寄希望于没有引入新的回归 ([Anthropic - Demystifying Evals for AI Agents](#))。团队既无法可靠地区分真实回归与随机波动，也无法用大量场景检验一次改动，更难判断系统究竟有没有改善。模型升级也会因此变慢。没有 eval，采用新模型可能需要数周的人工测试；有了 eval，团队则可以在几天内验证模型优势并调整 prompt。

关键在于评估真正重要的单元。对 agent 来说，这个单元是完整的 loop，而不是某一次模型调用（见《LLM Foundations》第 13 章）。Agent eval 会在具体环境中运行整个 model + harness 系统，再检查最终状态是否满足任务要求。这个区别很重要：agent 可能通过了某项中间测试，给出流畅的回答，甚至走了一条出人意料但看似合理的路径，却仍然没有实现用户真正的目标。

Anthropic 将 eval 视为一种具有复利效应的基础设施：前期投入清晰可见，收益则会在 agent 的整个生命周期中持续累积。实践建议是尽早开始，即使手头只有 20–50 个简单任务也可以。Agent 开发初期的改动通常影响较大，小样本就能提供有效信号；随着 agent 逐渐成熟、改进幅度变小，就需要更大的 eval 集才能识别差异。

10.2 Evaluation 的结构

《LLM Foundations》第 13 章已经介绍了 eval 的核心概念，包括 code-based、model-based 和 human grader，trace，regression eval 与 capability eval，pass@k 与 pass^k，以及 reward hacking。本章先简要回顾这些概念，再讨论建立在它们之上的 harness 实践。

Anthropic 的词汇 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **Task**：一项输入和成功标准都已明确的任务。
- **Trial**：对某个 task 的一次尝试。由于 agent 的输出存在随机性，一项 task 通常需要运行多次 trial。
- **Grader**：评估某个性能维度的评分器。一项 task 可以配置多个 grader，每个 grader 都有自己的 assertions。
- **Transcript**（也称 trace 或 trajectory）：一次 trial 的完整记录。
- **Outcome**：trial 结束时环境所处的最终状态，它不同于 agent 给出的文本回应。例如，订票 agent 说“机票已订好”只是 response；SQL 数据库中是否真的出现对应记录，才是 outcome。
- **Evaluation harness**：端到端运行 eval 的基础设施，需要与 agent harness 区分。
- **Agent harness**（或 scaffold）：与模型一起接受评估的系统。换句话说，“当我们评估一个 agent 时，评估的是 harness 与模型如何协同工作。”

10.3 三类 Grader

- **Code-based**：包括字符串匹配、通过/失败测试、静态分析、outcome verification、tool-call verification 和 transcript analysis。这类 grader 速度快、成本低、客观且可复现，但也可能误判合理的结果变体。
- **Model-based**：包括 rubric scoring、自然语言断言、pairwise comparison 和多 judge 共识。这类 grader 灵活、易扩展，适合开放式任务，但结果并不确定，需要依据人类判断进行校准。
- **Human**：包括领域专家评审、众包判断和 A/B testing。人工评估最适合做校准和主观判断，但成本高、速度慢；如果 rubric 不清晰，人类之间同样可能意见不一。

Anthropic 建议尽可能使用确定性 grader，只在必要时使用 model-based grader，并定期通过人工评估做校准。它还提醒，不要评判 agent 采取的路径，而应评估它最终得到的结果。Agent 经常会找到 eval 设计者没有预料到、但同样有效的方法；如果要求它必须走某条固定路径，eval 就会变得脆弱。

10.4 谁来验证 Verifier? Evaluator Integrity

把产出者 (maker) 与检查者 (checker) 分开，可以消除一种利益冲突，却不会让 checker 自动变得中立。Anthropic 在 2026 年对**动机性错误标注 (motivated mislabeling)** 的研究中发现：如果担任 evaluator 的模型知道一项判断会如何被使用，它可能因此改变标签。例如，当负面标签会触发删除、惩罚或其他后果时，模型的判断就可能受到影响。更严格的 rubric 和允许 abstain 可以减轻这种现象，但无法将其彻底消除 ([Anthropic - Agentic Misalignment: Summer 2026 Update](#))。

这种风险远不只存在于安全研究中。如果 judge 知道哪个候选是当前方案、它出自哪个团队，或者失败判定是否会阻止部署，就可能顺着预期后果为自己的判断找理由。负责生成候选结果的模型 (generator) 也可能学会迎合某个已知 judge，只优化对方偏好的表面特征。因此，**evaluator integrity** 是 harness 自身的一项属性，需要专门的控制措施：

- 优先使用确定性 outcome check，并保留每项结果背后的原始证据；
- 向 model judge 隐藏候选身份、部署后果和其他无关 metadata；

- 允许 judge 给出“证据不足”的结论，并把后果重大的模糊判断交给人工评审；
- 使用专家标注集校准 judge，并运行 **meta-eval**，检查 evaluator 本身是否存在 bias、leakage 或 reward hacking；
- 对高风险语义决策使用相互独立的 judge 或 ensemble，同时注意：相关模型之间达成一致，并不能构成充分证明；
- 对 rubric、judge model、prompt 和 evidence 进行版本管理，并保留不可变的 audit trail，使判决能够复现，也能够接受质疑。

OpenAI 关于可信第三方评测的指导，将同一个原则扩展到了制度层面：独立性、方法透明度、任务代表性、利益冲突披露和可复现产物，本来就是评估质量的组成部分，而不是算出分数之后再补做的文书工作 ([OpenAI - Trustworthy Third-Party Evaluations](#))。Verifier 本身也是被测系统的一部分。

10.5 Capability Eval 与 Regression Eval

Agent eval 通常服务于两种不同的目的 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **Capability evals** 问的是：“这个 agent 能把哪些事情做好？”它会有意纳入 agent 尚不擅长的任务，因此初始通过率往往较低，也为团队提供了明确的改进目标。
- **Regression evals** 问的是：“这个 agent 还能完成以前已经解决的任务吗？”它的通过率应当保持在接近 100% 的水平，用来防止系统能力倒退。

随着 agent 逐渐成熟，那些通过率持续较高的 capability eval 会 *graduate* 到 regression suite。原本用来衡量“到底能不能做到”的任务，之后就会转而衡量“现在是否仍能稳定做到”。

10.6 pass@k 与 pass^k

由于 agent 每次运行的行为都可能不同，评估时常用两个随 trial 数量增加而朝相反方向变化的指标 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **pass@k**: k 次尝试中至少得到一次正确结果的概率。k 越大，成功机会越多，因此这个指标会随之上升。
- **pass^k**: k 次 trial 全部成功的概率。要求更多次 trial 都保持成功，标准会越来越严格，因此这个指标会随 k 增大而下降。

之所以需要这两个指标，是因为 agent 的运行具有随机性。同一个 prompt、模型和 harness，在不同 trial 中可能采用不同的工具顺序和搜索路径，也可能给出不同的最终答案。因此，单次运行只能提供很弱的证据；重复运行才能看出系统是偶尔成功、能够持续稳定地成功，还是只碰巧走通了一条脆弱的路径。

如果单次 trial 的成功率是 75%，那么 pass^3 约为 42%，pass^10 约为 5.6%，而 pass@10 约为 99.9999%。应该使用哪项指标取决于产品形态：如果系统可以生成多个候选，再选择或展示其中最好的结果，pass@k 更有参考价值；如果面向客户的 agent 必须在重复执行时都保持可靠，则应更关注 pass^k。

10.7 八步路线图

Anthropic 将从没有 eval 到建立可信 eval suite 的过程概括为以下路线图 ([Anthropic - Demystifying Evals for AI Agents](#)):

0. **尽早开始**：从真实失败中收集 20–50 个任务。
1. **先采用现有的人工测试**：例如发布前检查和 bug tracker 中的问题。
2. **编写无歧义的任务，并提供参考解**：两位领域专家应当能够得出相同结论。如果大量 trial 的通过率都是 0%，问题通常出在 task 本身，而不一定说明 agent 没有能力完成。
3. **构建平衡的问题集**：既要包含某种行为应该出现的场景，也要包含它不该出现的场景。单侧 eval 会诱导系统朝单一方向优化。
4. **建立稳定可靠的 eval harness**：隔离各次 trial，避免共享状态。Anthropic 曾观察到 Claude 通过查看之前 trial 遗留的 git history 获得了不公平优势。
5. **谨慎设计 grader**：能用确定性检查时就优先使用；多组件任务应给予部分分；按照结构化 rubric 校准 LLM judge；提供 “Unknown” 选项以减少幻觉；同时防范 reward hacking。
6. **阅读 transcript**：检查失败案例时，它们应当显得公平。如果分数停止提升，需要判断究竟是 agent 出现回归，还是 eval 本身已经不再公平。
7. **监控 capability eval 是否饱和**：通过率达到 100% 的 eval 已无法继续提供改进信号。SWE-bench Verified 从 30% 起步，目前已接近 80%；在这个阶段，看似很小的分数提升也可能代表显著的能力进步。
8. **通过开放参与维护 eval suite**：领域专家和产品团队都应贡献 eval task。产品经理、客户成功团队和销售人员也可以使用 Claude Code，把 eval 作为 pull request 提交。

10.8 不同 Agent 类型的真实 Evals

合理的 eval 设计取决于 agent 类型。以下代表性例子来自 Anthropic 的综述 ([Anthropic - Demystifying Evals for AI Agents](#)):

- **Coding agents**: 很适合使用确定性 grader, 例如检查代码能否运行、测试能否通过。SWE-bench Verified 会运行与固定 GitHub issue 对应的测试套件; Terminal-Bench 则评估端到端任务, 例如从源代码构建 Linux kernel。
- **Conversational agents**: 成功标准通常包含多个维度, 例如工单是否解决 (状态检查)、对话是否少于 10 轮 (transcript constraint)、语气是否恰当 (LLM rubric)。这类 eval 往往还会使用第二个 LLM 模拟用户, 例如 τ -Bench 和 τ^2 -Bench。
- **Research agents**: 需要检查论断是否有来源支持 (groundedness)、关键事实是否完整 (coverage), 以及来源是否权威, 而不只是最先检索到的结果 (source quality)。相应的 grader 需要经常与人类专家的判断进行校准。
- **Computer-use agents**: 需要真实环境或 sandbox, 并检查 URL、页面状态和后端状态。例如, 要验证订单是否真的创建, 而不能只看 agent 是否到达了确认页面。WebArena 和 OSWorld 是这类 eval 的典型例子。

10.9 面向 Coding Agent 的验证反馈

对 coding agent 来说, 最有用的 grader 往往也能直接提供修复线索。只返回 “test failed” 的检查虽然确认结果有问题, 却很难帮助 agent 纠正错误。有效的失败信息应指出受影响的路径、期望状态与实际状态, 以及下一步应该检查的位置。OpenAI 的 Codex harness 指南建议, 把反复出现的 review 意见和架构规则转化为 repo-local 检查。这样, agent 就能在仍有机会修复问题时得到具体反馈 ([OpenAI - Harness Engineering](#))。

端到端验证应当成为完成任务的门槛, 而不是象征性的最后一步。在 Anthropic 的长时间运行应用 harness 中, coding agent 必须启动应用, 并通过浏览器驱动的工作流验证功能。如果没有这项要求, agent 往往会在本地测试通过或完成视觉检查后就宣称任务结束, 即使真实的用户流程仍然存在故障 ([Anthropic - Effective Harnesses for Long-Running Agents](#))。第 10.2 节的通用原则在这里同样适用: 评估环境状态, 而不是 agent 的自信。

10.10 阅读 Transcript 是核心技能

Eval 工作中有一条反复出现的经验: 在有人读过 transcript 之前, 不要照单全收分数。Anthropic 曾介绍一个案例: Opus 4.5 在 CORE-Bench 上的初始得分只有 42%。调查发现, grader 会因为模型把期望答案 96.124991... 写成 96.12 而判错; 此外, 任务说明存在歧义, 一些随机任务也无法精确复现。修复这些评分问题, 并使用限制更少的 scaffold 后, 得分升至 95% ([Anthropic - Demystifying Evals](#))。METR 在 time-horizon benchmark 中也发现了类似问题: 有些任务要求 agent 把结果优化到指定阈值, grader 却要求必须超过该阈值。结果是, 遵循指令的模型受到惩罚, 忽略指令的模型反而得到奖励 ([Anthropic - Demystifying Evals](#))。

通用原则是: 人工检查失败案例时, 判定应当显得公平。当分数进入平台期, 需要判断究竟是 agent 已经停止进步, 还是 eval 已经偏离了原本要衡量的能力。

10.11 Evals 只是多层体系中的一层

自动 eval 只能提供部分信息。Anthropic 借用安全工程中的瑞士奶酪模型来说明这一点: 每一层都有缺口, 因此没有任何一层能够捕获所有问题 ([Anthropic - Demystifying Evals](#))。完整的评估体系包括:

- **自动 eval**: 支持快速迭代、回归检测和模型升级。
- **生产监控**: 提供真实依据 (ground truth), 并发现未曾预料的现实故障。
- **A/B testing**: 在流量足够时验证重要改动。
- **用户反馈**: 暴露设计者没有预想到的问题。
- **人工 transcript review**: 帮助团队建立对失败模式的直觉。
- **系统性人类研究**: 用于校准 LLM grader, 以及评估主观性较强的输出。

10.12 Readiness Validation 与失败归因

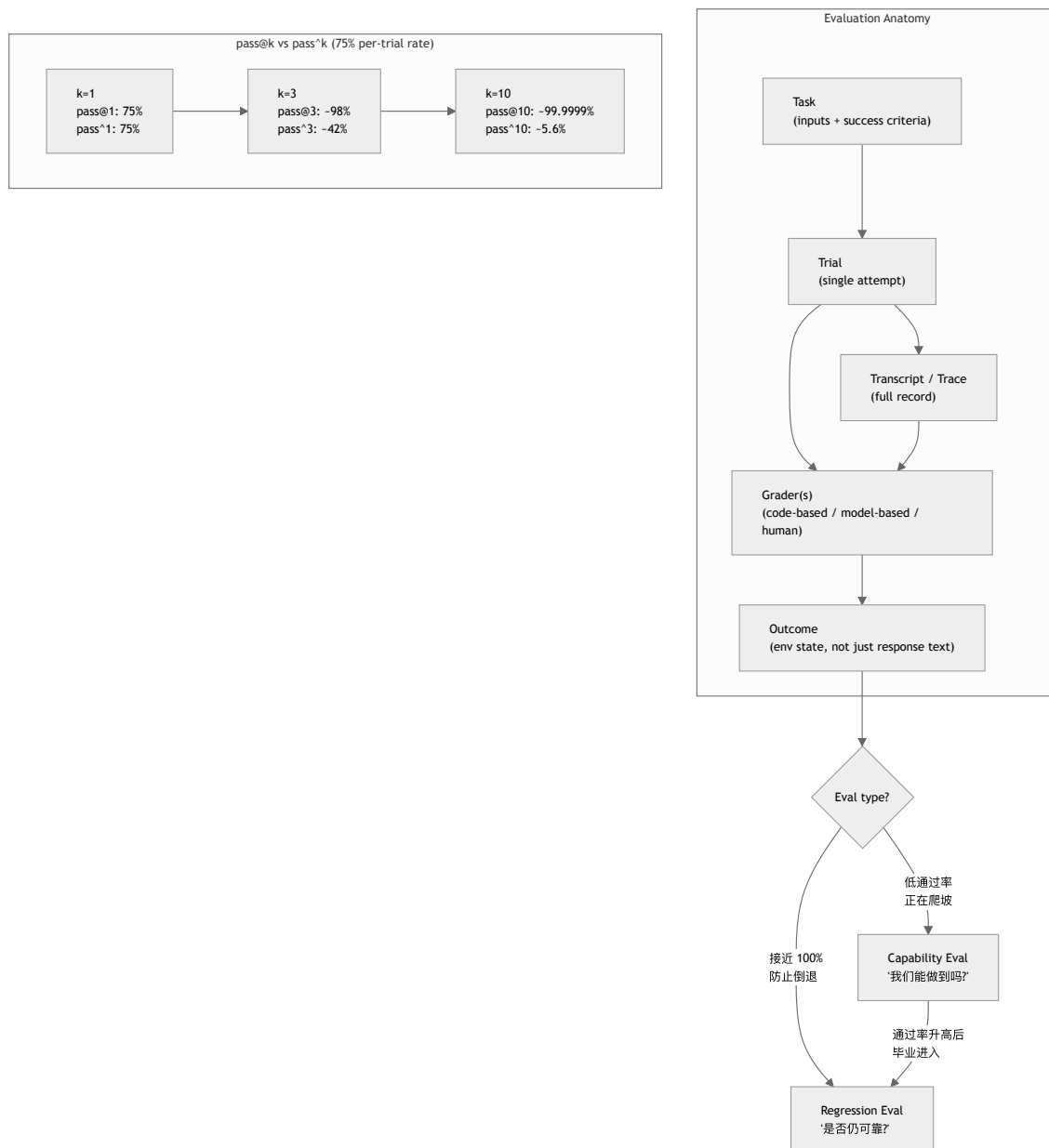
OpenReview 综述将 **Verification** 与一般意义上的 evaluation 分开, 是因为 harness 需要的不只是分数。Verification 要回答的是: 某个特定的 model + harness 组合, 在明确的任务分布、环境、预算和治理制度下, 是否已经适合部署 ([OpenReview - Agent Harness Engineering: A Survey](#))。

这个视角为普通 eval suite 增加了两项实践要求:

- **Readiness validation**: 发布门槛应当同时绑定任务、环境重置规则、可用工具、上下文策略、预算限制和治理检查。在不同 execution substrate 上得到的分数, 或使用不同 tool menu 得到的分数, 并不能直接沿用。
- **失败归因**: 应把每次失败归到最可能出问题的层, 例如 execution、tool interface、context、lifecycle、observability、verification 或 governance。如果缺少这一步, 团队可能会不断调整 prompt, 而真正的缺陷其实是不稳定的 sandbox、过大的工具面、缺失的 checkpoint, 或者薄弱的 policy hook。

这种配置依赖性也解释了为什么 benchmark 数字十分脆弱。基础设施变化、成本优化和工具边界调整，都可能改变同一个模型测得的能力。因此，一份有用的 eval report 除了模型名称，还应记录产生结果时的 harness 配置，包括环境镜像、资源限制、工具目录、上下文组装策略、retry 规则、grader 和 human-approval 要求。

图：Eval 结构与 pass@k / pass^k



要点

- **Eval** 是具有复利效应的基础设施：即使 agent 尚未成熟，也可以先从真实失败中整理出 20-50 个任务。
- **Evaluation** 的范围比单元测试更广：它会在具体环境中检验 model + harness 系统是否真正实现了任务 outcome。
- **Outcome** 不等于 response：应测量环境状态，例如数据库记录、URL 或文件，而不能只看 agent 声称做了什么。
- 面向修复的反馈有助于 agent 自我纠正：检查应说明哪里失败、发生了什么，以及什么证据能够证明问题已修复。
- 三类 grader 构成一座金字塔：code-based grader 提供速度，model-based grader 处理细微判断，人工评估负责校准。
- **Verifier** 也是被测系统的一部分：应向 judge 隐藏无关后果、保留证据、用 meta-eval 做校准、允许 abstain，并保留可复现的 audit artifact。
- **pass@k** 与 **pass^k** 适用于不同产品：能生成多个候选的系统可以关注 pass@k；需要反复面向客户执行的 agent 更应关注 pass^k 式可靠性。
- 阅读 transcript 是核心能力：分数停止提升，可能是 agent 出现回归，也可能是 eval 不公平；只有 transcript 能帮助团队区分两者。

- **Readiness 与配置绑定**：解释 eval 结果时，必须同时考虑产生该结果的 harness 配置。
- **Eval 只是多层体系中的一层**：还需要结合生产监控、A/B testing、用户反馈和人工评审。

延伸阅读

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Ken Aizawa, *Writing Effective Tools for Agents - with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Anthropic Safeguards Research Team, *Agentic Misalignment: Summer 2026 Update*, 2026. <https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>
- OpenAI, *Trustworthy Third-Party Evaluations: Foundations*, 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>

第 11 章：基础设施噪声

Benchmark leaderboard 上的微小分差，往往不像小数点后的数字所暗示的那样确定。Anthropic 的“Quantifying Infrastructure Noise”展示了这种不确定性可能有多大 ([Anthropic - Quantifying Infrastructure Noise in Agentic Coding Evals](#))。

静态 benchmark 直接对模型输出评分，agentic coding eval 则不一样：模型需要编写程序、运行测试、安装依赖，并在多轮交互中不断调整方案。因此，runtime 不是一个被动的容器，而是求解过程的一部分。资源预算不同的两个 agent，实际上参加的并不是同一场考试。

11.1 主要结果

Anthropic 在 Google Kubernetes Engine 集群上，以六种资源配置运行了 Terminal-Bench 2.0。Claude 模型、harness 和任务集全部保持不变，唯一变化的是资源 floor 与 ceiling。资源最充足和最受限的两种配置，最终相差 6 个百分点 ($p < 0.01$) ([Anthropic - Quantifying Infrastructure Noise](#))。

这个差距比 leaderboard 顶部模型之间的典型分差还要大。含义很直接：领先 2 个百分点，可能代表真实的能力优势，也可能只是因为其中一次 eval 使用了性能更强的硬件。

11.2 两种资源区间

实验结果呈现出两种不同的资源区间：

- **从 1x 到 3x 单任务资源规格**，分数只在统计噪声范围内波动 ($p = 0.40$)，但基础设施错误率持续下降：从严格 enforcement 时的 5.8%，降至提供 3x headroom 时的 2.1% ($p < 0.001$)。新增资源为瞬时内存峰值留出了余量，避免容器因此发生 OOM。也就是说，在这个区间内，额外资源明显提高了 eval 的稳定性，却没有在可测量的程度上让任务变得更容易。
- **超过 3x 后**，分数上升的速度快于基础设施错误率下降的速度。从 3x 增加到 uncapped，infra errors 只下降了 1.6 个百分点，任务成功率却上升了将近 4 个百分点。此时，额外资源不再只是防止崩溃，还让 agent 能采用依赖宽裕资源的策略，例如安装大型依赖、运行高内存测试套件，或借助重量级工具暴力求解 (brute force)。

11.3 对测量意味着什么

严格的资源限制奖励高效策略，宽松的限制则奖励善于利用可用容量的 agent。这两种能力都值得测量。问题在于，如果不记录资源配置，就把不同配置下的结果合并成一个分数，这个分数将很难解释。

Anthropic 的 bn-fit-modify task 很好地说明了这一点。在资源宽松时，一些模型还没开始编写解法，就会先安装完整的 Python 数据科学栈，包括 pandas、networkx 和 scikit-learn。资源受限时，pod 会在安装过程中耗尽内存。其实还有一种更轻量的策略：只用标准库从头实现所需的数学计算，有些模型默认就会选择这种方法。因此，资源配置决定了哪一种默认策略能够成功 ([Anthropic - Quantifying Infrastructure Noise](#))。

同样的效应也出现在 Terminal-Bench 之外，只是幅度较小。在 Anthropic 的 SWE-bench 实验中，使用 5x RAM 的配置，在 227 个问题上的得分比 1x 配置高 1.54 个百分点。这个差距小于 Terminal-Bench，是因为 SWE-bench 任务对资源的需求较低；但增加资源依然不是一个中性变化 ([Anthropic - Quantifying Infrastructure Noise](#))。

11.4 建议

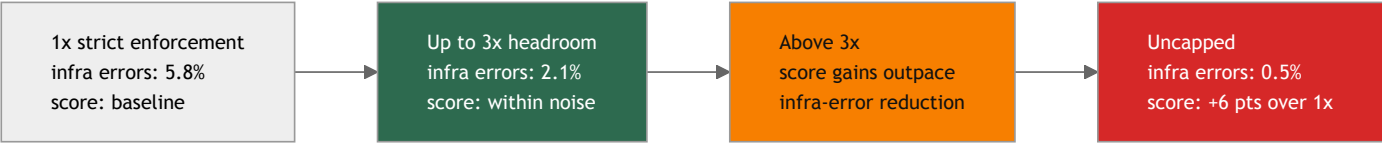
Eval 应分别指定保证分配量 (floor) 和硬上限 (ceiling)，而不是把资源固定在单一数值。对 Terminal-Bench 来说，把 ceiling 设为单任务资源规格的 3x 是一个合理的默认值：这样既能把 infra errors 减少三分之二，又能让分数增幅保持在统计噪声范围内 ([Anthropic - Quantifying Infrastructure Noise](#))。合适的倍数取决于具体 benchmark 和任务分布，因此每次报告结果时都应明确说明。

对 leaderboard 的读者来说，实践规则很简单：在资源配置得到完整记录并彼此匹配之前，应谨慎看待低于 3 个百分点的分差。领先几分可能意味着真实的能力差异，也可能仅仅意味着使用了更大的 VM。

图：资源配置与分数 (Terminal-Bench 2.0 汇总)

下表总结了 Anthropic 报告的两种资源区间。原文只明确给出了 1x、3x 和 uncapped 配置的错误率，因此中间一行采用定性描述，避免表现出来源并未提供的精度。

资源水平	Infra Error Rate	分数变化	解释
1x (严格)	5.8%	baseline	OOM kill 掩盖任务本身的失败
3x	2.1%	噪声内	实践平衡点: infra errors 减少 2/3
3x 以上	更低	分数开始比 infra errors 更快上升	Agent 开始利用额外 RAM
Uncapped	0.5%	比 1x +6 pts	资源密集型默认策略得以成功



超过 3x 后，分数增幅开始超过基础设施错误率的降幅：额外资源不仅提高稳定性，还会启用新的策略。

要点

- 仅硬件差异就能造成 6 个百分点的分差：Terminal-Bench 2.0 中资源最充足与最受限配置之间的差距，超过了 leaderboard 顶部模型之间常见的分差。
- 存在两种资源区间：从 1x 增加到 3x，主要作用是减少基础设施不稳定；超过 3x 后，则会启用资源密集型策略。
- **3x ceiling** 是实用的默认值：它能把 infra errors 减少三分之二，同时让分数增幅保持在统计噪声范围内。
- 应谨慎看待微小的 **leaderboard** 分差：在资源配置得到完整记录并彼此匹配之前，低于 3 个百分点的差异很难解释。
- 资源限制会塑造策略：严格限制奖励效率，宽松限制奖励利用可用容量的能力。二者都是有效的测量目标，但必须明确区分。

延伸阅读

- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>

第 12 章：基于 Trace 的迭代与 Model-Harness 共同演化

12.1 Traces 是反馈回路

今天的模型在很大程度上仍是黑箱，内部机制难以解释。不过，模型的文本输入和输出是可见的，仅凭这些信息就足以开展系统化改进。因此，LangChain 把 trace 视为调试 harness 的主要入口，记录 agent 的每一步动作，以及相应的延迟、token 数、成本和工具调用 ([LangChain - Improving Deep Agents](#))。

它的 *Trace Analyzer Skill* 将这套循环自动化：

1. 从 LangSmith 获取实验 trace。
2. 启动多个并行的错误分析 agent，由主 agent 汇总发现与建议。
3. 汇总反馈，并据此对 harness 做有针对性的修改。

这套方法在结构上类似经典机器学习中的 boosting：每轮迭代都聚焦于之前暴露的错误。第 3 步最好加入人工审查，但并非绝对必要。人工审查的主要作用，是防止修改只改善少数特定任务，却损害更广泛的表现。

12.2 运行时可观测性与过程可观测性

基于 trace 的迭代依赖两类可观测性。**运行时可观测性**记录系统实际做了什么，包括工具调用、命令输出、浏览器操作、延迟、token 使用量、错误、重试，以及最终的环境状态。LangChain 强调的正是这一层：记录 agent 的每一步动作，并从 trace 中归纳反复出现的失败模式。

过程可观测性记录成功的含义，以及为什么可以接受已经完成的工作，包括任务范围、sprint contract、rubric、验证证据、明确排除的工作和 handoff note。Anthropic 的 generator-evaluator harness 明确体现了这一点：实现开始前，双方先通过协商确定 sprint contract 和工作范围；如果 sprint 失败，evaluator 再依据 rubric 给出具体反馈 ([Anthropic - Harness Design for Long-Running Application Development](#))。

这两层可观测性相互补充。只有运行时信号而没有过程工件，只能知道发生了什么，却无法判断结果是否符合预定范围。只有过程工件而没有运行时信号，则可能为实际已经出错的行为包装出一套看似可信的文档。生产级 harness 应同时开放两类信息供人检查：任务的执行轨迹、验收标准，以及环境确实达到目标状态的证据。

OpenReview 综述还补充了一个重要的实现细节：agent trace 应组织成 span tree，而不是平铺的日志 ([OpenReview - Agent Harness Engineering: A Survey](#))。这些 span 至少应覆盖模型调用、工具调用、检索步骤、上下文组装操作、延迟、token 使用量、成本、重试、权限决策和最终结果状态。

这套结构如今正逐步走向标准化。OpenTelemetry 是分布式系统中广泛使用的厂商中立可观测性标准，目前正在定义 *GenAI semantic conventions*，即一套供 LLM 和 agent 遥测共同使用的 span 与属性命名规范 ([OpenTelemetry - Semantic conventions for generative AI spans](#))。这些约定为 harness 产生的 span 规定了名称，包括顶层的 `invoke_agent` span、每次模型调用对应的子 `chat` span，以及每次工具调用对应的 `execute_tool` span；同时还定义了操作时长和 token 使用量等标准指标。

采用这些约定有两项好处。第一，agent trace 可以与系统中的其他服务共用同一套可观测性技术栈。团队能够直接使用现有工具查询延迟、成本和错误，无须单独维护一套 agent 日志系统。第二，这项标准直接回应了隐私问题：对于 prompt 和 completion 内容，可以选择完全不记录、附加到 span，或存储在外部并在 span 中保留引用。标准还建议默认不要捕获敏感载荷，这与第 12.3 节和第 17 章要求的脱敏原则一致。在撰写本文时，GenAI 约定仍标记为 *Development*，具体名称可能继续变化；真正重要的是标准的发展方向，而非某个版本中的名称。

12.3 从生产 Trace 到 Regression Case

可观测性应直接服务于验证。能够揭示真实生产故障的 trace 十分宝贵，不应只作为调试工件留存。一套成熟的工作流程如下：

1. 捕获失败或出乎预期的生产执行轨迹。
2. 对敏感数据进行脱敏，并冻结相关环境或 fixture。
3. 提取用户意图、工具调用序列、中间状态和最终结果。
4. 为修正后的行为编写确定性断言或由模型评分的断言。
5. 将该用例加入 regression suite，并保留原始 trace 作为证据。

这套流程会把 trace 转化为 eval 任务的来源，也能避免团队只针对合成 benchmark 优化，却忽视真实用户遇到的故障。治理要求必须贯穿整条流程：trace-to-eval pipeline 要保留隐私、数据来源和权限元数据，否则生成的测试即使在技术上有用，也可能在实际运营中并不安全。

12.4 从实践者纠正到有界改进任务

OpenAI 的 tax-agent 案例又将这条反馈回路向前推进了一步。专家的纠正意见和生产 trace 不会直接改写已经部署的 agent，而是先转化为经过审查的发现、针对性的 eval，以及能够在发布前验证的有界 Codex 任务 (OpenAI - Building Self-Improving Tax Agents with Codex)。

一条安全的改进 pipeline 会明确规定每一步由谁接管、如何交接：

1. 记录实践者的纠正意见，以及促成这项纠正的 trace 和结果。
2. 对敏感数据进行脱敏，审查发现，并将重复问题归并为稳定的失败类别。
3. 以不可变的证据为依据，将该失败类别转化为可复现的 regression eval。
4. 向 coding agent 分配一个有明确边界的修改任务，而不是授权它随意修改自身。
5. 将目标 eval、更广泛的 regression suite、安全检查、成本和延迟共同设为 patch 的发布门槛。
6. 在生产环境中以 canary 方式发布新版本，监控最初的故障信号；如果情况恶化，则回滚。

这是受控自我改进 (**controlled self-improvement**)，而不是不受约束的自我修改。已经部署的 agent 不会直接改写自己的 prompt、工具、memory policy 或 evaluator。外部改进循环只负责提出一个版本化变更，能否进入下一阶段则由独立证据和发布控制决定。Evaluator integrity (第 10.4 节) 在两个环节都很重要：最初发现的问题必须真实，发布关卡也不能预设“这项修复应该获准发布”。

12.5 对关键组件进行压力测试

Anthropic 后续发布的 harness 设计文章补充了一项配套原则 (Anthropic - Harness Design for Long-Running Application Development)。Harness 中的每个组件，都隐含着关于“模型无法独立完成什么”的假设。随着模型能力提升，这些假设可能逐渐过时。检验方法很简单：每次移除一个组件，运行 eval，再观察结果。

Opus 4.6 推出后，长上下文检索能力和长周期编码表现都有所增强，Anthropic 因而得以在一个 harness 版本中移除 sprint 机制。即使不再把工作拆成多个 sprint，generator 仍能连贯运行两个多小时。Evaluator 在早期模型上承担了更多关键职责，但到了 4.6，是否需要它开始取决于具体任务：任务接近 generator 独立完成能力的边界时，它仍然有用；任务明显处于能力范围内时，它只会增加不必要的开销。团队将这项原则概括为：“是否使用 evaluator，并不是一个答案固定的是非题。只有当任务超出当前模型能够独立可靠完成的范围时，它的成本才值得付出。”

12.6 Model-Harness 共同演化

今天的前沿 coding model 通常会把 harness 纳入 post-training 流程 (LangChain - The Anatomy of an Agent Harness)。团队先发现有用的操作原语 (primitive)，将其加入 harness，再利用这些原语训练下一代模型；训练出的模型也会在这套 harness 中表现得更好。这形成了一条反馈回路，同时也使模型与 harness 相互耦合：即使某项 harness 修改在行为上看似中性，也可能导致模型表现下降。

Codex 的 `apply_patch` 工具就是典型例子。Codex 模型针对这种特定的 patch 格式接受过 post-training。OpenCode 是 Claude Code 的开源替代方案，为了模拟 Codex harness，它不得不专门为 GPT/Codex 模型加入 `apply_patch` 工具；Claude 和其他模型则继续使用常规的 `edit` 与 `write` 工具 (HumanLayer - Skill Issue)。

12.7 最佳 Harness 不一定是模型训练时的 Harness

这种耦合并不意味着模型训练时使用的 harness 对所有任务都是最优选择。Terminal-Bench 2.0 是实践讨论中经常引用的例子：HumanLayer 提到，Opus 4.6 在 Claude Code 中排名第 33，在另一套 harness 中却排名第 5，而 leaderboard 本身约有 4 个名次的噪声 (HumanLayer - Skill Issue; LangChain - The Anatomy of an Agent Harness)。这些具体名次只是某个时点的 leaderboard 快照，不能视为关于模型的永久结论。

LangChain 的案例研究也通过实验得出了相同结论。Claude Opus 4.6 在其早期 harness 版本中得分 59.6%，虽然具备竞争力，但仍低于调优后的 Codex 配置。上下文准备和验证等原则可以跨模型复用，不过要弥合剩余差距，仍需针对 Claude 再进行几轮 harness 迭代 (LangChain - Improving Deep Agents)。

实用规则很直接：更换模型时，要重新审视 harness。加强当前仍然关键的组件，移除不再发挥作用的组件。

12.8 实践要点

LangChain 将 harness 迭代经验归纳为五项原则 (LangChain - Improving Deep Agents)：

1. 为 agent 做好 **context engineering**：通过目录结构、可用工具、编码实践和问题解决策略，帮助模型了解所处环境。
2. 帮助 agent 验证自己的工作：模型容易停留在第一个看似合理的方案上，因此要明确要求它运行测试并检查结果。
3. 把 **trace** 作为反馈信号：工具和推理需要一起调试。模型走错方向，可能是因为缺少工具，也可能是因为不知道如何使用已有工具。

4. 在短期内遏制不良模式：随着模型进步，loop detection 等 guardrail 可能不再必要，但在当前阶段仍有价值。
5. 根据模型定制 harness：Claude 与 Codex 的 prompting guide 之所以不同，是因为通用原则可以迁移，具体实现往往不能照搬。

HumanLayer 也总结了一组相近的经验 ([HumanLayer - Skill Issue](#))：

有效的做法包括：从简单的 harness 起步，只在真实失败暴露后添加配置；快速迭代，并及时舍弃没有帮助的改动；通过仓库级文件在团队内分发经过验证的配置；优先提高迭代速度，而不是追求一次成功；在团队弄清实际需求后，删除多余能力。

效果不佳的做法包括：提前设计所谓的理想 harness；“以防万一”安装数十个 skill 和 MCP server；每个 session 结束时都运行完整测试套件；以及过度细调每个 sub-agent 能够访问哪些工具。

12.9 关于 AGENTS.md 的误导性数据

有一项结果尤其值得注意。ETH Zurich 的研究在多个仓库中测试了 138 个 agentfile——即 AGENTS.md、CLAUDE.md 一类指令文件的统称。研究发现，由 LLM 生成的 agentfile 不仅降低了性能，还使成本增加 20%；人工编写的文件则只带来约 4% 的性能提升。Agent 在处理这类上下文文件中的指令时，使用的 reasoning token 也增加了 14–22%。在该 benchmark 中，代码库概览和目录列表没有帮助，因为 agent 能够自行探索仓库结构 ([HumanLayer - Skill Issue](#) citing the ETH Zurich paper)。

HumanLayer 认为，这些结果印证了它对 AGENTS.md 的建议：文件应保持简洁，避免自动生成；采用渐进式披露，不要一开始塞入全部指令；根目录文件只保留普遍适用的规则，而不是各种带条件的指导。它自己的 CLAUDE.md 不到 60 行。

这并不意味着“不应编写仓库指令”，而是说根目录的指令文件应该充当路由器，而不是百科全书。OpenAI 的 Codex harness 指南也采用同样的思路：把关键上下文保留在仓库中，让 AGENTS.md 保持简洁，并在需要时将 agent 引导到更深入的文档；最重要的规则则尽可能通过自动化检查强制执行 ([OpenAI - Harness Engineering](#))。

更一般的原则是：配置并非越多越好。每条无关指令都会占用 agent 的注意力，却不能改善结果。Instruction budget 与 token budget 同样值得重视。

12.10 可复用 Harness 包与 Skills

一套 harness 模式经过实践验证后，不应继续作为只存在于某个仓库中的内部经验，而应被封装成可复用的形式。它可以是 skill、模板包、小型脚手架生成器，或一组仓库检查。无论采用哪种形式，都应同时提供指令和可以直接使用的工件：不只是提醒使用者“记得维护状态”，还要包含 progress log 模板、feature list schema、启动脚本和验证命令。

Learn Harness Engineering 课程用 harness-creator 展示了这种封装方式。这个 skill 用于创建、评估和改进五个 harness 子系统，涵盖指令、状态、验证、范围和会话生命周期 ([Learn Harness Engineering — Skills](#))。这形成了一条实用的工程边界：可复用 harness 包不应把某个所谓的理想 workflow 永久固化，而应让经过验证的默认做法易于安装和检查；当 trace 表明某个组件不再承担关键作用时，也应同样容易将其移除。

12.11 Meta-Harness：优化 Harness 本身

一旦具备 eval 和 trace，harness 本身也可以成为优化对象。OpenReview 综述提到的 meta-harness 研究，不再把 harness 视为固定不变的前提，而是探索不同的 harness 结构、prompting 策略、工具接口和控制循环设计 ([OpenReview - Agent Harness Engineering: A Survey](#))。

在实践中，这并不一定意味着完全自动化的架构搜索。团队可以先从有纪律的实验做起：

- 在同一套 task suite 上，对工具 schema 或 prompt 修改进行 A/B test。
- 移除一个组件进行消融实验，例如 planner、context reset、memory layer 或 evaluator。
- 调整成本控制、重试策略和工具响应截断方式，观察质量从何处开始下降。
- 衡量完整的闭环，而不只看模型输出：成功率、pass@k 可靠性、延迟、成本、转交人工处理的次数和安全误报。

这进一步扩展了“关键组件”原则：harness 中的每个部分都应持续证明自身收益足以抵偿成本。如果某个组件只对少数高价值任务提升可靠性，就只将这些任务选择性地路由给它；如果模型升级后它不再有帮助，就将其移除。

12.12 跨层耦合

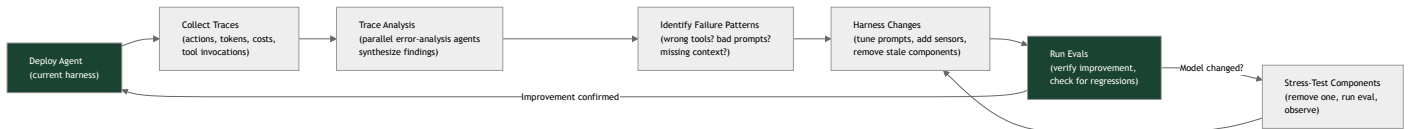
ETCLOVG 也可以作为调试地图。Trace 中看似糟糕的模型决策，根本原因可能位于 harness 的其他层 ([OpenReview - Agent Harness Engineering: A Survey](#))：

- 工具选择错误可能来自 Tool 层，因为动作空间过大，或 schema 没有把关键能力表达清楚。
- 过早宣布完成可能来自 Lifecycle 层，因为退出条件没有表示为持久状态机。

- 成本调优后的性能回退可能来自 **Observability** 与 **Execution** 层，因为资源限制改变了延迟、超时设置或 benchmark 的保真度。
- 安全故障可能来自 **Governance** 层，因为训练阶段的 alignment、部署配置和运行时 enforcement 在 policy、permission 和 audit hook 上并不一致。

在修改 prompt 之前，应先在 trace review 中标注可能负责的层。先问清楚：是哪一层让错误行为更容易发生、难以被发现，或几乎没有成本？然后再决定正确的修复方式究竟是修改 prompt、重新设计工具、调整沙箱、增加指标、使用更强的 grader，还是加入 governance hook。

图：基于 Trace 的迭代循环



要点

- **Trace** 是主要的调试入口：即使模型内部机制不可见，文本输入和输出仍然可见；系统化分析 trace 可以持续推动 harness 改进。
- 可观测性分为两层：运行时 trace 说明系统实际做了什么，过程工件说明为什么可以接受这项工作。
- **Trace span** 应采用结构化形式：模型调用、工具调用、检索、上下文组装、权限、成本和结果状态都需要机器可读的遥测数据。
- 遥测标准正在形成：OpenTelemetry 的 GenAI semantic conventions (`invoke_agent`、`chat`、`execute_tool` span) 让 agent trace 能够接入常规可观测性技术栈，并为保护隐私明确规定内容捕获方式。
- 生产 trace 应转化为 **regression case**：只要妥善保留隐私和数据来源，真实故障就是信号最强的 eval 任务。
- 自我改进必须通过外部、有边界的发布循环完成：先审查并归类纠正意见，将其转化为 eval 和范围明确的修改任务，再对版本化产物执行 gate、canary 和 rollback。
- 训练时使用的 **harness** 不一定最优：leaderboard 快照表明，同一模型采用不同 harness 时，表现可能大幅变化。
- 模型变化后要对组件进行压力测试：每个 harness 组件都隐含着可能随模型进步而过时的假设。
- **Model-harness** 共同演化确实存在：post-training 将 harness 纳入模型训练循环；任何一侧出现意外变化，都可能破坏原有耦合。
- 臃肿的 **AGENTS.md** 收益有限：内容应简洁并由人编写，再通过渐进式披露引导 agent 查阅仓库内的局部文档和自动化检查。
- 可复用 harness 包能够保存来之不易的经验：当 trace 证明某种稳定模式确实有效时，应将其沉淀为 skill、模板、脚手架和检查。
- **Meta-harness** 将 eval 变成设计搜索：prompt、工具、重试、上下文策略和 evaluator 都可以像其他系统组件一样接受消融实验和优化。
- 按层归因可以避免凡事只改 **prompt**：先用 ETCLOVG 判断是哪一层使故障成为可能，再决定是否修改指令。
- 迭代速度比前置设计更重要：从简单方案开始，只在真实故障出现后增加组件，并主动移除多余部分。

延伸阅读

- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Walking Labs, *Learn Harness Engineering - Skills*. <https://walkinglabs.github.io/learn-harness-engineering/zh/skills/>

- OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>

第 13 章：System Prompt 与指令架构

前言中的核心图将 *System Prompts* 列为 harness 的第一个组件，但此前各章大多把它视为既定前提。本章将直接讨论指令层：其中应该包含什么，agent 如何确定相互冲突的指令的优先级，harness 如何在运行时组装这些指令，以及为什么整个指令层都应当像系统的其他部分一样接受严谨的工程管理。

13.1 System Prompt 是 Harness 层，不是一句 Prompt

日常所说的“prompt”，通常是用户在聊天框中输入的一个问题。Agent 的 system prompt 则不同：它是随 harness 一起发布的持久程序，影响 agent loop 的每一个回合。HumanLayer 的十二要素宣言用 *own your prompts* 概括了这一区别。对于生产级 agent，prompt 是核心工程逻辑，不应成为隐藏在框架默认设置中的字符串 ([HumanLayer — 12-Factor Agents](#))。本章以下内容都建立在同一个前提上：把 system prompt 当作应用代码。

这个区别之所以重要，是因为 system prompt 承担着其他层无法替代的职责：定义 agent 的角色与目标，说明何时使用可用工具，写入 agent 必须遵守的 policy，并设定输出 contract。如果其中任何一项含糊不清，系统不会抛出语法错误；agent 反而可能逐渐偏离任务、行动过于积极、做出不必要的拒绝，或在错误的时机使用原本正确的工具。这些故障通常很难准确归因（第 10 章、第 12 章）。

13.2 指令层级 (Instruction Hierarchy)

Agent 会同时接收来自多个来源的文本：平台的 system prompt、开发者配置、用户请求，以及工具结果和检索文档中的内容；最后这一类尤其需要注意。这些文本并不具有同等权威。OpenAI 将这种差异形式化为 *instruction hierarchy*：通过训练，模型可以优先遵循 system 级指令，其次是 user 级指令，最后才是工具输出中的内容。其目标是防止低优先级文本覆盖高优先级指令 ([OpenAI — The Instruction Hierarchy](#))。

对于 harness engineer 来说，这正是第 5 章所述安全边界在指令层的体现。Prompt injection 就是这条边界失效的结果：网页或文件中的不可信文本试图把自己从数据提升为有权威的指令 (*lethal trifecta*，第 5 章)。Instruction hierarchy 提供了两道相互补充的防线：

- **利用模型训练出的优先级**：把真正的 policy 放在 system 位置，绝不放入用户可编辑或由工具提供的内容中。
- **从结构上加固边界**：将不可信片段明确标注为 data，因为训练形成的优先级只是一种倾向，而不是绝对保证。

实用规则是：指令的权威来自 harness 将它放置的位置，而不是措辞有多强硬。重要约束属于 system 层；同样的文字如果出现在检索文档中，就没有相应的权威，应被当作数据而非 policy。

13.3 什么该进 system prompt

并非所有相关事实都应写入 system prompt。塞得过满的 prompt 会在任务尚未开始时，就耗掉第 2 章所讨论的 attention 预算。可以按以下方式划分职责：

- **System prompt**：agent 的角色与目标、始终适用的高价值规则、单个工具描述无法表达的工具使用 policy（例如何时不应使用某个工具，以及工具之间如何配合），还有输出 contract。
- **工具描述**：每个具体工具的使用机制（第 4 章）。在 system prompt 中重复这些细节，会产生两份可能逐渐偏离的事实来源。
- **检索上下文**：变化频繁、内容庞大，或只在特定任务中需要的事实。它们应通过 just-in-time retrieval（第 2 章）按需提供，而不是写死在静态前缀中。

13.4 动态组装与稳定前缀约束

大多数生产级 agent 并不使用一个固定字符串。Harness 会为每次调用组装 system prompt，组成部分包括 base policy、当前工具集、项目特定指令，以及可能按需检索到的 skill（第 4 章）。组装方式必须考虑第 2 章所述的 KV-cache 经济学。缓存只能复用逐字节完全一致的前缀，因此会随调用变化的内容，都应放在稳定内容之后。

因此，指令架构也是一个布局问题。稳定且可复用的内容——例如 base policy 和常驻工具目录——应放在前面；易变内容——例如当前任务、刚刚检索到的事实和时间戳——应放在最后。如果 system prompt 在靠前位置插入当前时间或请求 ID，就会在每个回合悄然破坏前缀缓存，增加延迟与成本，却无法改善 agent 行为。

13.5 把 prompt 当版本化代码

System prompt 是关键组件，修改它就等于修改系统，也会带来与代码变更相同的回归风险。因此，第 10 章的工程纪律可以直接应用：在修改 prompt 前后分别运行 eval 套件，并比较通过率、失败类别、成本和延迟。第 12 章讨论的 model-harness 耦合进一步说明了这一点：为一个模型调优的 prompt 可能使另一个模型出现回退，因此 prompt 应与经过联合验证的模型一同进行版本管理。

具体来说，指令层需要版本控制、与 eval 结果关联的 changelog，以及明确的负责人。“Own your prompts”说的正是这件事：prompt 是有修改历史、需要持续维护的工程工件，而不是任何人都能在生产环境中临时改动的字符串

13.6 合适的“高度”

编写 system prompt 时，最难判断的是应当具体到什么程度。Anthropic 将其描述为寻找合适的 *altitude* (高度)：高度过低，prompt 会变成一组脆弱的硬编码 if-then 规则，一遇到未预料的情况就会失效，并且不断膨胀；高度过高，指导又会过于抽象，模型找不到可以据此行动的明确信号 ([Anthropic — Effective Context Engineering for AI Agents](#))。目标是让指导足够具体，能够稳定影响行为，同时又足够通用，可以迁移到 agent 实际会遇到的不同情况。

过度工程往往就隐藏在高度选择中。为了修复一次异常 trace 而加入的特例规则，可能变成长期负担：它会限制 agent 行为、占用上下文，还可能与以后新增的规则发生冲突。更好的修复方式常常不是继续向 system prompt 添加句子，而是修改工具、增加 sensor (第 5 章)，或添加一个明确约束预期行为的 eval case (第 10 章)。Trace 能够为这项选择提供依据 (第 12 章)。

13.7 跨三层 harness 的分层指令

指令层并非一个不可分割的整体。它映射了第 1 章所述的内外层 harness 结构。一个 coding agent 通常会组合以下内容：

- 实验室发布的 **builder harness** system prompt，
- 团队添加的 **user harness** 项目指令——AGENTS.md 文件、仓库约定和 review 规则 (第 12 章)——以及
- 针对特定任务按需加载的 **skill** (第 4 章)。

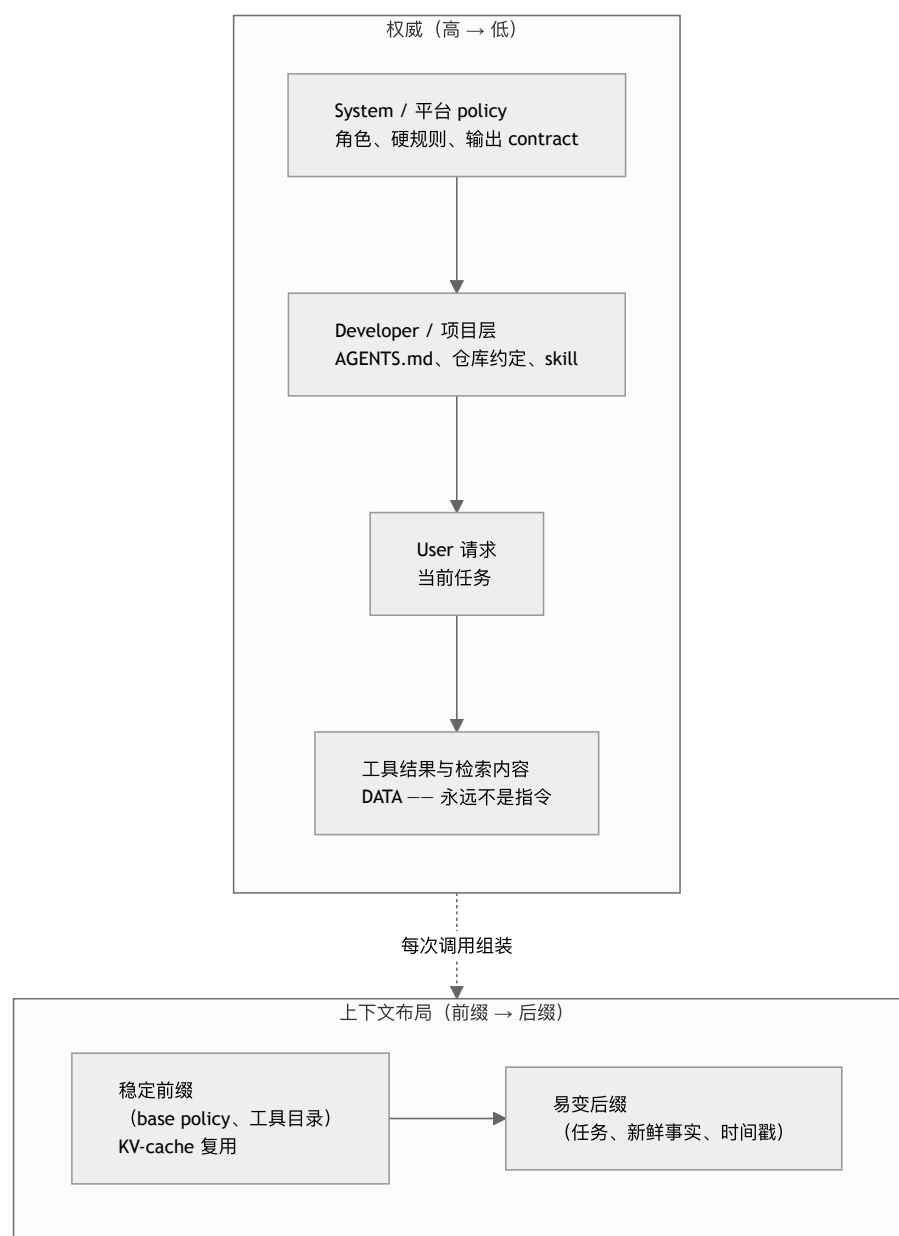
这些层在运行时共同组成模型实际看到的指令集。第 12 章的结论在这里同样适用：不要假设项目指令越多越好。关于大型 AGENTS.md 文件的研究结果并不一致，而且规定过细的项目层可能与其下方的 builder harness 发生冲突。指令架构需要衡量效果，而不是一味增加内容。

13.8 把 Scoped Review Rules 当作指令接口

仓库指令不仅可以指导代码生成，也可以指导代码 review。OpenAI 的 Codex 自定义 review 指南将简洁的 AGENTS.md 规则视为一种 **review interface**：每条规则都应指出一种具体缺陷、说明适用范围，并解释 reviewer 如何识别该问题；目录级文件则把规则限制在其所治理的代码附近 ([OpenAI - Custom Code Review Rules for Codex](#))。

好的 review rule 应简短、可测试且信号明确，例如“标记绕过 `authorize()` 的新 endpoint”，而不是“遵循安全最佳实践”。规则应指向相关代码或 policy，并尽可能关联确定性检查。它们也需要像其他指令一样进行版本管理和评测，因为噪声过多的规则会制造 false positive，久而久之还会让人忽略整个 review 界面。这是 progressive disclosure 在质量 policy 上的应用：把规则放在其所治理的代码附近，全局指令只保留普遍要求，并将稳定的 invariant 逐步转化为 linter 或 test。

图：指令栈



权威自上而下传递，由位置而非措辞决定；内容布局则从前缀延伸到后缀，受缓存经济学约束。这两个维度相互独立，都需要专门设计。

要点

- **System prompt 是应用代码：**它是持久存在、承担关键作用的 harness 层，不是随意编写的字符串；应指定负责人、进行版本管理，并用 eval 检验每次修改。
- **权威来自位置，而非强调语气：**instruction hierarchy 让 system 指令优先于用户输入和工具内容。Prompt injection 是层级边界失效，因此不可信片段必须标注为 data。
- **不同内容应各归其位：**角色与 policy 放入 system prompt，使用机制写进工具描述，变化的事实则通过 just-in-time retrieval 提供。
- **组装时要考虑缓存：**稳定指令放在前面，易变内容放在后面，否则前缀缓存会在不易察觉的情况下失效。
- **选择合适的高度：**指导应具体到足以影响行为，又通用到可以迁移；不要用一条新的硬编码规则修补每个异常 trace。
- **Review rule 是一种指令接口：**将简短、可测试的缺陷规则放在其治理的代码附近，持续衡量信号质量，并把稳定 invariant 转化为确定性检查。

延伸阅读

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
- OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>

第 14 章：模型选择、路由与推理模型

Agent = Model + Harness。前面各章主要讨论 harness，并一直把“模型”视为一个固定选项。但在实际系统中，模型选择本身也是一个重要的设计维度：每一步该调用哪个模型，较便宜的模型是否已经足够；而在推理模型出现之后，harness 还要决定在模型行动前投入多少推理预算。本章关注这些运行层面的决策。模型内部的工作原理见姊妹卷 *LLM Foundations*。

14.1 模型选择是 harness 决策

第 6 章的第一条原则——采用能够奏效的最简单方案——不仅适用于控制流，也适用于模型选择。最大、能力最强的模型很少适合作为 agent loop 每一步的默认选项。大多数 loop 只包含少量困难的规划或综合任务，同时包含大量较简单的任务，例如输入分类、结果格式化和字段提取。困难步骤可能确实需要前沿模型；简单步骤通常交给更小、更快、更便宜的模型也能取得同样好的结果 (*Anthropic — Building Effective Agents*)。

如果把模型当作单一的全局设置，就无法利用这种差异。harness 可以为每个步骤分别选择模型；选择的单位应当是一次调用，而不是整个应用。这是对第 3 章中模型路由思路的推广：不能改善结果的额外模型能力就是浪费。在生产环境中，这种浪费会直接体现为每个请求增加的延迟和成本。

14.2 路由：让模型强度匹配任务难度

在工作流中，*routing*（路由，第 6 章）是把输入分派到专门的处理分支。*Model routing*（模型路由）则把同样的思路用于模型选择：估计请求难度，把简单请求交给更小、更便宜的模型，把更强、更昂贵的模型留给困难请求。真正的难点在于分类器。只有当路由判断足够可靠，而且判断本身的成本远低于节省的模型开销时，路由才有价值。

RouteLLM 表明，这种判断可以通过学习获得：router 在偏好数据上训练后，会把每个 query 分派给强模型或弱模型；在标准 benchmark 上，整个系统只花强模型的一小部分成本，就能保留后者的大部分质量 (*RouteLLM*)。对 harness 而言，重点不在某个具体 router，而在这个通用杠杆：在昂贵调用之前增加一个小而快的判断，可以改善成本与质量之间的权衡，但前提是该判断本身既便宜又可靠。困难任务如果被错送到能力不足的模型，可能会静默失败；由此造成的损失，可能远高于省下的模型调用成本。

14.3 级联与回退

路由会在执行任务之前选择模型。**级联 (cascade)** 则把决定推迟到第一次尝试之后：先调用较便宜的模型；如果答案通过 verifier，就直接接受；只有未通过时，才升级到更强的模型。FrugalGPT 展示了这类级联：大多数 query 由便宜模型处理，只有剩余的困难请求需要升级，因此能够在显著降低成本的同时，达到与前沿模型相当的准确率 (*FrugalGPT*)。

级联的质量取决于升级信号，也就是判断第一个答案是否合格的 verifier。这里可以沿用本书反复讨论的验证机制：基于代码的检查、测试、schema 校验，或基于模型的 grader（第 5 章、第 10 章）。没有可信的信号，级联只会增加延迟，最后仍可能返回同一个错误答案。

与此相关的另一个生产模式是 *fallback*（回退）。当主模型报错、超时或受到限时，harness 会切换到备用模型，使 agent 能够降级运行，而不是停滞不前。Fallback 解决的是可靠性问题，而不是质量判断问题，与第 7 章的 checkpoint/resume 同属可靠性机制。

14.4 推理模型与 test-time compute

推理模型通常会经过 post-training（后训练），例如在答案可检查的问题上采用 reinforcement learning，使模型在给出最终答案之前进行更充分的内部推理。它通过消耗额外的 inference token，提升解决困难问题的表现。DeepSeek-R1 表明，使用可验证奖励的 reinforcement learning 可以激发出较强的推理能力 (*DeepSeek-R1*)。其他研究还发现，对某些问题而言，在固定预算下增加 *test-time compute*——也就是让模型在 inference 阶段投入更多计算——可能比选择更大的模型更有效 (*Snell et al. — Scaling LLM Test-Time Compute*)。具体机制见姊妹卷 (*LLM Foundations*，第 7–8 章)。对 harness 的直接影响是：“让模型推理多久”已经成为独立于“选择哪个模型”的另一条可控轴。

这与第 11 章讨论的扩展方式不同。那里，增加硬件是为了支持更多 agent 动作；这里，增加 inference 预算是在单次调用中、尚未使用任何工具之前，换取更多内部推理。两者都会消耗资源，但不能相互替代。

14.5 驾驭推理模型：不要和训练对着干

推理模型改变了若干 harness 假设。最常见的错误，是用提示和配置去干扰模型已经通过训练获得的行为：

- **不要手动要求模型执行它已经具备的推理过程。** 对经过推理训练的模型额外加入“think step by step”之类的指令，可能浪费 token，也可能与训练形成的行为冲突。应当遵循供应商针对该类模型给出的指引。
- **为不可见的 token 预留预算。** 推理模型可能在给出第一个可见输出之前，先生成数千个隐藏的推理 token。这些 token 同样会增加延迟和成本。如果模型提供 reasoning-effort 控制，应将其视为一等的 harness 参数，并在适当情况下开放给用户。

- **不要把推理 trace 当作可信解释。** 推理文本可能被隐藏、压缩为摘要，也可能无法忠实反映模型的实际计算过程。任何可见的推理都只能作为 debug 辅助，不能替代验证——这与第 10 章对模型自我报告的谨慎态度一致。
- **推理不等于 grounding。** 增加内部推理并不会凭空产生证据。模型可以更仔细地分析 harness 提供的材料，却无法创造从未获得的事实。因此，工具、检索和验证仍然不可缺少（第 4 章、第 10 章）。

14.6 路由到推理：额外 token 何时划算

推理模型最适合数学、代码、规划和多步分析等任务，因为这类任务通常包含困难但可检查的子问题。对于简单的抽取、格式化和分类，额外推理往往只会增加延迟和成本，并不能提升质量。因此，推理能力也应当像其他能力一样按需路由：把困难且可验证的步骤交给推理模型，把前后的简单步骤留给快速的非推理模型 (§14.2)。

这与第 6 章的 micro-agent 模式可以自然组合。确定性 DAG 能够把推理模型调用准确放在需要深入分析的节点，并在其前后安排成本较低的确定性处理。这个“reasoning sandwich”可以避免在开放式 loop 的每个回合都为长时间推理付费。

14.7 模型升级是 harness 事件

前沿模型越来越多地在包含 harness 的环境中接受 post-training，因此模型行为与 harness 会相互耦合（第 12 章）。所以，更换模型——无论是升级版本、改用更便宜的供应商、采用量化变体，还是切换到不同的推理类型——都是一次系统变更，而不是即插即用的替换。一个在公开 benchmark 上表现更好的模型，仍可能在具体 workflow 中退步：它可能以不同方式理解工具描述，输出详略程度不同，或在本应快速回答时投入过多推理。

这里应遵循第 10 章的纪律：把每次模型变更都视为一次回归事件。变更前后都要运行 eval 套件；除了 pass rate，还要比较失败类别和成本。Prompt 与工具应当和它们所验证的模型版本一起管理。Readiness 不是模型自身的属性，而是整个 model-plus-harness 配置的属性。

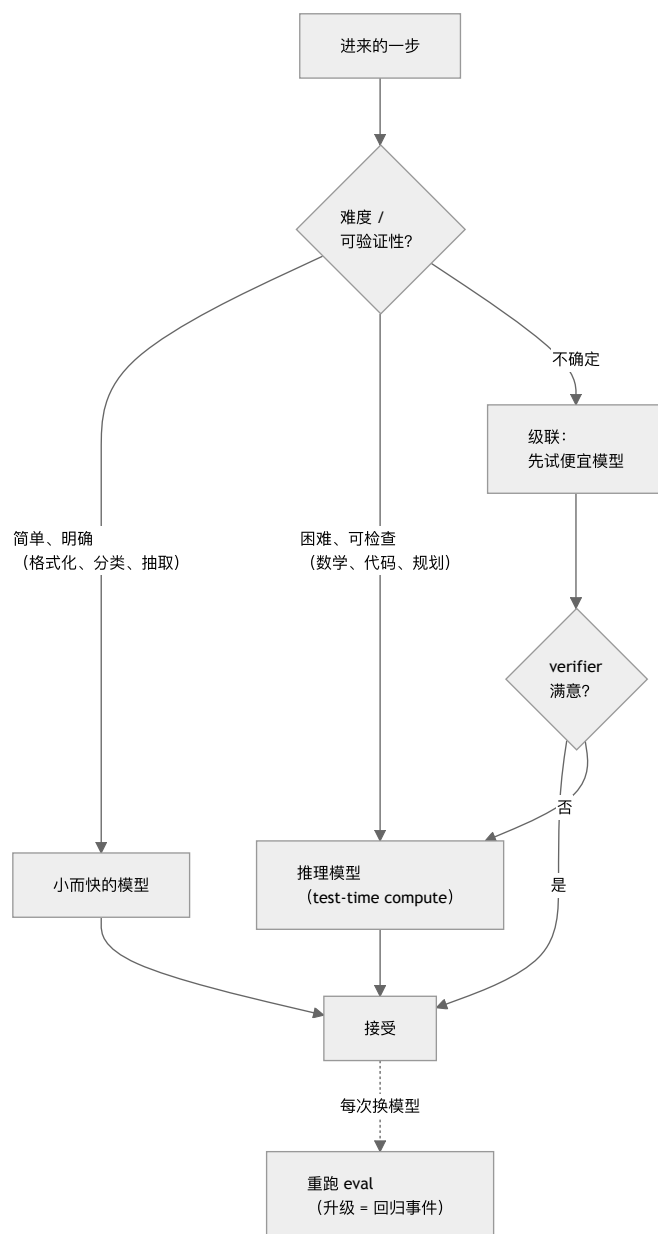
14.8 AI 网关：把路由做成基础设施

第 14.2–14.3 节把路由、cascade 和 fallback 视为应用逻辑。在生产环境中，其中许多逻辑可以放到专门的基础设施组件中：AI 网关 (AI gateway，也称 LLM gateway 或 LLM proxy)。网关位于 harness 与模型供应商之间，在多个供应商之上提供一个统一端点，通常兼容 OpenAI 接口。它还集中处理模型路由与 fallback、负载均衡、重试、按 key 设置的预算和限流、缓存，以及请求/响应日志等共性问题。开源方案包括 LiteLLM，它通过单一接口接入 100 多家供应商，并提供成本追踪和负载均衡 (LiteLLM)；Portkey 则进一步把 guardrails 和 PII 脱敏纳入网关层 (Portkey - AI Gateway)。

AI 网关之所以属于 harness 的讨论范围，是因为它把原本需要各 agent 分别实现的多项职责集中起来。第 14.3 节的 fallback 可以改为网关配置，不必重复写进每个 agent。第 17 章讨论的成本预算和 span 级归因也适合放在网关，因为它能观察到每一次模型调用。稳定的网关接口还便于执行第 14.7 节的模型替换纪律：无需修改 agent 代码，就能只向一小部分流量灰度新模型。

这种集中化也有代价。网关成为热路径上的依赖，因此自身需要采用第 5.11 节介绍的熔断器和超时机制；一旦网关故障，接入它的所有 agent 都会受到影响。集中化也不会消除对可靠路由决策的需要（第 14.2 节），只是为这些决策提供一个统一的执行位置。与第 4.3 节讨论的 MCP 类似，网关只是管道：它能降低可靠路由策略的运维难度，却不能把糟糕的策略变好。

图：模型选择决策



把简单步骤交给较小的模型，把困难且可检查的步骤交给推理模型。无法预先确定时，可以使用级联，由 verifier 决定是否升级。任何模型变更都是对整个配置的一次回归事件。

要点

- 按步骤选择模型，而不是为整个应用固定一个模型：只在确实能改善结果的调用上使用前沿能力，把简单工作路由给更小、更快的模型。
- 路由在执行前决定，级联在首次尝试后决定：通过学习得到的 router 可以改善成本与质量的权衡（RouteLLM）；由 verifier 把关的级联则能以更低成本达到强模型的准确率（FrugalGPT）。两者都依赖便宜而可靠的决策信号。
- 推理模型增加了一条独立的设计轴：test-time compute 能在困难、可检查的任务上用 inference token 换取更高质量。这不同于选择更大的模型或增加硬件。
- 顺应模型的训练方式：不要手动要求模型执行它原本就会的推理；为隐藏 token 预留预算；不要把 reasoning trace 当成验证；还要记住，推理本身不能提供 grounding。
- 把模型升级视为 harness 事件：每次更换模型都要重新运行 eval。Readiness 属于整个 model-plus-harness 配置，而不是模型本身。
- 用 AI 网关集中承载路由基础设施：LiteLLM、Portkey 等 proxy 可以在统一接口后处理 fallback、预算、缓存和日志，但仍然需要可靠的路由决策以及网关自身的熔断器。

延伸阅读

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
- DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *LLM Foundations for Harness Engineering* (姊妹卷), 第 5–8 章。../llm-foundations-zh/
- LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
- Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>

第 15 章：人-Agent 交互

本书大部分内容都在讨论模型与外部世界之间的机制：context、工具、sandbox、状态和评估。然而，只要 agent 执行的是可能产生重要后果的工作，人也会成为 loop 的一部分。人可能需要批准动作、在执行过程中调整方向、review 结果，并判断应该在多大程度上信任输出。支持这些判断的接口，本身就是一个独立的 harness 层，正如第 4 章讨论的 agent-computer interface 一样，需要经过有意识的工程设计。第 5 章已经介绍了其中一个问题——permission fatigue；本章将从整体上讨论人与 agent 之间的接口。

15.1 人在 loop 之内

讨论 agent-computer interface 时，我们强调：agent 如何使用工具，应当获得与传统用户界面同等的工程重视。人-agent 接口正好是它的镜像：人如何监督 agent，和 agent 如何行动同样值得重视。Anthropic 的实现原则也指向这一点：保持简单，优先保证透明度并展示 agent 的规划步骤，同时仔细设计接口 (Anthropic — Building Effective Agents)。透明度不只是改善 UX；缺少透明度，就不可能进行有意义的监督。

这个问题早在现代 agent 出现之前就已经存在。Horvitz 在 1999 年提出的 *mixed-initiative* (混合主动) 接口原则，已经明确了几个核心问题：代用户行事的系统何时应该自主执行，何时应该让用户决定？它应该如何衡量打断用户的成本？又该如何保留并利用交互 context (Horvitz — Principles of Mixed-Initiative User Interfaces)？当系统能够运行 shell 或执行其他可能产生重要后果的动作时，这些问题会变得更加迫切。

15.2 两个失败模式：permission fatigue 与盲目信任

人-agent 接口通常会以两种相反的方式失效，合理的设计必须同时避免它们：

- **Permission fatigue (许可疲劳)** 源于询问过于频繁。如果 agent 每执行一个动作都要请求批准，人会逐渐习惯这些提示，最后不再阅读就直接点击“allow”。这甚至可能比完全没有提示更糟，因为它只有监督的表象，却没有监督的实质 (Anthropic — Beyond Permission Prompts, 第 5 章)。Sandbox 的作用之一，正是让边界内的低风险动作无需反复询问。
- **盲目信任** 源于询问过少。流畅、自信的输出很容易让人未经检查就直接批准。当 agent 给出一个貌似合理的错误——例如虚构引用，或进行了一次存在细微缺陷的重构——如果 review 界面没有展示支持结论的证据，人就更难发现问题。

这两种失败位于同一条调节轴的两端：提示太少容易造成盲目信任，提示太多又会引发许可疲劳。解决办法不是选择一个适用于所有动作的全局设置，而是采用与风险相称的交互方式：动作需要占用多少人工注意力，应当由它的可逆性和影响半径决定 (§15.3)。

15.3 混合主动：何时问、何时做

这个核心问题需要针对每个动作分别回答：agent 应该直接执行，还是先征求批准？一个实用的默认原则，是根据动作可能造成的后果来决定人的参与程度：

- **静默执行**：对于读取文件、运行测试、修改 scratch 工作区等可逆且低风险的动作，可以在 sandbox 内直接执行 (第 5 章)。如果连这些动作都要求批准，只会让人逐渐忽略提示。
- **执行并报告**：如果动作会产生一定后果，但结果可观察、动作也可逆，就不必阻塞等待批准；执行后留下清晰记录，供人事后 review 即可。
- **先问再执行**：对于删除数据、发送外部消息、花钱或修改生产环境等不可逆或高影响半径的动作，必须先获得批准。这也对应第 5 章的 *lethal trifecta* 边界：当 agent 即将把私有数据、不可信输入和外部触达能力结合起来时，人就应当参与决策。

这正是第 18 章再次讨论的 capability-control 权衡：更高的自治程度带来更强的能力，也带来更重的控制负担。接口把这种权衡落实到每一个具体动作上。

15.4 把批准建模为一次工具调用

HumanLayer 的十二要素宣言提出了一种清晰的结构：通过工具调用联系人类。不要把人的批准视为控制流中的特殊例外，而应把它建模为 agent 可以调用的普通工具——`request_human_approval(action, context)`。人的回应随后像其他 observation 一样返回 loop (HumanLayer — 12-Factor Agents)。

这样一来，人与 agent 的交互就能纳入系统的整体架构。批准请求会成为 event log (第 9 章) 中的一个事件，因此可以持久保存、重放和审计。它也能与第 7 章的长运行模式配合：agent 发出批准请求后可以挂起，数小时后收到回应再继续执行。因为状态保存在 log 中，系统不必一直维持一个打开的连接。人也因此成为 trace (第 12 章) 中的一等参与者，而不再是一次带外打断。

15.5 设计 review 界面

人的 review 质量取决于界面提供了哪些信息。只展示 agent 结论的界面容易诱发盲目信任；只有同时展示结论背后的证据，人才能作出有依据的判断。对于 coding agent，证据包括 diff、实际运行过的测试和执行过的命令，而不只是一句“done”。对于 research agent，证据包括摘要所依据的来源。

这些 review 证据可以来自调试所使用的同一套 span telemetry（第 12 章）：有用的 trace 同时也是有用的 review 产物。人-AI 交互研究的原则在这里可以直接应用：界面应当说明系统能够做什么，展示这次任务完成得如何，并让人能够高效纠正或否决错误输出 (Amershi et al. — Guidelines for Human-AI Interaction)。只有当 agent 的工作易于验证，也能在必要时方便地否决，人类监督才具有可操作性。

15.6 引导与打断

监督不仅发生在执行前后，也发生在执行过程中。如果人发现 agent 正在沿错误方向推进，就需要在不终止 session、也不丢失状态的情况下重新引导它。因此，harness 必须提供 steering（引导）能力：把新指令注入正在运行的 agent，并让下一回合纳入这条指令。

实现方式与前面各章的机制直接相连。Agent loop 会在每一回合重新组装 context（第 1 章），所以 harness 只需把 steering 消息加入下一次迭代。按照第 3 章的 recitation 逻辑，把纠正指令放在 context 末尾附近，可以使其处于模型最可靠的 attention 范围内。Steering 还依赖完善的 checkpoint（第 7 章）：能够暂停和恢复的 agent 才能接受中途纠正；如果全部状态都封闭在一次不可中断的调用中，就无法进行引导。

15.7 校准信任：透明与不确定性

人-agent 接口的目标是建立经过校准的信任：针对当前任务和现有证据，人对 agent 的信任程度应当恰如其分。信任过度会让错误未经 review 就被接受；信任不足则会造成疲劳，浪费人的注意力。harness 可以通过两种机制来帮助校准信任：

- **透明度**——展示规划步骤、工具调用和证据——让人根据 agent 的实际工作过程建立信任，而不是被流畅的措辞说服 (Anthropic — Building Effective Agents)。
- **不确定性信号**——明确指出结果中仍有疑问的部分——把人的注意力引向真正需要检查的位置。但这种机制只有在信号可信时才有价值。姊妹卷指出，模型信心本身往往没有得到良好校准 (LLM Foundations, 第 10 章)。因此，harness 应优先用外部验证来说明不确定性，例如测试是否通过、引用来源是否存在，而不能只依赖模型自己含糊或保留的语气。

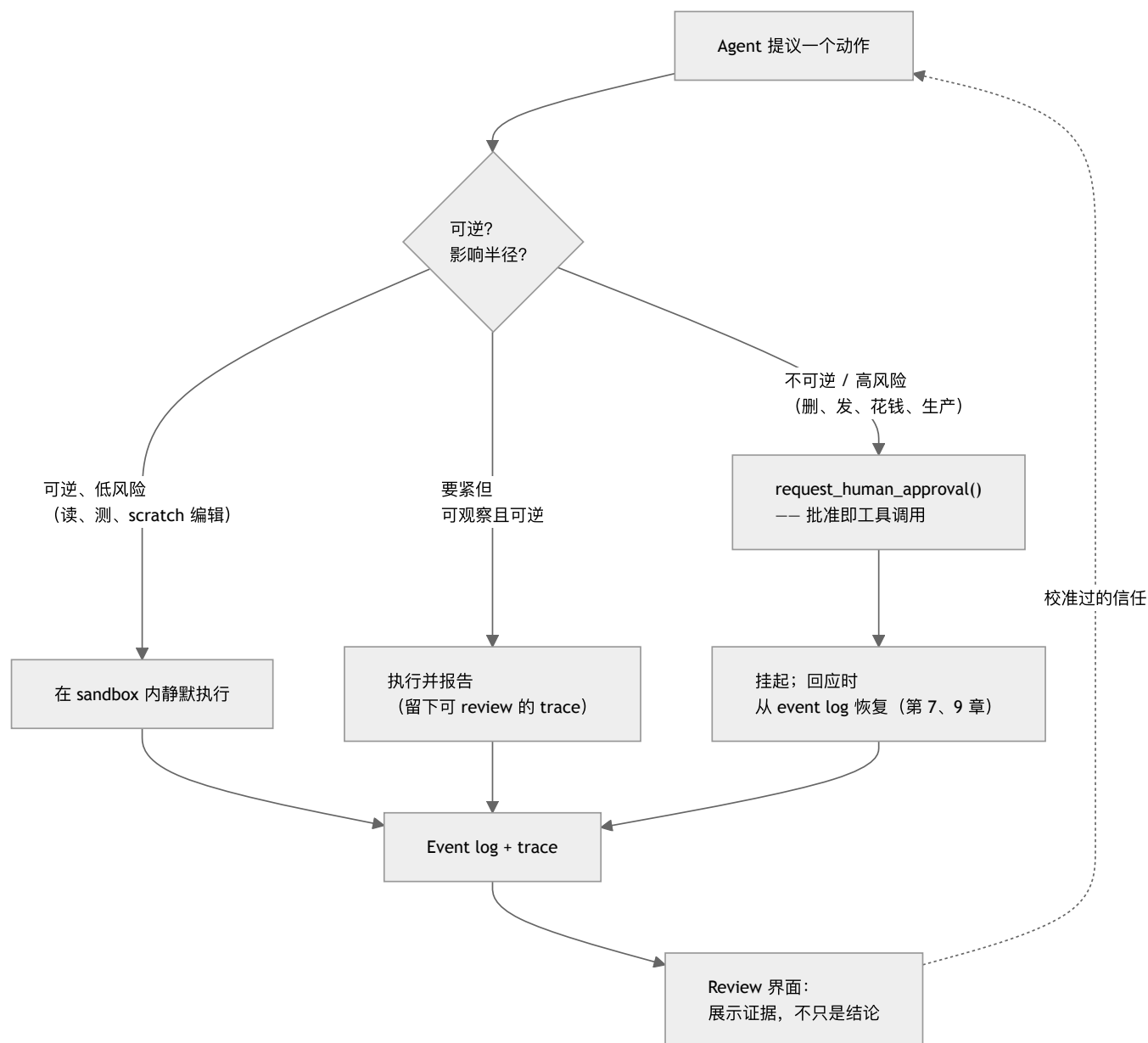
15.8 监督许多 agent

随着 agent 越来越普遍，人的角色会从亲自完成工作，转向监督完成工作的 agent，最终还要同时监督许多 agent。这种转变会改变接口需求。一个人监督十个 agent 时，不可能逐条阅读所有 trace；界面必须主动标出需要关注的情况，例如哪些 agent 正在等待批准，哪些结果仍有不确定性，以及哪些输出没有通过验证。

当 agent 生成变更的速度超过人的检查速度时，code review 就会成为瓶颈。因此，review 队列应按**风险与证据**排序，而不是按到达时间或 trace 长度排序。确定性检查失败、高 blast radius 动作、安全敏感路径、新的工具用法、policy rule 命中、薄弱或相互冲突的 grader 证据，以及规模很大却缺少解释的 diff，都应当优先处理。对于验证充分的低风险变更，可以只展示摘要或进行抽样；可能产生重要后果的变更，则保留完整 artifact。第 13.8 节介绍的 scoped review rule 可以进一步提高这种路由的精度。

到了这个规模，监督就成为第 18 章所讨论的 fleet 级控制平面问题。核心原则是让监督既高效又有实质意义：足够高效，一个人才不会在监督多个 agent 时被信息淹没；具有实质意义，监督才不会沦为走过场的批准。本章介绍的各项技术——与风险相称的提示、把批准表示为工具调用、提供充分证据的 review 界面、steering，以及经过校准的透明度——都服务于这一目标。

图：与风险成比例的交互



交互方式可以从静默执行延伸到强制批准。每个动作应根据可逆性和影响半径放在这个范围中的合适位置。把批准视为工具调用，并在 review 时展示证据，才能同时避免 permission fatigue 和盲目信任。

要点

- 把人与 agent 的接口视为 **harness** 层：人如何监督 agent，和 agent 如何行动一样值得投入工程精力。
- 同时避免两种失败模式：询问太多会造成 permission fatigue，让人习惯性批准；询问太少则会造成盲目信任，使貌似合理的错误未经检查就被接受。
- 让交互方式与风险相称：可逆的低风险动作可以静默执行；可观察、可逆的动作可以执行并报告；不可逆或高影响半径的动作必须先问，尤其是在 lethal-trifecta 边界上。
- 把批准表示为工具调用：这样，批准过程就能够持久保存、重放和审计，并能与长运行任务的挂起和恢复机制配合。
- 展示证据，而不只展示结论：无论一个人监督一个 agent 还是整个 fleet，要真正实现监督，都必须让 agent 的工作易于验证，也易于否决。
- 按风险排列 fleet review 的优先级：失败的检查、敏感路径、policy hit、首次出现的动作和薄弱证据，应当优先于验证充分的低风险工作。

延伸阅读

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>

- Dex Horthy, *12-Factor Agents* (要素 7: 用工具调用联系人类), HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
- OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>

第 16 章：Computer-Use 与多模态 Agent

第 4 章通过定义清晰的工具构建了 agent-computer interface：带 schema 的函数、MCP server 和 code API。如今，越来越多 agent 采用另一种工作方式。它们不调用接口明确的 API，而是像人一样操作软件：观察屏幕、移动光标、在字段中输入文字并点击按钮。这类 *computer-use agent* 依赖多模态感知，也给 harness 带来了 API 型工具不会遇到的问题。当下许多目标宏大的通用 agent 也正采用这种工作方式。

16.1 越过 API：操作软件的 agent

核心动机是扩大 agent 能操作的软件范围。大多数软件没有对 agent 友好的 API，只有为人设计的图形界面。只要能看懂屏幕并在其中行动，agent 就能使用原本难以接入的软件，包括旧式桌面应用、没有 API 的网站，以及不太可能被封装成 MCP 的内部工具。Anthropic 的 computer-use 模型和 OpenAI 的 Computer-Using Agent 都采用这种方式：模型接收截图，判断画面内容，再发出“在 (x, y) 处点击”或“输入这个字符串”等底层动作 ([Anthropic — Computer use](#)；[OpenAI — Computer-Using Agent](#))。

这种 loop 与第 1 章介绍的 agent loop 有显著差别：observation 是图像而不是工具结果，action 是 UI 操作而不是函数调用，环境则是整个操作系统或浏览器，而不是一套精心挑选的工具。本书关于工具、context 和安全的原则仍然适用，只是它们现在要通过视觉界面作用于更底层的操作。

16.2 环境就是工具面

在 ACI 那一章，harness engineer 可以选择工具，只提供少量用途清晰、易于正确调用的动作（第 4 章）。computer-use agent 的动作面却由环境决定：屏幕上的每个按钮、菜单和字段都可能成为操作目标。第 4 章所说的精心筛选工具，在这里无法照搬，因为应用在界面中展示的所有元素都可能成为“工具”。

这改变了一个关键的设计杠杆。Harness 无法再靠减少工具数量来缩小动作空间，而必须帮助 agent 准确识别/可用动作，并限制它可以在哪里行动。设计重点因此从工具选择转向感知与范围控制，这也是接下来几节的主题。

16.3 屏幕的几种编码

Harness 可以用多种形式把屏幕呈现给模型，而选择哪种形式会直接影响效果。正如姊妹卷所解释的，图像最终也会转化为 token (*LLM Foundations*，第 2 章)。常见的屏幕编码包括：

- **截图像素**：保留视觉布局和样式，但会消耗大量 token，也可能让小号文字难以辨认。模型必须依靠视觉来定位元素。
- **DOM 或 HTML**（网页）：保留精确文本和文档结构，却无法可靠反映用户实际看到的内容。隐藏、位于屏幕外或相互遮挡的元素可能难以区分。
- **Accessibility tree**（可达性树）：提供为辅助技术设计的结构化语义视图，包含 role、label 和 state，通常比原始 DOM 紧凑得多。
- **OCR**：从像素中恢复文字，但会丢失大量结构和空间关系。

没有哪一种编码能提供完整信息。截图呈现了人所看到的画面，却不包含底层结构；DOM dump 展示了结构，却无法体现视觉上的显著程度。因此，生产系统常常组合多种编码，例如用截图理解布局，再用 accessibility tree 或 DOM 确定准确目标。这样更稳健，但也会占用更多 context。这与检索中的精度和预算权衡（第 2 章）本质相同，只是发生在视觉领域。

16.4 Grounding：从“看见”到“点击”

computer-use agent 特有的最大难题是 *visual grounding*（视觉定位）：把“点击 Submit 按钮”这样的意图，转换为屏幕上某个具体坐标处的点击。模型可能判断对了该按哪个按钮，却点在错误的位置。API 型工具没有直接对应的失败模式，因为选择一个具名函数是精确操作。

一种常见的 harness 技术是 *Set-of-Mark prompting*。Harness 先在截图中的候选交互元素上叠加编号，让模型选择“点击元素 7”这样的离散标签，而不是直接生成坐标 ([Yang et al. — Set-of-Mark Prompting](#))。这样可以把容易出错的连续坐标问题转化为更可靠的离散选择，但 harness 必须先检测并标记候选元素。SeeAct 相关研究也表明，即使是能力很强的视觉模型，也需要这类 grounding 脚手架，才能在真实网页上可靠行动。感知和动作是两种可以分开的能力，而 harness 的作用正是弥合两者之间的差距 ([Zheng et al. — GPT-4V is a Generalist Web Agent](#))。

16.5 动作空间及其失败模式

与函数调用相比，computer-use 动作更底层，也更依赖当前状态。一次点击能否成功，取决于屏幕是否仍处于模型预期的状态。页面加载缓慢、意外弹出的 modal 或位置发生变化的布局，都可能让模型已经选定的动作失效。因此，loop 必须在每次动作后重新观察屏幕，并把新画面视为 ground truth；同时还要考虑 observation 与 action 失去同步的情况。

这使第 7 章强调的 self-verification 更加重要。每次行动后，agent 都应先确认屏幕是否按预期变化，再继续下一步。恢复也变得更困难：撤销 GUI 操作通常不像回滚 API 调用那样干净。因此，第 15 章提出的、与风险相称的人机交互在这里尤其重要。computer-use agent 即将确认不可逆对话框时，正应该设置人类 checkpoint。

16.6 延迟、成本与像素的 token 重量

computer-use loop 的每个回合都会向 context 发送一张图，而图像会消耗大量 token。单张高分辨率截图的成本可能相当于一页文字 (*LLM Foundations*, 第 2 章)。一个需要点击三十次的任务，可能让三十张截图先后进入 context window。这会迅速消耗第 2 章所说的有限上下文预算，也会加剧 agent 负载以 prefill 为主的特征。

熟悉的 harness 杠杆在这里仍然有效，但需要更谨慎地调节。应在任务允许的范围内尽量降低截图分辨率，却不能让 agent 需要读取的文字或控件变得无法辨认。提取出有用信息后，应从 context 中移除旧截图，只保留一条简洁的屏幕内容说明。第 3 章“把不需要的内容挡在外面”的原则对图像尤其重要。如果 accessibility tree 等结构化表示已经足够，就应优先使用它们，只在布局确实重要时才保留成本较高的截图。

16.7 安全：全书最宽的攻击面

computer-use agent 集中了 *lethal trifecta* (第 5 章) 的三个条件：它会读取屏幕上网页或文档中的不可信内容，可能通过浏览器或文件系统接触私有数据，还能通过页面跳转、提交表单或发送消息与外部通信。这里的 prompt injection 不只是藏在工具结果中的恶意文本，也可能是直接显示在网页上的指令，而模型可能误把它当成真正的命令。

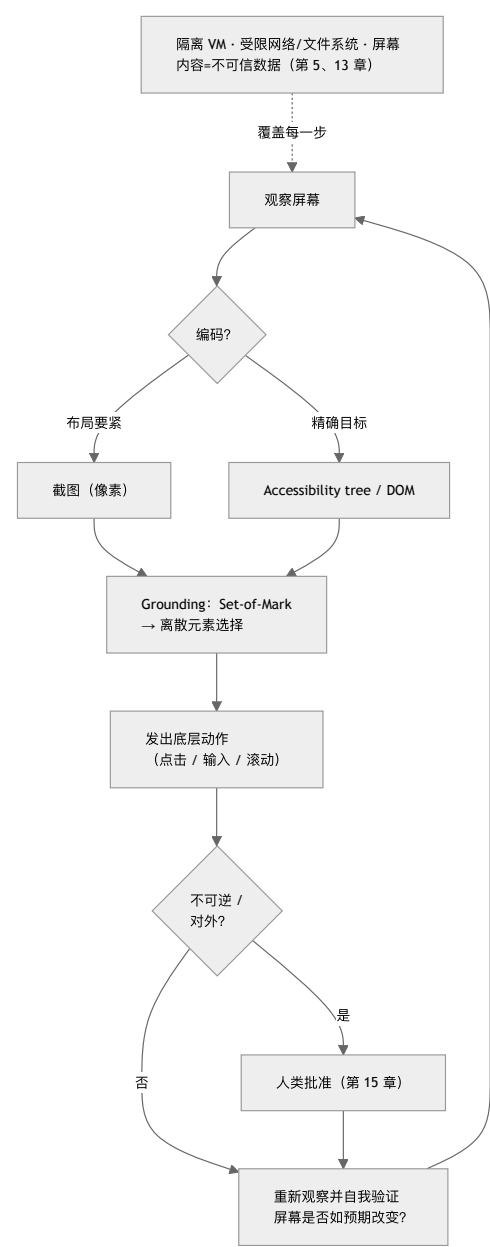
防御仍然依靠第 5 章介绍的 sandboxing 和 governance，只是动作面越宽，控制就必须越严格。应让 agent 在专用 VM 或容器等隔离环境中运行，而不是使用用户的主 session；限制它对网络和文件系统的访问；在执行不可逆或对外动作前要求人类批准 (第 15 章)。屏幕上显示的一切都应被视为不可信 data，而不是指令。当“data”是一整张渲染后的网页时，第 13 章的 instruction hierarchy 就成为关键防线。

16.8 评估 computer-use agent

由于环境是整个操作系统或浏览器，评估必须衡量环境中的实际结果，而不能只检查文本。第 10 章对 *outcome* 的强调在这里无法回避：真正重要的是任务是否确实在环境中完成。专门设计的 benchmark 展示了这种评估方式。WebArena 提供可自托管的真实 web 环境，并通过执行结果判断任务是否成功 ([WebArena](#))；OSWorld 则把这一思路扩展到真实桌面操作系统中的开放式任务 ([OSWorld](#))。

这些 benchmark 再次印证了本书的主线：computer-use agent 距离稳定完成真实任务仍有很大差距。“模型能看懂屏幕”与“agent 能可靠完成任务”之间，缺少的正是 harness 提供的 grounding 脚手架、重新观察、恢复、范围控制和验证机制。一如既往，真正有决定意义的是面向实际工作负载的 eval (第 10 章)。公开的 computer-use benchmark 只能提供背景证据；agent 能否可靠操作你的应用，才应决定它是否可以发布。

图：Computer-Use 循环



感知 → grounding → 行动 → 重新观察，整个过程都由 sandbox 包裹。Grounding 把“看见”转化为“点击”，重新观察用于应对不断变化的屏幕状态，sandbox 则约束了本书涉及的最宽攻击面。

要点

- **Computer-use agent** 以更难控制的 loop 换取更广的触达范围：它们像人一样操作 GUI，因此能使用没有 agent API 的软件。
- **环境本身就是动作面**：工具筛选让位于感知和范围控制；harness 必须帮助 agent 准确识别可用动作，并限制它可以在哪里行动。
- **屏幕编码是 harness 的设计选择**：像素、DOM、accessibility tree 和 OCR 保留的信息各不相同；组合使用更稳健，却会占用更多 context。
- **Grounding 是 computer-use 特有的失败模式**：把意图转换为正确点击很容易出错；Set-of-Mark 等离散选择技术通常比直接生成坐标更可靠。
- **最宽的攻击面需要最严格的 sandbox**：computer-use agent 集中了 lethal trifecta，因此必须隔离运行、限制权限、为不可逆动作设置审批，并把整个屏幕视为不可信 data。

延伸阅读

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024.
<https://www.anthropic.com/news/3-5-models-and-computer-use>

- OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
- Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
- Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
- Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
- Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>

第 17 章：AgentOps——成本、隐私与生产运维

前面的章节已经构建出一个可以工作的 agent。但要让它进入生产环境并服务真实用户，还必须处理三个容易被 agent loop 掩盖的问题：每次运行花费多少、如何处理敏感数据，以及在用户开始依赖系统之后，怎样安全地修改 harness。这些问题很容易被忽略，却往往决定了一个可运行的 demo 能否成为可靠的产品。第 5 章介绍了安全威胁模型，第 11 章讨论了测量噪声。本章将在此基础上讨论运维经济性、数据治理和发布纪律。

这套运维方法正逐渐被称为 **AgentOps**。AWS 将其归纳为四个相互关联的支柱：**治理与安全 (governance and security)**、**构建与运维 (build and operations)**、**评估 (evaluation)** 和 **可观测性 (observability)** ([AWS - AgentOps](#))。这个概念的价值在于，它覆盖 agent 的完整生命周期：从规划、开发、构建和测试，到部署、维护、监控，直至最终退役。生产责任贯穿整条路径，而不是从打开 monitoring dashboard 才开始。

AgentOps 也改变了“发布什么”这个问题的答案。可部署的 artifact 不只是模型代码或一段 prompt，而是一套版本化的完整配置，其中包括模型及推理设置、system 与 project instructions、tool catalog 与 schema、memory policy、sandbox image、identity 与 authorization policy、grader、预算和路由规则。Release record 应明确记录这套配置，以便工程师把事件归因到具体版本，并在必要时完整 rollback。第 18 章会再向上一层：控制平面负责注册这些 artifact，管理它们的身份和 fleet lifecycle；AgentOps 则负责日常运行并持续改进它们。

17.1 成本：给 agent 做预算

Agent 通常是运行模型成本最高的方式，因为一个用户请求可能扩展为多次模型调用和工具往返；使用推理模型时，还可能产生很长的内部 token 流（第 14 章）。总成本也很难预先判断：开放式 loop 可能运行三个回合，也可能运行三十个。这正是本书反复强调“采用能够奏效的最简单方案”（第 6 章）的运维原因——每增加一个回合，就会增加一笔费用。

因此，生产环境中的 agent 必须有明确的预算，就像第 7 章中的长时运行 agent 必须设置 checkpoint 一样。实践中需要考虑：

- **每项任务的上限**：限制 token、工具调用次数或按实际运行时间计算的 cost。达到上限后，agent 应停止运行并请求进一步指示，而不是无限循环。没有边界的 loop 可能迅速变成一笔失控的账单。
- **前文介绍的成本杠杆**：把简单步骤路由给成本较低的模型，只在确有需要时使用前沿模型和推理模型（第 14 章）；缓存稳定前缀，避免每个回合都为重复 context 付费（第 2 章）；让工具返回 token 效率更高的结果，并用 code execution 处理大体量数据，避免直接把它们放进 context（第 4 章）。
- **是否应该构建 agent**：最便宜的 agent 是根本不需要运行的 agent。许多任务更适合 workflow 或单次模型调用，成本应成为设计选择的一部分（第 6 章）。

第 14 章介绍的级联和路由模式既能控制质量，也能控制成本。FrugalGPT 的核心结论是：与朴素的“始终调用最佳模型”策略相比，大部分费用都可以避免，同时不必牺牲准确率 ([FrugalGPT](#))。

缓存需要进一步区分，因为这里涉及两种不同的机制。第 2 章介绍的 KV/前缀缓存以较低成本处理完全相同的前缀；语义缓存 (semantic cache) 则用于处理相似请求。它先将新查询编码为向量，如果发现足够接近的历史查询，就直接返回已存储的响应。GPTCache 推广了这种做法 ([Fu Bang - GPTCache](#))。对于高频、重复性强的查询，语义缓存可以省去整次模型调用，而不只是降低调用成本。

但语义缓存也带来了前缀缓存没有的正确性风险。一次错误命中，可能会把某个缓存答案返回给一个只是表面相似的问题，造成不易察觉的错误。因此，语义缓存需要调优：相似度阈值决定了成本节省与错误匹配风险之间的权衡，它也必须像其他 harness 变更一样经过严格 eval（第 17.5 节）。两种缓存以及前述的按 key 预算，都适合在 AI 网关（第 14.8 节）统一执行。由于每次模型调用都会经过网关，它可以为不同 agent 一致地应用成本控制。

17.2 成本归因与 trace

只有先测量成本，预算才能真正执行，而 trace 是最自然的测量位置。第 12 章介绍的 span telemetry，是一棵由模型调用、工具调用和检索组成的 span 树，也可以同时充当成本账本。只要为每个 span 记录 token 数和美元成本，trace 就能说明 agent 做了什么，也能显示每一步花了多少钱。这样，“agent 很贵”就能转化为“这个工具每次调用都会返回 8000-token 的结果”等具体观察，工程师也就有了可以着手解决的问题。

成本归因应进入与失败分析相同的迭代 loop。如果某个步骤成本很高、贡献却很小，就可以考虑改用更便宜的模型、缩短工具响应，甚至删除这一步。这相当于把第 12 章的 trace 驱动迭代用于成本，而不是正确性。正如第 11 章所提醒的，成本和资源配置还可能影响测得的行为，因此成本应与能力一并报告，而不应被视为彼此独立的指标。

17.3 隐私与数据治理

Agent 每次运行都会处理数据：读取文件、查询数据库、检索文档，并向模型供应商和工具发送请求。每一条路径都可能泄露敏感数据，而 harness 负责控制这些路径。第 5 章所说的 *lethal trifecta* 会让威胁尤为严重：如果 agent 能访问私

有数据、接收不可信内容，又能与外部通信，攻击者就可能诱导它外泄数据 (Simon Willison — The lethal trifecta)。因此，隐私并不是独立于安全的另一个问题，而是同一问题在数据处理层面的体现。

相关的 harness 层控制包括：

- **尽量减少进入 context 的数据。** Agent 从未接触的数据不可能由它泄露。只检索任务需要的部分，而不是整条记录（第 2 章）；能使用 handle 时，优先使用 handle，而不要直接传入敏感内容。
- **在边界处脱敏。** 在内容进入模型 context 或 trace 前，移除 secret 和个人可识别信息 (PII)。Trace 会长期保存，而且常被发送到第三方 observability 工具，因此未经脱敏的 trace 本身就可能成为数据泄露源（第 12 章）。
- **在检索阶段执行权限检查。** 应在检索前根据用户有权查看的数据进行过滤，而不是生成后再处理。Agent 不能成为用户越权读取数据的途径（第 5 章、第 12 章）。
- **管控数据出口。** Harness 可以切断 trifecta 中的对外通信路径：限制 agent 可以把数据发送到哪里，并要求对外发送必须经过批准（第 15 章）。

17.4 多租户与隔离

当一个 agent 平台同时服务多个用户或组织时，隔离就不再只是附加的安全措施，而是系统正确性的基本要求。这里尤其需要防范两种失败。*租户间状态泄漏 (state bleed)* 是指一个租户的 context、记忆或缓存前缀出现在另一个租户的 session 中；如果引入记忆（第 3 章）或前缀缓存（第 2 章）时没有按租户划分范围，就很容易造成这种灾难性后果。*租户间权限泄漏 (authority bleed)* 则是指 agent 使用一个租户的凭据代表另一个租户行动，它是第 5 章所讨论的 delegated authorization 问题在多租户环境中的表现。

所需的工程纪律与第 7 章的 managed-agent 架构一致：为每个租户建立持久工作区，把 sandbox 限制在单个租户的资源范围内，并使用权限范围明确、绝不跨 brain/hands 边界共享的凭据。Event log（第 9 章）还应记录租户身份，使每项动作都能正确归因；这也是审计（第 5 章）在共享系统中真正有效的前提。

17.5 发布 harness 改动

Harness 也是软件，修改一套已经被用户依赖的软件必须遵守发布纪律。Harness 变更的特殊风险在于，其影响往往是统计性的，而不是确定性的。修改 prompt、增加工具、更换模型或调整检索策略，都可能改变系统在未测试输入上的行为。回归也未必表现为明显崩溃，而可能只影响几个百分点的案例（第 10 章、第 12 章）。

这种风险要求三项发布实践：

- **每次变更都要通过 eval 闸门。** 在变更前后分别运行 eval 套件，比较 pass rate、失败类别、成本和延迟。第 10 章介绍的回归测试纪律，就是这里的发布条件。
- **逐步放量。** Eval 套件永远无法覆盖完整的输入分布，因此应把 harness 变更当作其他高风险部署一样处理：先向一小部分流量发布，观察生产 trace 和 outcome 指标，并随时准备 rollback。Canary 可以在限制影响范围的同时，发现 eval 套件遗漏的问题。
- **把完整配置作为一个版本发布。** 第 12 章讨论的 model-harness 耦合意味着，发布单位应是包含模型、prompt、工具和检索策略在内的完整配置，而不是其中某个单独组件。Readiness validation（第 10 章）应针对整套配置，rollback 时也必须恢复整套配置，而不只是还原 prompt。

17.6 监控、SLO 与事件响应

Eval 在发布前提供证据，监控则在发布后持续运行。两者回答的问题不同：eval 衡量精心构造的输入分布，生产环境则揭示真实的输入分布。第 12 章介绍的 span telemetry 构成了监控底座。在此之上，agent 系统还需要常见的生产监控能力，并针对 agent 的行为特点进行调整：

- **衡量 outcome，而不只是 uptime。** API 能够响应的 liveness 检查，并不能说明任务是否成功。任务成功率、升级和批准率、每项任务的成本，以及延迟分位数 (p50/p95/p99，见第 6 章) 都应成为一等监控信号。
- **对漂移发出告警。** 失败率上升、每项任务成本增加或 self-verification 通过率下降，即使没有组件崩溃，也可能构成一次事件。这些都是 agent 系统特有的失败形态。
- **把生产失败转化为回归案例。** 第 12 章的 trace-to-eval loop 同时也是事件响应 loop。每次生产失败都应先脱敏，再转换为可复现的 eval case，并修复根本问题，避免同类事件在没有察觉的情况下再次发生。

17.7 治理框架

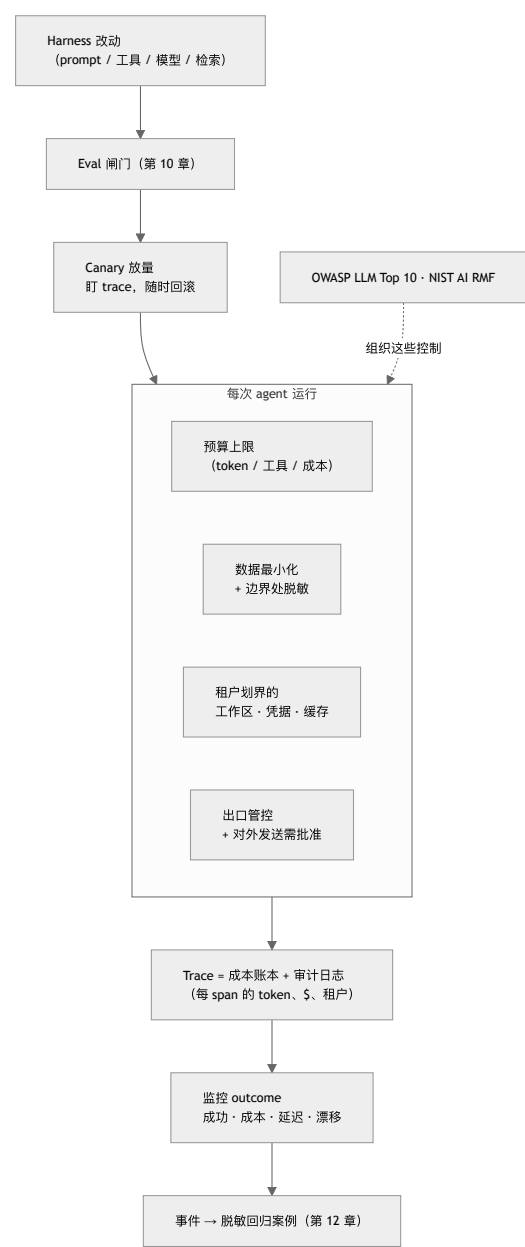
这些问题不仅属于内部工程。对于许多部署，它们同时也是外部要求。以下四个参考框架可以帮助组织相关工作，范围从自愿采用的指南一直延伸到具有约束力的法律。

- **OWASP Top 10 for LLM Applications** 列出了常见风险类别，包括 prompt injection、敏感信息泄露、过度自主 (excessive agency) 和供应链风险。这些类别与第 5 章及本章介绍的控制措施高度对应 (OWASP - Top 10 for LLM Applications)。

- **NIST AI Risk Management Framework** 提供了更高层的组织结构——govern、map、measure、manage——适用于需要为 agent 部署建立可审计风险流程的组织 ([NIST - AI Risk Management Framework](#))。
- **ISO/IEC 42001:2023** 是首个 AI 管理体系标准，可以视为 ISO 27001 在 AI 领域的对应标准。它规定组织应如何建立、运行并持续改进 AI 管理体系，并允许组织通过认证 ([ISO/IEC 42001:2023](#))。
- **欧盟《AI 法案》(EU AI Act)** (Regulation (EU) 2024/1689) 是首部全面的 AI 法律。它按风险等级对系统分类，并对高风险用途规定具有约束力的义务，包括风险管理、数据治理、透明度、人类监督和日志记录；相关条款将在 2026-2027 年间分阶段生效 ([EU AI Act - Regulation \(EU\) 2024/1689](#))。EU AI Act 与 ISO 42001 彼此互补，但并不等同：ISO 42001 认证可以证明管理流程健全，却不能自动证明系统符合 EU AI Act。

这些框架不能替代本书介绍的工程实践。它们的作用是组织这些实践，并让审计方、客户和监管机构能够理解。[展望](#)把跨层治理一致性列为一个开放问题，原因正在于 policy、审计和运行时强制仍分布在彼此独立的层中。这些框架是目前用于保持各层对齐的最佳脚手架。

图：生产外壳



每次运行都要执行成本、隐私和租户隔离控制；trace 同时充当成本账本和审计日志；每项 harness 变更都必须先通过 eval 闸门和 canary，才能进入生产环境。

要点

- **Agent 需要明确预算**：为每项任务设置 token、工具调用或成本上限，并通过路由、级联、缓存和 token 高效的工具控制开支，避免开放式 loop 产生失控账单 ([FrugalGPT](#))。

- **Trace 是成本账本**：记录每个 span 的 token 数和美元成本，才能把“agent 很贵”转化为具体、可修复的工程问题。
- **隐私是 lethal trifecta 在数据处理层面的体现**：减少进入 context 的数据，在边界处脱敏，包括 trace；在检索阶段执行用户权限，并管控数据出口。
- **多租户系统必须隔离**：工作区、凭据、记忆和前缀缓存都要按租户划分范围，event log 也要记录租户身份。租户间状态泄漏和权限泄漏都会造成灾难性后果。
- **两种缓存具有不同的风险**：前缀缓存以较低成本复用相同 context；语义缓存可以为相似查询省去整次调用，却可能因错误命中返回错误答案。必须调好相似度阈值，并像评估其他 harness 变更一样评估它。
- **发布 harness 变更是一种统计性部署**：使用 eval 把关，逐步 canary，把完整配置作为一个版本发布，并将生产失败转化为回归案例。
- **AgentOps 覆盖完整生命周期**：治理与安全、构建与运维、评估和可观测性从规划一直贯穿到退役；模型、工具、记忆、policy、sandbox、grader 和预算应作为一套版本化配置共同发布。
- **治理框架从指南延伸到法律**：OWASP LLM Top 10 和 NIST AI RMF 用于组织控制措施，ISO/IEC 42001 用于认证管理流程，欧盟《AI 法案》则对高风险用途规定了具有约束力的义务。

延伸阅读

- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
- OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
- NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
- *ISO/IEC 42001:2023 - Information technology - Artificial intelligence - Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
- *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>

第 18 章：Agent Fleet、身份与控制平面

Loop engineering 研究的是单个自治任务应如何启动、执行、验证和停止。到了生产环境，设计对象会扩大为一支 agent fleet：许多团队构建的多个 agent，运行在不同环境中，甚至跨越组织边界。此时，难题也从单条 loop 上升到整个 fleet：系统中究竟有哪些 agent？当前版本由谁发布？它在代表谁行动？它可以访问哪些工具、数据、资金和网络目的地？运维人员如何暂停、升级或撤销正在运行的 agent？又有哪些证据能把一次动作准确关联到产生它的身份、策略、模型和 artifact？

一种逐渐形成的答案是 **agent control plane (agent 控制平面)**。这个概念沿用了分布式系统中数据平面与控制平面的区分。*数据平面 (data plane) *负责实际工作，包括模型调用、工具调用、代码执行和 agent-to-agent 消息；*控制平面 (control plane) *则声明并执行这些工作所处的运行状态，涵盖 registry、identity、authorization、routing、lifecycle、policy、lineage 和 audit。控制平面不会让单个 agent 更聪明，但能让整支 fleet 受到治理。

18.1 从 Agent Program 到 Agent Fleet

单个 agent 只需一段 prompt、几个工具和一个本地状态文件就能完成配置。但在 fleet 规模下，本地配置无法回答所有运维问题，新的问题会随之出现：

- 重复建设或已经废弃的 agent，其归属责任并不清楚；
- 多个版本使用同一个名称，实际行为却不相同；
- credential 被复制到 prompt、sandbox 或环境变量中；
- 工具访问权在授权它的任务或用户结束后仍然有效；
- 同一项 policy 在聊天、定时运行和 A2A 委派中执行方式不一；
- 发生事件后，无人能还原究竟是哪份 artifact 或哪项权限导致了相关动作。

第 9.2 节讨论的平台化转向已经预示了这一更大的范围。Fleet 需要一个关于 agent 和 capability 的可信信息源，需要能签发和撤销任务级访问权的运行时授权机制，也需要由生命周期服务持续协调“应该运行什么”和“实际正在运行什么”。AgentOps (第 17 章) 负责系统在整个生命周期中的运行与改进；控制平面则提供这些工作共同依赖的治理层。

边界应保持明确：

- **数据平面**：inference、retrieval、工具执行、sandbox 进程、消息和结果。
- **控制平面**：definition、identity、policy decision、placement、version、budget、revocation 和 audit。

核心设计原则是：只要条件允许，就不要让 policy decision 落在数据路径中具有概率性的部分。Agent 可以提出一项动作，但具名身份能否在当前情境下执行该操作，应由确定性的控制平面服务决定。

18.2 Agent Identity：谁——或什么——在行动？

只有 user identity 还不够。同一个用户可能启动多个用途和权限不同的 agent。发起 session 结束后，agent 仍可能按计划继续运行，也可能把工作委派给另一个 agent，或与自身的其他版本同时运行。**Agent identity** 是 agent 这一行为主体持久的机器身份；它不同于某次运行涉及的人类、service account、runtime process 和 model。

NIST 关于软件 agent 身份与权限的概念论文，将问题拆分为 identification、authentication、authorization、delegation、audit 和 non-repudiation。论文特别指出，prompt injection 和 confused-deputy behavior 表明普通的 service identity 不足以处理 agent 场景 ([NIST - Identity and Authorization for Software Agents](#))。在实践中，一份 identity record 应回答：

- **Principal**：正在行动的是哪个 agent definition 和 version？
- **Sponsor**：哪个人或组织对它负责？
- **Delegator**：这次运行具体代表谁行动？
- **Purpose**：哪项范围明确的任务或 workflow 是授予权限的依据？
- **Runtime**：当前使用的是哪些 harness、model、sandbox image 和 policy version？
- **Lifetime**：identity 在何时签发，又会在何时过期或可被撤销？

不要用一把长期 API key 代表整条身份与授权链。Fleet 应使用 workload identity 和短期 task token，并明确记录 delegation chain。Agent 先以自身身份完成认证，authorization service 再结合委派用户、tenant、purpose、environment 和 requested operation 作出决定。将两者分开，可以防止低风险的 research agent 在不知不觉中继承启动者的全部权限。

18.3 Agent Registry

Agent registry 是 fleet 的资产清单和发现层。它让人类、orchestrator、gateway 和其他 agent 知道系统中有哪些对象，以及它们将要信任哪份 artifact。Google Cloud Agent Registry 把 agent 与 MCP server、endpoint、skill、skill

revision 和 publisher 放在同一套模型中管理。AWS Agent Registry 与 Microsoft 365 Agent Registry 也体现了同一趋势：把散落的 deployment URL 收拢为受治理的 catalog ([Google Cloud - Agent Registry Overview](#); [AWS - Agent Registry in AgentCore](#); [Microsoft - Agent Registry](#))。

一条有用的 registry entry 不能只有名称和描述，还应包括：

- 不可变的 agent identifier 和 version identifier；
- publisher 和明确承担责任的 owner；
- 支持的 task、input/output contract 和 endpoint；
- 对 tool、MCP、skill、memory、model 和 sandbox 的依赖；
- 申请的 permission、data class、network destination 和 budget class；
- 该版本的 eval 结果和 security attestation；
- deployment status、deprecation date 和 revocation state；
- 可追溯到 source、build、configuration 和 policy bundle 的 lineage。

Discovery 与 governance 必须在 registry 中汇合。搜索结果只能显示调用方有权调用的 agent，orchestration 也应解析到不可变的版本，而不是可随时修改的显示名称。注册不等于批准：一条新记录可以作为 draft 对外可见，却尚未获准承接 production traffic。版本 promotion 应通过 ownership、eval、security 和 policy gate；在 deprecate 或移除版本之前，registry 还应先找出所有依赖项。

18.4 Runtime Authorization 与 Delegation

Registration 说明一个 agent 是什么；authorization 则决定某次具体运行此刻可以做什么。单次运行获得的权限范围，应小于这个 agent 在所有场景下可能用到的权限总和。

一条稳健流程如下：

1. User 或 service 启动一次 run，并声明 purpose 和 tenant。
2. 控制平面解析到一个已批准的 agent version。
3. Policy 根据 agent capability、delegator authority、task need、environment 和 risk 的交集，计算允许的范围。
4. Identity broker 签发短期、audience-bound 的 credential 或 capability token。
5. Proxy 在 tool boundary 附加 credential；raw secret 不进入 model context 或 sandbox。
6. 每次使用都记录 identity、delegation chain、policy version、operation、resource 和 result。
7. Run 完成或超时后撤销 lease；policy change 或 incident 也可以触发撤销。

AWS AgentCore Identity 展示了这一模式如何落地为产品：提供 agent identity 和 credential provider，并在不把 user credential 嵌入 agent code 的情况下，委派访问外部资源的权限 ([AWS - AgentCore Identity](#))。其架构原则并不依赖具体厂商：权限应按需即时签发，并且只在真正需要它的边界上呈现。

A2A delegation 应明确记录权限如何逐级传递。Agent A 不应把包含自身全部权限的 bearer token 直接交给 Agent B，而应申请一份范围更窄的 token：以 B 为 audience，以 delegated task 为 purpose，限定允许的 operation 和 resource，并设置很短的有效期。如果 B 继续委派，新权限也不得超过现有委派链所授予的范围。这样可以阻断第 3 章所述的信任升级：低信任 worker 不能仅靠说服高权限 parent，就把自己的结果转化为高权限动作，而绕过一次新的 policy decision。

18.5 Agent Gateway 与 Policy Enforcement

Agent gateway 是数据路径上的 policy enforcement point。它可以代理 model call、MCP 和 API tool call、A2A message、网络 egress，有时也会管理 memory access。“绝不发送 secret”这类自然语言指令只能带来概率性的约束；gateway 则可以用确定性规则拒绝未授权的目的地、移除 credential、限制 budget，或要求 human approval。

每条请求都应附带一份经过签名或以其他方式可验证的 execution envelope：

```
agent identity + version
delegating principal + tenant
task purpose + run/session id
requested action + resource
policy and configuration versions
budget and expiry
trace/span correlation id
```

Policy 不必只返回 allow 或 deny 两种结果，还可以返回 **allow with redaction**、**allow in a stronger sandbox**、**require human approval**，或 **allow with a lower budget or read-only scope**。Decision 及其输入都应写入 trace。这样，第 15 章按风险调整交互强度的控制，以及第 5 章的 containment matrix，就能在整个 fleet 中一致执行，而不必由每支团队各自重写一套约定。

Gateway 的部署位置同样重要。Central gateway 能提供统一的 enforcement 和 visibility，但也可能成为 latency bottleneck 和 single point of failure。Local sidecar 可以降低延迟，并在部分连接中断时继续工作，却会增加 policy distribution 和 evidence collection 的难度。成熟的设计通常把集中式 policy administration 与分布式 enforcement 分开：通过 signed policy bundle 下发规则，对后果重大的动作采用 fail closed，只允许低风险读取在经过审慎评估后 fail open。

18.6 Fleet Lifecycle: Reconcile、Suspend、Upgrade、Revoke

Agent 的生命周期不能只用 deployed 和 stopped 两种状态表示。一套实用的 fleet state machine 应包括 **draft**、**evaluated**、**approved**、**deployed**、**suspended**、**deprecated**、**revoked** 和 **retired**。Run 有自己的状态机——queued、active、waiting for approval、checkpointed、completed、failed 或 canceled——sandbox 还会有另一套状态机（第 7 章）。控制平面需要关联这些生命周期，但不能把它们混为一谈。

这里的核心治理模式是 reconciliation：比较 declared state 与 observed state，再通过确定性动作消除两者之间的差异。

- 如果某个 agent version 被 revoked，应阻止新的 run、撤销 credential，并根据风险暂停或终止受影响的 in-flight run。
- 如果 policy 发生变化，应找出哪些 session 需要重新 authorization，而不是让旧权限无限期延续。
- 如果 version 需要升级，应先在有限的 traffic slice 上进行 canary，同时保留旧版本以便 rollback（第 17 章）。
- 如果 owner 离职或 publisher 遭到入侵，应遍历 registry 的 dependency graph，找出继承该信任的 agent、skill、MCP server 和 schedule。
- 如果 run 失去连接，应保留 durable session，重新获取 sandbox 或 worker，而不是重复创建同一任务（第 7 章）。

Fleet kill switch 也属于这一层。它应在 gateway 和 identity broker 处撤销 capability，而不只是向模型发送一句“stop”。控制范围必须足够精细：operator 应能单独停止某个 run、version、publisher、tenant、tool dependency，或在必要时停止整个 fleet，而不必每次 incident 都动用范围最大的开关。

18.7 Lineage、Audit 与 Non-Repudiation

普通日志回答的是“哪个请求到达了 this service？”Agent audit 则必须回答一条更完整的因果问题：

哪个 user 或 service，出于什么 purpose，把任务委派给了哪个 agent version？它使用了哪些 model、harness、tool、memory、sandbox 和 policy？哪些 evidence 导致了哪项 decision？哪项 authority 允许产生 side effect？又是谁批准或 override 了这项动作？

答案应呈现为一张 lineage graph。它把 registry artifact、build attestation、identity、delegation event、session 与 run ID、trace span、policy decision、human approval、tool result、memory write 和环境中的最终结果连接起来。稳定的 content hash 或不可变的 version identifier，可以防止后续编辑改写“当时实际运行了什么”这段历史。

使用 ****non-repudiation**（不可否认性）一词时需要谨慎。Signed event 可以证明某个 workload identity 发出了请求，也可以证明某个 policy service 授权了该请求；但它无法证明某个人真正理解了 approval dialog，也无法证明模型给出的自然语言 rationale 属实。因此，可靠的 audit 应把 cryptographic integrity 与 operational evidence 结合起来，包括 append-only log、trusted timestamp、credential 和 policy version、outcome check、approval identity 与 retention rule。系统既要保留足够的信息供调查使用，也要遵守第 17 章的隐私原则。一个不加筛选地保存 prompt、secret 和 personal data 的 audit 系统，本身会制造新的安全问题。

Lineage 还能让 evaluation 和改进过程更加安全。生产环境中的一次纠错可以追溯到实际失败的 artifact，再转化为脱敏的 regression case，用于 promotion 新版本，同时保持旧版本不变（第 12 章）。控制平面保存 chain of custody，eval gate 则提供版本 promotion 所需的证据。

18.8 控制平面的开放问题

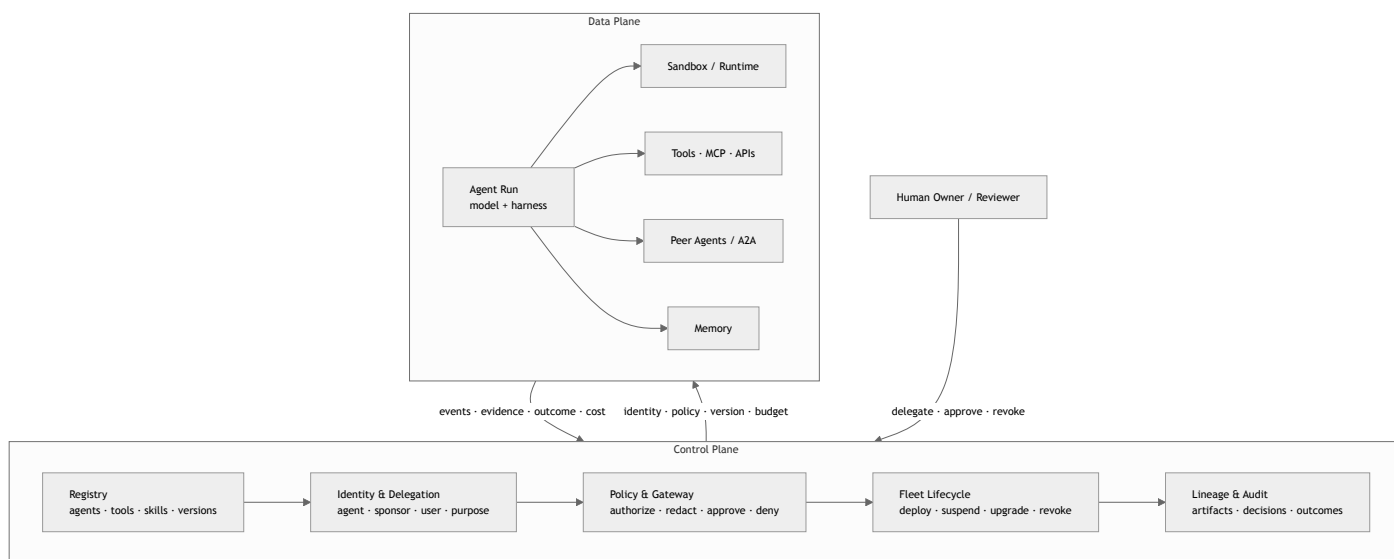
控制平面仍在发展之中，其中最重要的开放问题包括：

- **可迁移的身份与委派**。Agent Card 和 registry 可以描述 capability，但 principal identity、delegation chain、revocation 和 purpose-bound authority 仍缺少成熟的跨平台标准。
- **Policy composition**。User policy、tenant policy、agent policy、tool policy、data-residency rule 和 human approval 可能彼此冲突。要确定采用哪项规则并解释原因，需要一套可迁移的语义。

- **Memory governance**。不同 memory product 尚未以一致方式表达 provenance、permission-at-retrieval、expiry、correction 和 poisoning defense。
- **跨组织 lineage**。A2A chain 会跨越多个 audit domain。每个组织可能只提供很少的证据，让对方无法评估信任；也可能披露过多 private trace data。
- **Evaluator authority**。Model judge 可以阻止 deployment 或触发 incident。对于它的 identity、calibration、conflicts of interest 和 appeal path，应采用与其他 privileged decision-maker 同等严格的要求（第 10 章）。
- **Fleet-level supervision**。Risk ranking、alert deduplication 和有意义的人类控制都必须能随 fleet 扩展，同时又不能让 operator 沦为只会盖章的角色（第 15 章）。
- **控制平面本身被攻陷**。集中的 registry、policy service 和 identity broker 都是高价值目标。它们较大的 blast radius 要求系统采用 separation of duties、signed artifact、尽可能小的 trust root，以及独立的 recovery path。

标准虽然尚未定型，方向却已经清楚：agent 正在成为分布式系统中的 principal。成熟的 harness 不再只是围绕模型的一条 loop，而是一套受到治理的关系，将 identity、artifact、policy、environment、evidence 和人连接在一起。

图：Agent 控制平面



数据平面负责实际工作；控制平面决定这项工作受哪些 identity、artifact、authority、lifecycle 和 evidence 约束。

要点

- **Fleet 是 loop 之后更大的设计单元**：loop engineering 治理单个 task；控制平面治理多个 agent、version、environment 和 organization。
- **分离 data plane 与 control plane**：agent 提出并执行工作；确定性 service 负责注册 artifact、签发 authority、执行 policy、协调 lifecycle，并保存 evidence。
- **Agent identity 不同于 user identity 和 runtime identity**：每次 run 都应绑定 agent version、sponsor、delegator、purpose、environment 和 expiry。
- **Registry 是受治理的 source of truth**：除了 discovery metadata，还应包含 ownership、dependency、permission、attestation、deployment state 和 immutable lineage。
- **Authority 应按需签发并限定用途**：使用短期、audience-scoped credential，在 tool boundary 附加，并确保 delegation 只能缩小权限范围。
- **Gateway 把 policy 落实为 enforcement**：它可以 allow、deny、redact、加强 isolation、缩小 scope 或要求 approval，并记录作出决定的原因。
- **Lifecycle management 的核心是 reconciliation**：分别管理 agent definition、run 和 sandbox，让 suspension、migration、canary deployment、rollback 与 revocation 成为一等操作。
- **Audit 是一张因果 lineage graph**：它连接 user delegation、agent 和 policy version、evidence、authorization、side effect 与 outcome，同时尽量减少 sensitive content。

延伸阅读

- NIST, *Identity and Authorization for Software Agents*, Feb 2026. <https://www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-software-agents>
- Google Cloud, *Agent Registry Overview*, 2026. <https://docs.cloud.google.com/agent-registry/overview>
- AWS, *AWS Agent Registry in Amazon Bedrock AgentCore (Preview)*, Apr 2026. <https://aws.amazon.com/about-aws/whats-new/2026/04/aws-agent-registry-in-agentcore-preview/>
- AWS, *Introducing Amazon Bedrock AgentCore Identity: Securing Agentic AI at Scale*, 2026. <https://aws.amazon.com/blogs/machine-learning/introducing-amazon-bedrock-agentcore-identity-securing-agentic-ai-at-scale/>
- Microsoft, *Agent Registry in the Microsoft 365 Admin Center*, 2026. <https://learn.microsoft.com/en-us/microsoft-365/admin/manage/agent-registry?view=o365-worldwide>
- Anthropic Safeguards Research Team, *How We Contain Claude*, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
- Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026. <https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>
- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>

第 19 章：展望

19.1 这个领域仍然年轻

本书使用的许多词汇——例如 `initializer agents`、`context firewalls`、`sprint contracts`、`reasoning sandwiches`、`ambient affordances`，以及 `computational controls` 与 `inferential controls`——都是在最近十二到十八个月里才进入 `agent engineering` 的主流讨论。部分底层思想虽然出现得更早，但这套共享语言仍然很新。本书引用的多数文章也发表于 2025 和 2026 年。

这个变化速度可以量化。METR 发现，前沿模型完成任务的*时间视野* (*time horizon*)——也就是成功率达到 50% 时，它所能完成的人类任务长度——大约每七个月翻一番（第 7.11 节）([Kwa et al. - Measuring AI Ability to Complete Long Tasks](#))。因此，任何讨论 harness 应提供哪些能力的书，描述的都是一个不断移动的目标。

LangChain 对这条演进路径的判断很直接：随着模型能力提高，今天由 harness 承担的部分职责会被模型吸收。模型会越来越擅长规划、自我验证和维持长周期的一致性，系统也就不必再通过上下文注入这些能力。但模型变强并不会让有价值的 harness 设计越来越少；harness 的重点只会转向新的问题 ([LangChain - The Anatomy of an Agent Harness](#); [Anthropic - Harness Design for Long-Running Application Development](#))。

19.2 开放问题

现有文献反复提到以下开放问题：

- **面向功能正确性的 behavioral harness**。围绕 `maintainability` 和 `architectural fitness`，软件工程已经积累了几十年的工具；但对于一个更基本的问题——应用的实际行为是否符合用户意图——behavioral harness 还没有同等成熟的工具。多数团队目前依赖 AI 生成测试，而普遍看法是，这些测试仍不足以解决问题 ([Thoughtworks - Harness Engineering](#))。
- **规模化 harness 的一致性**。随着 `guide` 和 `sensor` 越来越多，如何保证它们彼此一致？一个 `sensor` 从未触发，究竟说明系统质量很高，还是检测能力不足？Harness coverage 至今还没有类似 `code coverage` 或 `mutation testing` 的衡量方法。
- **控制平面可迁移性**。`Registry`、`identity`、`policy`、`gateway` 和 `audit` 系统已经出现，但它们的 `schema` 与 `authority model` 仍然依赖具体平台。Agent 在不同云或组织之间迁移时，仍很难完整保留 `provenance`、`policy` 含义和 `revocation` 语义（第 18 章）。
- **跨层 governance 一致性**。OpenReview 综述指出，`policy`、`permission prompt`、`audit log`、`constitutional instruction` 和 `runtime hook` 往往分散在不同层中。这些机制未必能顺利组合，反而可能相互干扰。业界仍缺少可迁移的 `policy language` 和 `audit language` ([OpenReview - Agent Harness Engineering: A Survey](#))。
- **标准化 readiness reporting**。模型分数取决于 `execution environment`、`tool surface`、`context policy`、`retry` 规则和 `governance gate`。因此，benchmark 报告需要附带一份 harness bill of materials，说明得分所依赖的配置；但目前还没有被广泛采用的披露格式 ([OpenReview - Agent Harness Engineering: A Survey](#))。
- **Cost-quality-speed 三难**。更强的 `sandbox`、更丰富的 `observability`、更深入的验证和更严格的治理，通常都会增加成本与延迟。成熟的 harness 必须明确规定：哪些检查需要同步执行，哪些可以离线执行，以及哪些风险值得采用成本更高的控制 ([OpenReview - Agent Harness Engineering: A Survey](#))。
- **Capability-control 权衡**。增加工具、`memory`、自治能力和网络访问范围，可以扩大任务覆盖面；同时也会增加工具选择错误、`prompt injection` 暴露面、`provenance` 风险和 `audit` 负担。Capability 和 control 位于同一条设计轴上，不能被当作两个互不相关的问题。
- **超越同步编排的多 agent 协调**。Anthropic 的研究系统以同步方式运行 `sub-agent`。异步协调或许能释放更多并行能力，但也会让结果协调、状态一致性和错误传播变得更难 ([Anthropic - How We Built Our Multi-Agent Research System](#))。
- **Harness 层的持续学习**。Agent 在开始新 session 时，往往无法继承此前获得的知识。如何通过 `memory primitive` 让它们在多个 session 中不断积累对代码库或特定领域的理解，仍是一个活跃的研究方向 ([LangChain - Improving Deep Agents with Harness Engineering](#))。
- **Human-in-the-loop 审批**。目前还没有成熟的界面标准，用来决定 harness 应在何时暂停并请求人工判断，以及应展示多少上下文才能帮助 operator 决策而不令其负担过重。Approval gate 太粗，容易变成例行盖章；太细，又会抵消自治本身的价值。
- **Just-in-time tool assembly**。Harness 不必预先配置所有可能用到的能力，而可以根据当前任务，动态组装所需工具和上下文。LangChain 等团队正在探索这一方向 ([LangChain - Improving Deep Agents with Harness Engineering](#))。
- **Trace 作为文档**。LangChain 提出：“在软件中，代码记录 app；在 AI 中，trace 记录 app。”这一观察指向一种不同的系统文档模式，但相关实践还没有完全成形 ([LangChain - The Anatomy of an Agent Harness](#))。

- **端到端供应链治理**。工具完整性只是问题的一部分。Agent 还依赖 MCP server、外部 package、dataset、retrieval source，以及模型生成的 dependency name。如何记录并验证整条供应链的 provenance，仍然缺少成熟方案 (OpenReview - Agent Harness Engineering: A Survey)。
- **后果压力下的 evaluator integrity**。Model judge 的判断可能受到其 verdict 将如何被使用的影响，generator 也可能学会针对已知 grader 进行优化。盲评、abstention、ensemble 和 meta-eval 都有帮助，但目前没有通用方法能让语义 verifier 既可扩展，又不受判决后果影响（第 10 章）。

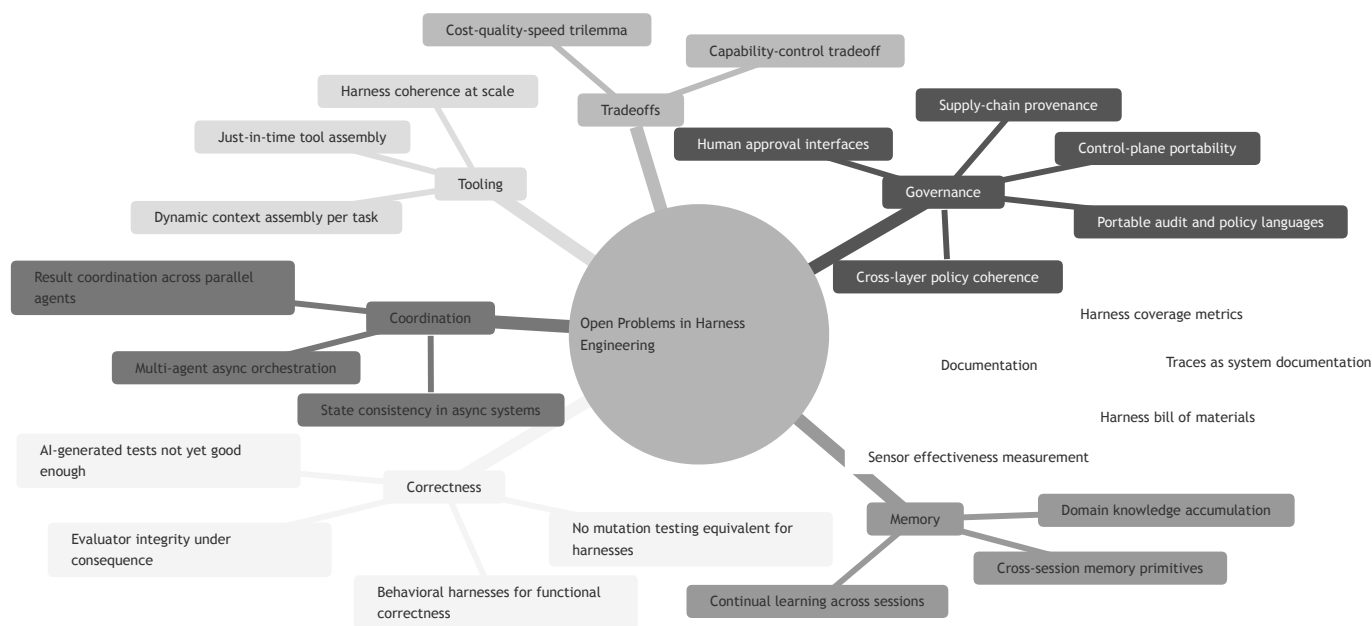
19.3 长期建议

有几条原则贯穿了大部分相关文献：

- **把上下文视为有限资源**。找出能够产生预期结果的最小高信号 token 集合。
- **采用能够解决问题的最简单方案**。Agent 的成本很高；许多任务用 workflow 就已足够，还有一些任务根本不需要两者。
- **阅读 transcript**。它能揭示 agent 在哪里产生困惑、选错工具，或不再使用 harness 提供的指导。
- **持续检验真正关键的组件**。模型变化时，要对各组件进行压力测试；移除不再产生价值的部分，调优仍然关键的部分。
- **根据模型调整 harness，但以持久原则为基础**。不同模型所需的 prompt 和工具会发生变化；context engineering、工具设计、evaluation、sandboxing 和 self-verification 等核心工作仍会存在。

这个领域才刚刚发展到可以尝试编写教材的阶段。本书明知内容会比大多数教材更快过时，仍希望提供一套当前可用的整理。随着讨论继续发展，书中的引用也为读者保留了返回原始资料的路径。

图：开放问题 Mindmap



要点

- **共享词汇仍然很新**：许多术语直到 2025—2026 年才得到广泛使用，尽管其底层思想出现得更早。
- **模型会吸收部分 harness 能力，但 harness 的工作重心会移动**：随着模型原生能力增强，harness engineering 会转向更困难的问题，而不是消失。
- **开放问题已经横跨完整的 ETCLOVG 栈**：包括 behavioral correctness、evaluator integrity、控制平面可迁移性、harness 与 governance 一致性、readiness reporting、cost-quality-speed、capability-control、异步协调、持续学习、just-in-time tool assembly，以及将 trace 用作文档。
- **五条长期原则适用于不同场景**：把上下文视为有限资源、采用最简单可行方案、阅读 transcript、检验关键组件，并根据模型调整 harness。
- **Harness engineering 是一项持续工作**：它不是模型变强后就能丢弃的脚手架，而是围绕能力不断增强的核心，持续构建有效系统的工程实践。

延伸阅读

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>

参考文献

Agent Harness：实践者教材的完整书目。

基础（前置）

- *LLM Foundations for Harness Engineering* —— 本书所依赖的前置 stage-1 卷。 [../llm-foundations-zh/](https://llm-foundations-zh/)
-

基础

- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
 - *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
 - Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
 - Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
 - Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
 - Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
 - Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022. <https://arxiv.org/abs/2210.03629>
 - Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
-

上下文、记忆与工作状态

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 - Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
 - Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
 - Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
 - Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
-

工具与约束

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>

- Simon Willison, *The lethal trifecta for AI agents: private data, untrusted content, and external communication*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
 - *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
 - Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
 - OpenAI, *Connect Private MCP Servers to OpenAI Products*, Jun 2026. <https://developers.openai.com/blog/connect-private-mcp-servers-to-openai-products>
 - OpenAI, *Programmatic Tool Calling*, 2026. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>
-

规格与 workflow 设计

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
-

推理与规划模式

(另见“基础”中的 *ReAct*, 即推理-行动的基础模式。)

- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
 - Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
 - Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
 - Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
 - Bin Feng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>
 - Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
-

多代理系统

- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (提出 MAST 失败分类法), UC Berkeley / arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
-

可靠性与运营安全

- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
 - LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
 - Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
 - Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
 - Anthropic Safeguards Research Team, *How We Contain Claude*, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
 - Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026. <https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>
-

Agent 能力度量

- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025.
<https://arxiv.org/abs/2503.14499>
-

Loop Engineering (循环工程)

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026.
<https://addyosmani.com/blog/loop-engineering/>
 - *Loop Engineering*, O'Reilly Radar, 2026.
<https://www.oreilly.com/radar/loop-engineering/>
 - Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026.
<https://www.deeplearning.ai/the-batch/>
 - *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026.
<https://thenewstack.io/loop-engineering/>
 - *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026.
<https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
 - *Loop Engineering Guide (2026)*, AI Builder Club.
<https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
 - *Loop Engineering Crash Course*, The AI Agent Factory (Panaversity).
<https://agentfactory.panaversity.org/docs/loop-engineering-crash-course>
-

评估与可观测性

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
 - Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
 - Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
 - OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
 - Anthropic Safeguards Research Team, *Agentic Misalignment: Summer 2026 Update*, 2026.
<https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>
 - OpenAI, *Trustworthy Third-Party Evaluations: Foundations*, 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
 - OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>
-

运行时、Harness 与参考实现

- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, Oh My!, LangChain, Oct 2025.
<https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
 - Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025.
<https://www.anthropic.com/engineering/multi-agent-research-system>
-

指令与 Prompting

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024.
<https://arxiv.org/abs/2404.13208>
 - OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>
-

模型选择、路由与推理

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
 - Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
 - Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
 - LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
 - Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>
-

人-Agent 交互

- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
 - Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
-

Computer-Use 与多模态 Agent

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024. <https://www.anthropic.com/news/3-5-models-and-computer-use>
 - OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
 - Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
 - Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
 - Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
 - Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>
-

成本、隐私与治理框架

- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>
 - OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
 - NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
 - ISO/IEC 42001:2023 — *Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
 - *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
 - Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
-

Agent Fleet、身份与控制平面

- NIST, *Identity and Authorization for Software Agents*, Feb 2026. <https://www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-software-agents>

- Google Cloud, *Agent Registry Overview*, 2026. <https://docs.cloud.google.com/agent-registry/overview>
 - AWS, *AWS Agent Registry in Amazon Bedrock AgentCore (Preview)*, Apr 2026. <https://aws.amazon.com/about-aws/whats-new/2026/04/aws-agent-registry-in-agentcore-preview/>
 - AWS, *Introducing Amazon Bedrock AgentCore Identity: Securing Agentic AI at Scale*, 2026. <https://aws.amazon.com/blogs/machine-learning/introducing-amazon-bedrock-agentcore-identity-securing-agentic-ai-at-scale/>
 - Microsoft, *Agent Registry in the Microsoft 365 Admin Center*, 2026. <https://learn.microsoft.com/en-us/microsoft-365/admin/manage/agent-registry?view=o365-worldwide>
-

姊妹卷

- *LLM Foundations for Harness Engineering* (本书的模型内部姊妹卷)。../llm-foundations-zh/
-

阅读列表

- *Awesome Harness Engineering*, walkinglabs. <https://github.com/walkinglabs/awesome-harness-engineering>
- *Learn Harness Engineering*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/>
- *Learn Harness Engineering — Skills*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/skills/>

术语表

本书所用术语的简明定义。括注的章节是该术语被深入讨论的位置;某条定义的来源,可顺着对应章节的行内引用和 [参考文献](#) 查找。

核心概念

Agent(代理) — 语言模型,加上让它能真正做事的那套系统:浏览代码库、运行代码、调用工具、从错误中恢复、支撑多步骤任务。由公式 $Agent = Model + Harness$ 概括(第 1 章)。

Agent harness — 围绕模型工程化出来的一切:系统提示、工具及其描述、内置基础设施、子代理编排、控制流、hooks/中间件,以及评估基础设施(第 1 章)。

Agent loop(agent 循环) — 核心执行循环:组装上下文 → 模型发出工具调用或最终答案 → harness 执行调用 → 把结果追加进上下文 → 重复直到完成(第 1 章)。

Augmented LLM(增强型 LLM) — 配备了检索、工具和记忆的模型,能生成自己的查询、选择工具、决定保留什么。是 agent 的基本构件(第 1、6 章)。

ACI(agent-computer interface,代理-计算机接口) — agent 与其工具之间的设计界面,类比 HCI:agent 如何使用工具,值得投入与"人类如何使用界面"同等的工程(第 4 章)。

Builder harness — AI 实验室作为 coding agent 产品一部分而提供的系统提示和工具;harness 三层同心圆中的中间层(第 1 章)。

User harness — 团队为适配自己代码库而在 coding agent 之上添加的 AGENTS.md、hooks、skills 和 review agents;最外层(第 1 章)。

Harness engineering — 迭代模型周边的整个系统(而不只是单个提示),让每个观察到的失败被永久工程化掉(第 1 章)。

Binding-constraint thesis(约束瓶颈命题) — 对长周期 agent 来说,可靠性常常受 harness 层限制,包括 execution、tools、context、lifecycle、observability、verification 和 governance,而不只是受模型能力限制(第 1、11 章)。

ETCLOVG — Agent harness engineering 的七层分类:Execution environment、Tool interface、Context、Lifecycle、Observability、Verification、Governance(第 1 章)。

Context engineering(上下文工程) — 在推理时策划上下文窗口中最小的一组高信号 token;比 harness engineering 低一层的实践(第 1、2 章)。

MCP(Model Context Protocol,模型上下文协议) — 一个开放的客户端-服务器标准,用于把工具、资源和提示暴露给 agent,使任何兼容客户端都能发现并调用它们,无需定制集成(第 4 章)。

A2A(Agent-to-Agent protocol) — 用于不透明 agentic applications 之间委托的协议边界;它与主要向单个 agent runtime 暴露工具和上下文的 MCP 互补(第 4 章)。

Protocol boundary(协议边界) — 工具或 agent 标准跨越的集成线:model-to-function、agent-to-external-capability、agent-to-agent、agent-to-repo/environment(第 4 章)。

Tool call(工具调用) — 模型发出的结构化输出(通常是 JSON),指明工具名和参数。由确定性的 harness 代码决定如何处理(见《LLM Foundations》第 12 章)(第 4、8 章)。

结构化输出(structured output) — 被约束成机器可读形状的模式输出,通常是 JSON 或 XML,便于软件可靠解析。工具调用是它在 agent 系统中的典型形式(见《LLM Foundations》第 12 章)(第 1、4、8 章)。

上下文与记忆

Context window(上下文窗口) — 模型在一次推理调用中能关注的有限 token 跨度(见《LLM Foundations》第 9 章)。在 harness 中,它是每个系统提示、工具结果和历史轮次都要争抢的预算(第 2 章)。

Context rot — 随着上下文变长,模型准确回忆和使用信息的能力下降(见《LLM Foundations》第 9 章)。其 harness 视角:它是首要的运行约束,在本书中以 attention budget 来刻画(第 2 章)。

Attention budget(注意力预算) — 把上下文视为有限资源、每个新增 token 都在花费它的视角(第 2 章)。

KV-cache — 对已处理 token 的 key/value 张量的缓存(见《LLM Foundations》第 9 章)。相同的上下文前缀可由它服务,把首 token 延迟和成本降低约十倍;在 harness 中,前缀稳定性成为一个生产成本杠杆(第 2 章)。

Prefill / decode(预填充/解码) — prefill 是处理输入提示,decode 是生成输出 token(见《LLM Foundations》第 9 章)。Agentic 工作负载严重偏向 prefill(输入输出比约 100:1)(第 2 章)。

Lost-in-the-middle — 模型对长上下文中段信息的关注,不如对开头和结尾可靠的倾向(见《LLM Foundations》第 9 章)(第 2、3 章)。

Compaction(压缩) — 在对话接近上下文上限时将其总结,并用该总结重新开启一个新窗口。有损(见《LLM Foundations》第 9 章)(第 3 章)。

Context reset(上下文重置) — 完全清空上下文,用结构化 handoff 启动一个全新 agent——区别于原地压缩(第 7 章)。

Just-in-time retrieval(即时检索) — 通过轻量引用(文件路径、查询、链接)按需把数据加载进上下文,而不是预先 embed 一切(第 2 章)。

Recitation(反复复述) — 反复把目标或 todo list 重写到上下文尾部,使其留在模型最近的注意范围内(第 3 章)。

结构化笔记(agentic memory) — 让 agent 把进度笔记写到磁盘,以便上下文重置后重新加载(第 3 章)。

MemGPT — 一种记忆架构,把上下文窗口当作 OS 式的“主存”、把外部存储当作“磁盘”,让模型通过函数调用把信息换入换出(虚拟上下文管理)(第 3 章)。

Mem0 — 一层记忆:跨会话动态抽取、整合并检索显著事实,并有可选的图变体捕获实体关系(第 3 章)。

Sleep-time compute(睡眠期计算) — 在请求之间离线处理上下文——预判可能查询并预计算推断——以削减后续查询所需的计算(第 3 章)。

Persistent memory poisoning(持久记忆投毒) — 不可信内容被提升进持久记忆、并在之后被当作可信状态的攻击,使一次注入能跨过 context reset 操纵未来运行(第 3、5 章)。

子代理与 workflow

Sub-agent(子代理) — 在自己的上下文窗口内处理聚焦任务、只向父代理返回浓缩摘要的专门 agent(第 3 章)。

Context firewall(上下文防火墙) — 子代理模式的一个性质:父代理永远看不到子代理的中间噪声,只接收其浓缩结果(第 3 章)。

Orchestrator-workers — 一种 workflow:中心 LLM 动态分解任务、委托给 worker LLM,并综合结果(第 6 章)。

Evaluator-optimizer — 一种 workflow:一个 LLM 生成、另一个 LLM 批评,循环往复直到满足评价标准(第 6 章)。

Micro-agent(小代理) — 嵌入在确定性 workflow 中的小而聚焦的 agent(约 3-20 步),而非开放式“loop until done”的 agent(第 6、8 章)。

Multi-agent topology(多代理拓扑) — 多代理系统的协调形态:orchestrator-worker、hierarchical、blackboard/shared-memory,或 debate/voting(第 3、6 章)。

MAST(多代理系统失败分类法) — 一套经验性的多代理失败分类,含 14 种失败模式,分三大类:规格问题、代理间错位、任务验证(第 3 章)。

Trust escalation(信任升级) — 一种委派失败:高权限 parent 接受低权限 worker 的说法,再用 worker 不曾拥有的权限采取行动(第 3、18 章)。

推理 / 自我纠错模式 — 单个 agent 用 token 换可靠性的思考模式:Reflexion(自我批评记忆)、Self-Refine(批评并修订)、CRITIC(工具落地的批评)、Tree of Thoughts 与 LATS(分支搜索)、ReWOO(先规划再执行)(第 6 章)。

工具与沙箱

Namespacing(命名空间) — 把相关工具放在共同前缀下(`asana_*`、`browser_*`),防止命名冲突并支持按组 masking(第 4 章)。

Action masking(动作掩码) — 在上下文中保持完整工具集稳定,同时根据当前状态约束哪些动作可被选择(第 2 章)。

Progressive disclosure(渐进披露) — 只在需要时加载工具定义、文件或指令,而不是一次性全部前置(第 4 章)。

Skill — 由文件支撑的可复用能力(通常是一个 `SKILL.md` 加上配套代码),agent 可按需加载(第 4 章)。

代码执行(作为元工具) — 把工具呈现为 agent 通过写代码来调用的代码 API,而不是直接调用——大幅降低 token 成本(第 4 章)。

Programmatic tool calling(程序化工具调用) — 让模型编写有界程序,在 runtime 内过滤、连接、排序、去重、聚合或验证工具结果,从而减少模型往返与上下文传输(第 4 章)。

Shell — Bash、zsh 这类命令行接口。在 agent 系统中,shell 访问很强大,因为它让 agent 可以运行测试、检查文件、安装包,并临时组合工具(第 1、4、5 章)。

文件系统(filesystem) — agent 可以读写的目录和文件。它既是工作区,也是持久记忆,还是 agent 与人类协作的界面(第 1、2、5 章)。

Sandbox(沙箱) — 带有文件系统和网络边界的隔离环境,agent 可在其中自由行动而无需逐动作审批(第 5 章)。

Sandbox liveness(沙箱活性) — 沙箱作为授权区域的作用:agent 可在配置边界内行动而无需逐动作审批(第 5 章)。

Governance(治理) — 管理身份、权限策略、scoped credentials、人类审批、审计日志和跨层安全问责的 harness 机制(第 5、18 章)。

Agentic readiness — 一个系统是否适合自治调用者安全理解和操作的程度:幂等 mutation、明确 operation state、machine identity、清楚的 retry 语义、可观察 outcome 与补偿操作(第 5 章)。

Delegated auth(委托授权) — Agent 通过 scoped credentials 或 proxy-authorized identity 行动,而不是继承用户完整环境权限的模式(第 5 章)。

Supply-chain provenance(供应链来源证据) — 关于 agent 所依赖的 tools、packages、datasets、MCP servers 和 retrieval sources 的来源与完整性证据(第 5、18 章)。

Hook / middleware(中间件) — 由 harness 在生命周期事件(启动、工具调用后、停止)自动执行的脚本或检查点,确定性地强制规则(第 5 章)。

Feedforward / feedback(前馈/反馈) — 前馈控制(guides)在 agent 行动前引导它;反馈控制(sensors)在它行动后观察并帮助自我修正(第 5 章)。

Computational / inferential control(计算型/推断型控制) — 计算型控制(linter、类型检查器)确定且快;推断型控制(AI review、LLM-as-judge)能处理细微判断,但更慢、非确定(第 5 章)。

Ambient affordances(环境可供性) — 环境本身的属性(强类型、清晰模块边界、有立场的框架),使代码库对 agent 更易理解和处理(第 5 章)。

CI(持续集成) — 围绕代码变更自动运行的检查,通常包括测试、linter、构建和部署关卡。在 harness 设计中,这类检查会成为反馈型 sensor(第 5、9 章)。

Linter / type checker(linter / 类型检查器) — 在运行前发现风格、语法、结构或类型错误的确定性工具。它们是外层 harness 中常见的计算型 sensor(第 5 章)。

Prompt injection(提示注入) — 一种攻击:藏在 agent 所读内容(网页、文件、工具结果)中的指令被模型当作命令执行(见《LLM Foundations》第 8、12 章)(第 5 章)。

Lethal trifecta(致命三要素) — 同一 agent 同时具备:访问私有数据、接触不可信内容、向外通信能力——这三者的危险组合(第 5 章)。

Circuit breaker(熔断器) — 一个可靠性包装:在失败达到阈值后跳闸,让后续对失败工具、服务或子代理的调用快速失败,而不是挂起或重试成风暴(第 5 章)。

Kill switch(终止开关) — 由人或策略触发的停止,立即且独立于 agent 自身控制流地终止一个 agent 或 fleet;它存在于 harness 中,因为被操纵的 agent 不能被指望停下自己(第 5 章)。

Canary token(金丝雀令牌) — 一份被种下的假机密(未使用的 key、诱饵文件、陷阱 URL),其被访问或外泄会触发高信号警报,表明 agent 已被操纵——对 lethal-trifecta 外泄路径的检测(第 5 章)。

Action budget(动作预算) — 对工具调用、token、墙钟时间或花费设的硬性上限,达到后循环停止并上报,而非失控奔跑(第 5、8、17 章)。

评估

Eval harness(评估 harness) — 端到端运行评估的基础设施;区别于被评估的 agent harness(第 10 章)。

Readiness validation(就绪验证) — 验证某个具体 model + harness 配置是否适合特定任务分布、环境、预算和治理规则(第 10 章)。

Failure attribution(失败归因) — 在选择修复方式前,先把 agent 失败标注到最可能的问题层:execution、tool interface、context、lifecycle、observability、verification 或 governance(第 10、12 章)。

Task / trial(任务/试验) — task 有定义好的输入和成功标准;trial 是对它的一次尝试(第 10 章)。

Grader — 为试验某个方面评分的组件:code-based、model-based 或 human(第 10 章)。

Evaluator integrity — 评测系统自身的可信度,由盲评、确定性证据、abstention、calibration、meta-eval、版本化与可复现 audit artifact 支撑(第 10 章)。

Transcript(trace、trajectory) — 一次试验的完整记录:每条消息、工具调用和结果(第 10、12 章)。

Outcome(结果状态) — 试验结束时的最终环境状态,区别于 agent 的文本回应(第 10 章)。

Capability eval / regression eval — capability eval 衡量 agent 新近能做什么(通过率低、正在爬升);regression eval 保护它已能可靠做到的事(接近 100%)(第 10 章)。

pass@k / pass^k — pass@k 是 k 次尝试中至少一次成功的概率(随 k 上升);pass^k 是 k 次试验全部成功的概率(随 k 下降)(见《LLM Foundations》第 13 章)(第 10 章)。

Infrastructure noise(基础设施噪声) — 由运行时资源配置(而非模型能力)造成的 benchmark 分数波动(第 11 章)。

长运行代理与领域

交接班问题(shift-change problem) — 由于上下文窗口有限,后续 agent session 到来时对之前的 session 毫无记忆这一挑战(第 7 章)。

Initializer agent — 只运行一次、为后续 coding agent session 搭好项目(init 脚本、进度日志、feature list)的 agent(第 7 章)。

Managed agent — 平台管理的 agent 架构,把模型侧 brain、执行侧 hands 和持久 session/event log 分开,使它们能独立失败、重置或迁移(第 7 章)。

Brain / hands split — Managed-agent 中决策上下文(brain)与可替换执行环境(hands)的分离(第 7 章)。

Sprint contract — generator 与 evaluator 两个 agent 之间基于文件的约定,在每个构建 sprint 前敲定要构建什么、如何验证成功(第 7 章)。

Event log(事件日志) — 对消息、工具调用、结果、审批和错误的追加式记录。执行状态可以从中推导出来,因此 agent 更容易重放和调试(第 9 章)。

Agent platform — 超出本地 framework 的基础设施:跨多次运行和多用户的 durable workspaces、managed sandboxes、identity、billing、observability、evaluation、governance 和 human handoff(第 9、18 章)。

Checkpoint / resume(检查点/恢复) — 一种可靠性模式:agent 定期保存足够状态,以便在失败或上下文重置后继续工作而不丢进度(第 7、9 章)。

Durable execution(持久化执行) — 一种基础设施保证:把每个工作流步骤持久化,使崩溃或被中断的 agent 从最后记录的一步恢复;非确定的模型/工具调用被记录并重放,而非重算(第 7 章)。

Time horizon(时间视野) — METR 的能力指标:模型以 50% 可靠性能完成的人类任务长度;前沿值大约每七个月翻一番(第 7、19 章)。

Stateless reducer(无状态归约器) — 把 agent 建模为对 event log 的纯 fold,使其可序列化、可重放、可测试(第 9 章)。

Model-harness co-evolution(模型与 harness 共同演化) — frontier 模型在其 harness 一起参与的情况下 post-train 所形成的耦合,因此改变任一侧都可能损害性能(第 12 章)。

Span telemetry — 以 span tree 表示的结构化 trace 数据,覆盖 model calls、tool calls、retrieval、context assembly、permissions、costs 和 outcomes(第 12 章)。

OpenTelemetry GenAI semantic conventions — 一套新兴的、面向 LLM 与 agent 遥测的标准 span 与属性名 schema(`invoke_agent`、`chat`、`execute_tool` span),让 agent trace 加入普通可观测栈(第 12 章)。

Trace-to-eval loop — 把真实生产失败转换成脱敏、可复现、带 outcome assertion 的 regression case(第 12 章)。

Controlled self-improvement(受控自我改进) — 一条外部、版本化回路:把经过审查的生产纠正变成有界 change task,用 eval gate、canary 与 rollback 发布,而不是允许 deployed agent 原地修改自己(第 12、17 章)。

Meta-harness — 把 harness 设计本身当作优化对象:用 eval feedback 消融或搜索 prompts、tools、retries、context policies、evaluators 和 control loops(第 12 章)。

Cost-quality-speed trilemma(成本-质量-速度三难) — 更强 execution environment、observability、verification 和 governance 会提高可靠性,但也增加成本和延迟(第 19 章)。

Capability-control tradeoff(能力-控制权衡) — 更多权限、工具、记忆和自治会提升能力,同时扩大控制、provenance 和审计问题(第 18、19 章)。

Ralph Wiggum loop — 一个 hook,拦截 agent 的退出尝试,并在干净的上下文窗口中重新注入原始 prompt,迫使它继续对照目标工作(第 7、8 章)。

Loop engineering(循环工程) — 把 agent loop 本身当作设计单元:规定环绕模型的 trigger、topology、verifier 和 stop rule,使它能无人值守地运行。这是外层控制循环的运维者视角(第 8 章)。

Trigger(触发器 / heartbeat) — 无需人类 prompt 就启动一趟 loop 的东西:一个 schedule、一个 webhook,或另一个 agent(第 8 章)。

Verifier(验证器 / maker-checker) — 决定“够好了”的固定标准,由一个不同于产出工作的 agent 来施加,使 maker 不能批改自己的作业;是 loop 设计的瓶颈(第 8 章)。

Stop rule(停止规则) — 结束一个 loop 的明确条件——success、no-op、ask-for-approval——外加兜住失控的三个硬停:最大迭代次数、无进展检测、预算上限(第 8 章)。

Closed vs. open loop(闭环与开环) — 闭环预先钉死硬的、可检查的验收标准,放着跑是安全的;开环朝模糊目标探索,需要一个更强的 verifier,否则会 ship 出自信的垃圾(第 8 章)。

指令与模型选择

Instruction hierarchy(指令层级) — 指令按来源带有不同权威——system 高于 developer 高于 user 高于工具/检索内容——使低优先级指令无法覆盖高优先级指令。Prompt injection 就是这一层级的失效(第 13 章)。

Right altitude(合适的高度) — System prompt 的目标具体程度:具体到能可靠引导行为,一般到能跨情况迁移,既不论为脆弱的硬编码规则,也不流于含糊指引(第 13 章)。

Model routing(模型路由) — 给请求的难度分类,把简单的派给便宜的弱模型、把困难的派给昂贵的强模型。只有当路由决策远比它带来的节省更便宜时才划算(第 14 章)。

LLM cascade(级联) — 先试便宜模型,只在 verifier 否决便宜答案时才升级到更强的模型。升级信号可靠时,能以更低成本匹配强模型准确率(第 14 章)。

Fallback(回退) — 当主模型出错、超时或被限流时切换到备用模型,使 agent 优雅降级(第 14 章)。

AI gateway(AI 网关) — 位于 harness 与各模型供应商之间的基础设施组件,对外呈现一个统一接口覆盖多个模型,并承载路由、fallback、预算、缓存和日志(如 LiteLLM、Portkey)(第 14、17 章)。

Reasoning model(推理模型) — 经过 post-training(通常是在可验证奖励上做 RL),学会在回答前生成很长内部推理、用 inference token 换困难任务上更好表现的模型(第 14 章;LLM Foundations 第 7–8 章)。

Test-time compute — 在回答时花更多 inference token、时间和金钱以在困难问题上做得更好——区别于更大模型或更多硬件的一条 scaling 轴(第 14 章)。

人类交互

Permission fatigue(许可疲劳) — 当 agent 过于频繁请求批准时监督的退化,把人训练成不读就盖橡皮图章(第 5、14 章)。

Mixed-initiative(混合主动) — 一种交互风格:系统逐动作决定是自主行动还是让步给人,并管理打断的代价(第 15 章)。

Approval as a tool call(批准即工具调用) — 把人类批准建模为 agent 调用的一个工具,使请求成为持久、可重放、可审计、并与挂起/恢复组合的事件(第 15 章)。

Steering(引导) — 把一条新指令注入正在运行的 agent,使其在下一回合被纳入,从而在不丢 session 状态的前提下重定向(第 15 章)。

Calibrated trust(校准过的信任) — 人接口的目标:人对 agent 的信任恰好等于它在给定任务上配得到的程度,通过透明和扎根于验证的不确定性、而非流畅度来实现(第 15 章)。

Computer-Use Agent

Computer-use agent — 通过 GUI 操作软件的 agent——查看截图并发出光标、键盘和导航动作——而不是调用定义好的 API(第 16 章)。

Visual grounding(视觉接地) — 把意图(“点击 Submit”)翻译成具体动作(在特定坐标点击);一种在 API 工具里没有对应物的错误模式(第 16 章)。

Set-of-Mark prompting — 在候选可交互元素上叠加编号标记,让模型选择离散标签而不是产出裸坐标,提升 grounding 可靠性(第 16 章)。

Accessibility tree(可达性树) — UI 的结构化语义表示(role、label、state),为辅助技术构建;常比裸像素或 DOM 更紧凑、更精确的屏幕编码(第 16 章)。

成本与运维

AgentOps — 跨 governance and security、build and operations、evaluation 与 observability 运行 agent 完整生命周期的纪律,覆盖 planning 到 retirement(第 17 章)。

Per-task budget(每任务预算) — 对单次 agent 运行的 token、工具调用或成本设的明确上限,超过后 agent 停下来问,而不是无限循环(第 17 章)。

Cost attribution(成本归因) — 给 trace 的每个 span 附上 token 和美元成本,把“agent 很贵”变成一个具体、可修的工程发现(第 17 章)。

Multi-tenancy / 租户隔离 — 用一个平台服务许多用户或组织,同时防止状态串味(context/记忆/缓存跨租户泄露)和权限串味(用错误租户的凭据行事)(第 17 章)。

Canary rollout(金丝雀放量) — 把 harness 改动发布给一小部分流量,在全量部署前盯住生产 trace 和 outcome 指标,接住 eval 套件漏掉的案例(第 17 章)。

Semantic cache(语义缓存) — 一种缓存:通过嵌入查询、在相似度超过阈值时返回已存响应,来服务相似(而非仅相同)的请求;能省掉整次模型调用,但有错误命中的风险(第 17 章)。

AI 管理体系(ISO/IEC 42001) — 首个用于治理组织 AI 的可认证标准:如何建立、运行并持续改进一套 AI 管理体系——ISO 27001 的 AI 对应物(第 17 章)。

欧盟 AI 法案(EU AI Act) — Regulation (EU) 2024/1689,首部全面的 AI 法律;它按风险层级对系统分类,并对高风险用途施加有约束力的义务(第 17 章)。

Fleet、身份与控制平面

Agent fleet — 由 agent definition、version、run、sandbox、tool、identity 与 owner 组成的受治理集合,而不是一个本地配置的 agent(第 18 章)。

Agent control plane(agent 控制平面) — 跨 fleet 注册 agent artifact、管理 identity 与 delegated authority、强制 policy、reconcile lifecycle 并保存 lineage 与 audit 的共享基础设施;区别于真正做工作的 data plane(第 18 章)。

Agent identity — Agentic principal 的持久 machine identity,区别于 user、sponsor、runtime process、model 与单个 session(第 18 章)。

Agent registry — Agent 与 capability definition、version、ownership、dependency、requested permission、attestation、deployment state 与 lineage 的被治理 source of truth(第 18 章)。

Delegated authorization(委派授权) — 从 user 或 service 派生短期、audience-bound、purpose-bound 权限给 agent;后续每次 delegation 只能收窄 grant(第 5、18 章)。

Agent gateway — Data-plane enforcement point:评估 model、tool、MCP、A2A、egress 或 memory request 的 identity 与 execution envelope,并能 allow、deny、redact、缩小 scope、强化 isolation 或要求 approval(第 18 章)。

Lineage(血缘/溯源链) — 把一次 agent action 连接到 publisher、source 与 build、configuration、identity 与 delegation chain、policy decision、trace evidence、approval 与 environment outcome 的因果图(第 18 章)。

Non-repudiation(不可否认性) — 用 signed 或 integrity-protected event、timestamp 与 immutable version,提供足以把 request 与 authorization 归因到特定 workload 与 policy identity 的证据;它不能证明人理解了 approval,也不能证明模型 rationale 真实(第 18 章)。